THE ADL DESKTOP APPS GUIDELINES · VERSION 1.4.1 · 2026-11-06

Pauta

UI, UX and platform rules for adl's JavaFX desktop apps. Editora is the reference implementation; every rule names the class or file to copy from.

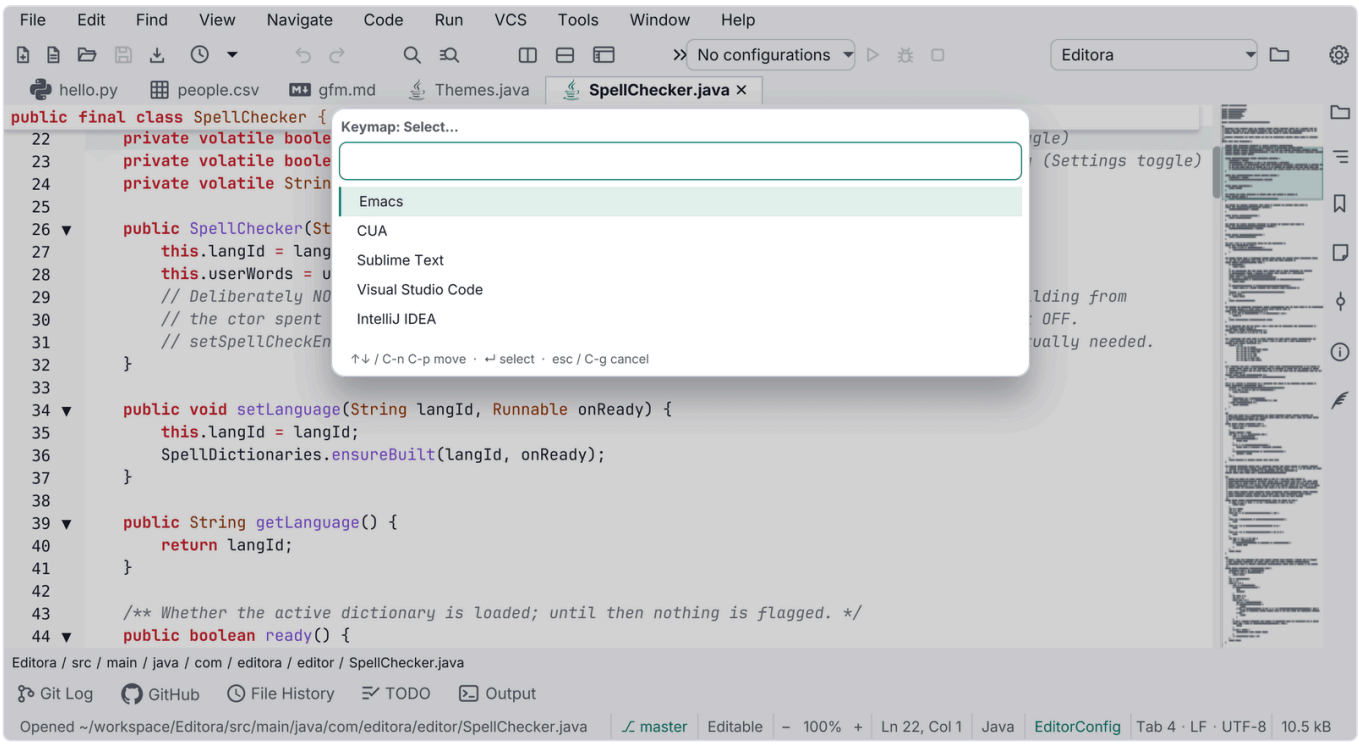
Editora · reference | Torrenta | Termina | Modula | Mirada | nux

MUST release blocker for a new app SHOULD expected unless the app says why not MAY copy when the app grows into it

CONTENTS

Principles
New app checklist
Platform baseline
Repository documents
Documentation
Code quality gates
Performance
Keyboard first
Window shell
Icons and color
Toolbars and tooltips
Tabs and tool panels
Menus
Tray and background
Settings
Diagnostics and support
Command line and launch
Internationalization
Command Palette
QuickOpen picker
Welcome page
About window
Confirm dialog
Find bar
Message Log
Settings Sync
Packaging, signing and release
Feature matrix
Shared features
Gaps per app

Principles



Editora's keymap picker: five keymaps bind keys to the same commands.

These come from what the six apps already do. The first is the reason the family and this guide exist: one app that is the same on Windows, macOS and Linux. The other twelve are how the apps get there and stay there. Editora is the reference; each principle names where else it is written down.

For the people using the apps

1. **Same app on every OS.** A user who moves between Windows, macOS and Linux finds the same features, the same commands and keymaps, the same look and the same installer quality, and never relearns the app. Platform differences live behind one class, never scattered `if (isMac)` checks. Every other principle serves this one.
2. **Keyboard first.** Every action works without a mouse. The mouse is a shortcut to commands, not the other way round.
3. **Every feature is a command.** One registry of commands with ids. Menus, toolbar buttons, the command palette and key bindings all run commands by id. Nothing is wired straight to logic.
4. **Speed comes first.** Start-up and the app's core action (typing in Editora, stepping to the next image in Mirada, a keystroke reaching the shell in Termina) are the product. When a feature pulls against them, the feature loses or waits until it can be made cheap. *(Mirada's "Priorities"; "fast first" in Termina and Mirada.)*
5. **The UI never waits.** The JavaFX application thread only touches the scene graph. Disk, network, processes, parsing and searching run on other threads and hand their result back; a keystroke, a scroll or a window opening never waits for them. See Performance.
6. **Changes apply as they are made.** Settings have no Save button. A change reaches every open window.
7. **Nothing fails silently.** Every failure reaches the status bar, the Message Log or the Debug Log, and the user can turn any of it into a GitHub issue.
8. **Never claim what is not true.** No link that looks like a link and does nothing, no "up to date" before an update check has answered, no setting that seems to apply when it cannot: a setting that waits for a restart says so on its own row. *(Modula's About, Editora's update status, Termina's settings rows.)*
9. **Local first, nothing sent without asking.** No telemetry. The only automatic network call is a once-a-day release check the user can turn off. Problem reports are built locally and submitted by the user.
10. **Users' data outlives versions.** Config files carry a schema version and migrate forward. An older build never overwrites a newer config.

For the people building them

11. **One source of truth for each fact.** The version lives in `pom.xml` and reaches About, `--version`, the update check and problem reports through a filtered `build-info.properties`. A key binding is declared once, so the menu and the accelerator cannot disagree. A list that repeats a fact (About's dependencies, the coverage table in the docs) has a test that fails when the two drift, in both directions. *(Modula's `DependenciesTest`, nux and Torrenta's `AppInfo`, Editora's `BuildHygieneTest`.)*
12. **Separate the decision from the UI, then test it.** Anything with a rule in it (which key does what, sort order, when a bar shows, whether a color is legible) is a small pure method or class with its own unit tests. UI classes stay thin and only wire decisions to controls. *(Termina, Torrenta, Mirada and nux "Conventions"; Editora's pure decision helpers.)* See Code quality gates.
13. **Verify by driving the real app.** UI bugs are visual and none of them throw. Before calling a UI change done, run the real window, press the real keys and look at the result, through a capture hook or a headless screenshot; reasoning about event order is not evidence. *(Termina and Torrenta's `DevCapture`, nux's `--screenshot`, the headless harness behind Editora's website screenshots.)*

New app checklist

Each group links to the page with the rules and the Editora code to copy. Everything here is a `MUST` unless it says `SHOULD`.

Start from the template. adl-desktop-app-template is a GitHub template repository that already ticks every `MUST` on this list: a small runnable app with its tests, CI and release workflows and every repository document. Click **Use this template**, then run `scripts/rename.sh <AppName> <package>`.

Platform

Platform baseline

- [ ] JDK 25 or newer (`release` 25 is the floor) and JavaFX 27 or newer
- [ ] One JPMS module, `com.<app>`
- [ ] Maven Wrapper committed (`mvnw` , `mvnw.cmd` , `.mvn/wrapper/`)
- [ ] AtlantaFX light and dark themes, following the system by default
- [ ] Inter bundled and attached to every scene and dialog `SHOULD`

Code quality

Code quality gates

- [ ] The version comes only from `pom.xml` (filtered `build-info.properties`); lists that repeat a fact have a drift test
- [ ] Rules live in pure, unit-tested helpers; UI classes stay thin
- [ ] A capture hook or headless screenshot drives the real window for UI checks
- [ ] Every package `@NullMarked` ; NullAway on Error Prone fails the build
- [ ] Spotless with Palantir Java Format
- [ ] JUnit 5 and headless FX tests (`-Dglass.platform=Headless`), FX tests tagged `fx`
- [ ] JaCoCo check at 0.90 lines and 0.90 methods

Performance

Performance

- [ ] No I/O, process, parsing or waiting on the JavaFX application thread; results return through `Platform.runLater`
- [ ] Background results applied under a generation guard; named daemon threads
- [ ] Redraws coalesced to one per pulse; streams delivered in batches
- [ ] Logs, histories and caches bounded; large inputs guarded
- [ ] Only the first frame built before it paints; update check and extras deferred

Keyboard and commands

Keyboard first

- [ ] One `CommandRegistry` : every action is a command with an id
- [ ] Command Palette as an in-scene overlay, not a `Popup`
- [ ] Emacs and CUA keymaps at least, with user overrides on top
- [ ] One QuickOpen-style picker for every list choice (themes, keymaps, recent items, a setting's values), in the overlay host `SHOULD`
- [ ] A find bar in the scene wherever there is text to search: searches as you type with a count, Enter and Shift+Enter step, Escape closes it and returns focus (`FindReplaceBar` , Termina's `SearchBar`)

Window shell

Window shell · Welcome page · Confirm dialog

- [ ] A Welcome page when there is nothing to restore, or an empty state that names the ways in with their chords `SHOULD`
- [ ] Every destructive action asks first through one helper: the question names the thing, the button names the action in the `danger` style, Escape cancels; or the action is undoable instead

Icons and color

Icons and color

- [ ] Inter attached to every scene by a scene stylesheet; state colors named once on `.root` from the theme's roles, no fixed hex in a component rule

- [ ] Readable text in `-color-fg-muted` or stronger, `-color-fg-emphasis` on an emphasis fill; a pure contrast test over the stylesheets `SHOULD`
- [ ] Icons as themed SVG paths in one `Icons` class, one style class per family, the same glyph for an action everywhere it is reached

Toolbars and tooltips

Toolbars and tooltips

- [ ] Icon-only buttons built by one helper that sets the tooltip and the accessible name (`Icons.button` , `Icons.name`)
- [ ] Toolbar buttons out of the Tab order; every tooltip names the command and its chord from the active keymap
- [ ] Groups separated by spacing on one chrome band, no hairlines; a toggle wears `:open` while it is on `SHOULD`

Tabs and panels

Tabs and tool panels

- [ ] One tab header carries the states: `•` and amber italic for unsaved, a pin and bold for pinned, muted italic for read-only (`TabHeaders`)
- [ ] Bulk close skips pinned tabs, and Reopen Closed Tab brings the last one back with its caret (`ClosedTabs`)
- [ ] Panels toggled by commands, with side, order and divider positions restored on the next launch (`ToolWindowManager`) `SHOULD`

Menus

Menus

- [ ] Menu rows are commands: icon, title and chord from the registry and the active keymap, no `KeyCombination` accelerators
- [ ] Right-click menus close on a press anywhere outside them and never stack (`ContextMenuDismissal` installed at startup)
- [ ] Right-click menus open from the Menu key and Shift+F10 at the selected row (`RowContextMenu`)

Tray and background

Tray and background (for an app that keeps working without its window)

- [ ] The shared `tray/` package: StatusNotifierItem on Linux, AWT elsewhere, created off the FX thread, and the app fully usable without one
- [ ] Closing the window hides it only while an icon is up and the setting says so; Quit is explicit and one path
- [ ] Start at login writes the OS entry (XDG autostart, LaunchAgent, `HKCU` Run) with no setting beside it, `--minimized` behind the icon `SHOULD`
- [ ] Desktop notifications through one `Notifier` , off the FX thread, one switch per event

Settings

Settings

- [ ] Settings window with a page sidebar, search and live apply
- [ ] A palette command for every setting
- [ ] Versioned config in `~/.<app>` , with migrations and downgrade backups
- [ ] Export and import of the config folder
- [ ] Settings sync through a Git/GitHub repository `SHOULD`

Diagnostics

Diagnostics and support

- [ ] Status-bar echo and Message Log
- [ ] Debug Log with a session file, and an uncaught exception handler
- [ ] Daily update check with an off switch; About shows versions and update status
- [ ] Report a Problem (redacted, prefilled GitHub issue) and `--report`

Command line and languages

Command line · Internationalization

- [ ] `--help` , `--version` , `--config-dir` , `--dev`
- [ ] Single instance per config folder
- [ ] At least six message catalogs, with a parity test

Repository

Repository documents

- [ ] `README.md` with the badge row and a light/dark hero screenshot
- [ ] `CHANGELOG.md` in Keep a Changelog format
- [ ] MIT `LICENSE`, and `NOTICE` with every third-party license
- [ ] `AGENTS.md`, with a one-line `CLAUDE.md` pointing to it
- [ ] `CONTRIBUTING.md`, `docs/` and `branding/`
- [ ] `.github/ISSUE_TEMPLATE/bug_report.yml`

Documentation

Documentation

- [ ] User docs on the app's website under `/docs/`: Getting started, a page per feature area, the keyboard reference, Settings and an FAQ
- [ ] Help → Documentation as a command (`help.documentation`), first in the Help menu after the Command Palette, and on the Welcome page
- [ ] A one-line `command.<id>.desc` for every command, held by the parity test
- [ ] `docs/README.md` indexes the contributor guides; `AGENTS.md` and `CONTRIBUTING.md` point into it
- [ ] Docs change in the same PR as the code; when they disagree with the code, the code wins and the docs are fixed
- [ ] Deep links from the app open the running version's docs, `/docs/v<version>/` `SHOULD`
- [ ] Command and keybinding references generated from the source, with a check that quoted ids and chords exist `SHOULD`

Packaging and signing

Packaging, signing and release

- [ ] jpackage installers through JReleaser: Linux x64 and arm64, macOS arm64 and x64, Windows x64
- [ ] macOS `.dmg` signed with Developer ID, notarized and stapled
- [ ] Linux downloads with `.asc` signatures from the shared release key
- [ ] Build provenance attestations for every download
- [ ] Windows `.msi` Authenticode, once the certificate is chosen `SHOULD`
- [ ] Homebrew cask in `adriandeleon/homebrew-tap` and a winget package `AdrianDeLeon.<App>` `SHOULD`

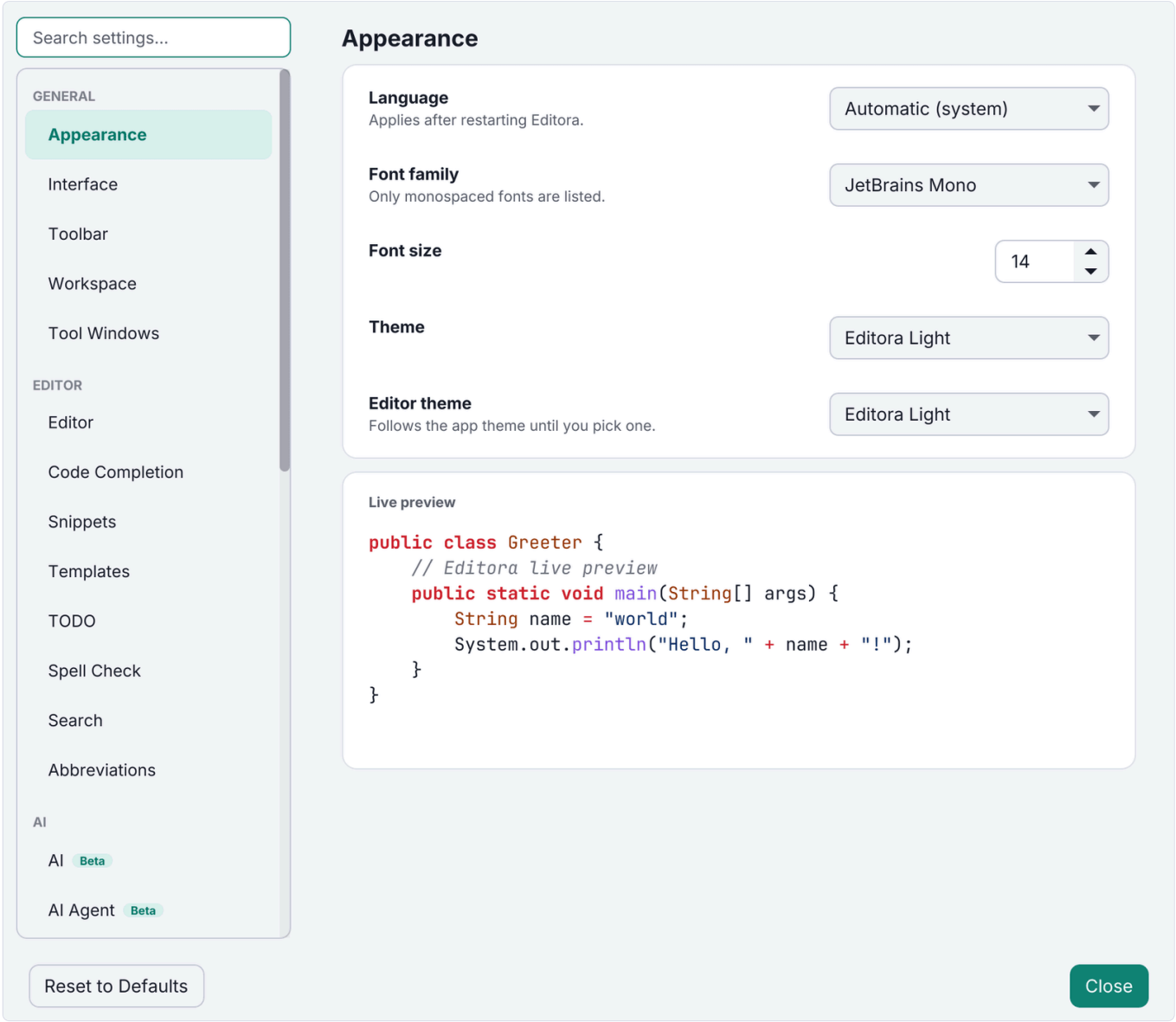
CI/CD (GitHub Actions)

CI/CD

- [ ] `ci.yml` on every push and pull request: Format and Build & Test, both required checks for `master`
- [ ] `full-tests.yml` on Linux, macOS and Windows: daily, on a `full-ci` label, before every release `SHOULD`
- [ ] `release.yml` on a `v*` tag that matches the pom, with a `workflow_dispatch` dry run
- [ ] Every third-party action pinned to a commit SHA
- [ ] Dependabot for `github-actions` and `maven`
- [ ] `permissions: contents: read` by default; write only in publishing jobs
- [ ] `timeout-minutes` on every job; superseded pull-request runs canceled
- [ ] Test and coverage reports uploaded; coverage totals in the job summary
- [ ] Signing skipped without secrets, but a tag without them fails
- [ ] A smoke launch of the packaged app on each OS before release

Platform baseline

Area	Rule	Level	Editora reference
JDK	**JDK 25 or newer.** `maven.compiler.release` 25 is the floor, not the ceiling: build and bundle a current JDK (the enforcer range is `[25,)`, open-ended). The bundled runtime is jlinked; users never install Java	MUST	`pom.xml`
JavaFX	**JavaFX 27 or newer** (`javafx.version` 27 is the floor)	MUST	`pom.xml`
Modules	One named JPMS module (`com.<app>`); `module-info.java` opens only what reflection, FXML and Jackson need	SHOULD	`src/main/java/module-info.java`
Theme toolkit	**AtlantaFX 3.x** (`atlantafx-base`) for controls; light and dark themes; app CSS uses AtlantaFX `-color-*` tokens, never hex literals	MUST	`ui/Themes.java` , `app.css`
UI font	Bundle **Inter** (Regular, Bold, Italic, BoldItalic + `OFL.txt`); register with `Font.loadFont` before any stylesheet; attach `ui-font.css` to every scene and dialog pane. Gives the family one look on every OS	SHOULD	`ui/Fonts.java`
Build	**Maven** with the **Maven Wrapper** (`mvnw` , `mvnw.cmd`) committed, so `./mvnw` works on a clean machine	MUST	`mvnw` , `.mvn/wrapper/`
Null safety	**JSpecify** annotations, checked by **NullAway** on **Error Prone** at compile time. Every package is `@NullMarked` in its own `package-info.java`	MUST	`docs/conventions.md#null-annotations-jspecify-checked-by-nullaway`
Formatting	**Spotless** with Palantir Java Format; CI fails an unformatted change	MUST	`pom.xml` (spotless plugin)
Tests	**JUnit 5** + **TestFX** over JavaFX's built-in **Headless** glass platform (`-Dglass.platform=Headless`), so FX tests need no display, no xvfb and no Monocle	MUST	`docs/testing.md` , `FxTestSupport`
Coverage	**JaCoCo** `check` at `mvn verify` : **≥ 0.90 lines, ≥ 0.90 methods**, ≥ 0.79 branches, plus per-package floors	MUST	`pom.xml` `jacoco-check`
Logging	`java.util.logging` , captured in-app by the Debug Log	SHOULD	`ui/DebugLog.java`
JSON config	Jackson POJOs with a `SCHEMA_VERSION` each	SHOULD	`config/Settings` , `config/migration/ConfigSchema.java`



Settings → Appearance in Editora: language, fonts, app theme and editor theme, with a live preview.

Everyday commands (identical in every repo)

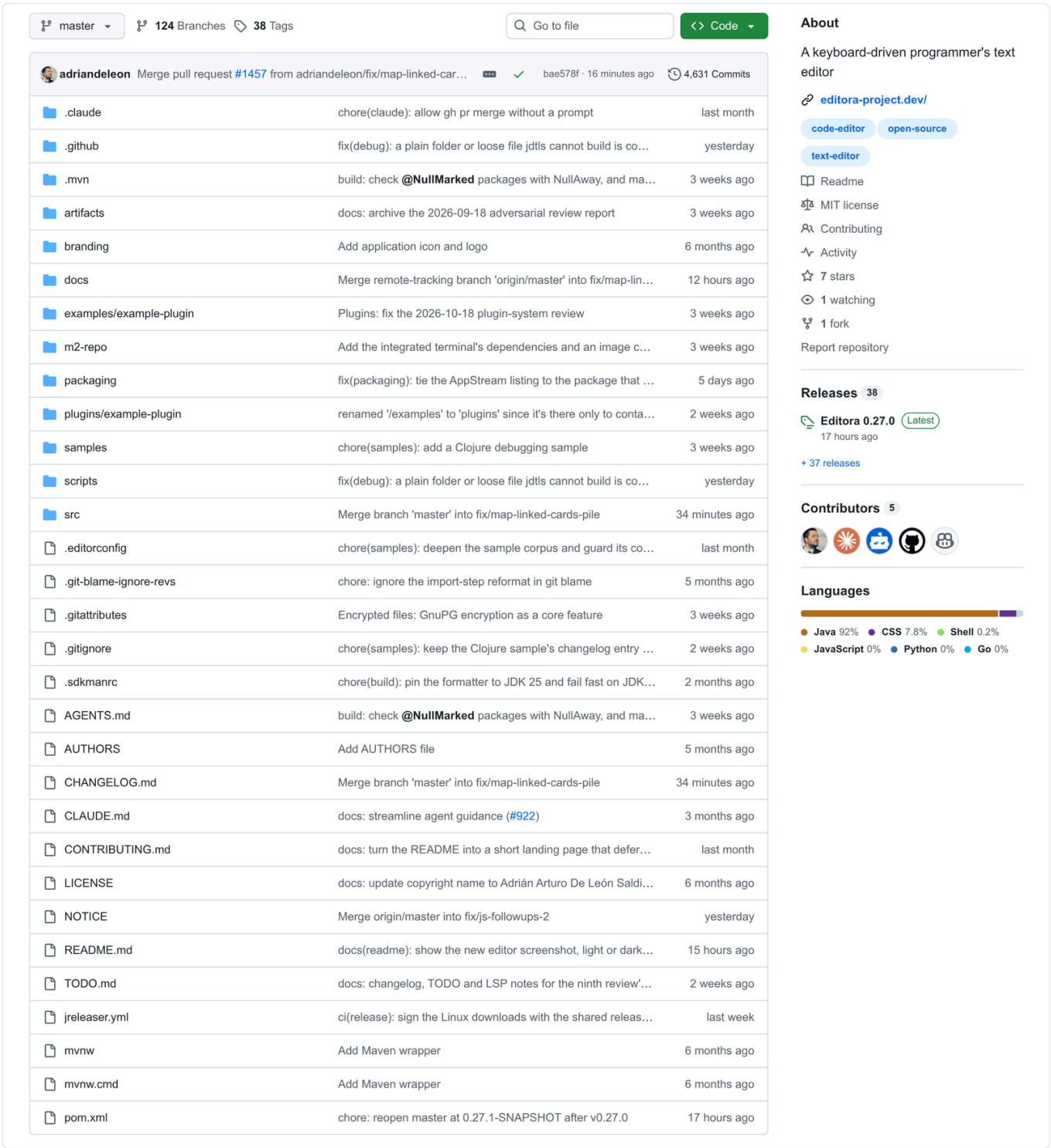
```
./mvnw javafx:run                # run the app
./mvnw test                      # unit + headless FX tests, writes the JaCoCo report
./mvnw spotless:apply            # format
./mvnw verify                    # full gate: tests, coverage check, NullAway
./mvnw -Pfatjar package          # runnable fat jar for this OS
./mvnw clean -Pdist package      # native installer for this OS (clean is mandatory)
```

The `clean` in every `-Pdist` build is mandatory: incremental compilation can leave synthetic enum-switch classes out of `target/classes`, and jlink then ships an app whose keyboard fails on the first keypress.

Repository documents

Start from the template. adl-desktop-app-template carries an example of every file on this page, filled in for a placeholder app, `ExampleApp`. `scripts/rename.sh <AppName> <package>` renames it to yours.

File	What it holds	Level
`README.md`	A landing page, about 200 lines: logo, badges, one-paragraph pitch, hero screenshot, highlights, install, build, command line, config folder, license. Detail lives on the website.	`MUST`
`CHANGELOG.md`	Keep a Changelog 1.1 + SemVer. An `## [Unreleased]` section with Added / Changed / Fixed / Removed; each entry opens with a bold one-line summary written for users.	`MUST`
`LICENSE`	MIT, "Copyright (c) 2026 Adrián Arturo De León Saldivar".	`MUST`
`NOTICE`	App name, copyright, "licensed under the MIT License", then every bundled third-party component with its license (Inter is OFL, AtlantaFX is MIT, OpenJFX is GPLv2+CPE...). Ship it inside the installers too.	`MUST`
`AGENTS.md` (+ a one-line `CLAUDE.md` pointing to it)	Operational rules for coding agents: essential commands, worktree-per-task, change checklist. Not a changelog.	`MUST`
`CONTRIBUTING.md`	How to build, test, format, and open a PR.	`MUST`
`docs/`	`architecture.md`, `conventions.md`, `testing.md`, `gotchas.md`, `release.md`, `building-and-packaging.md`, `decisions/` (ADRs).	`MUST`
`TODO.md`	The open list, kept in the same PR as the change.	`MAY`
`.github/ISSUE_TEMPLATE/bug_report.yml`	Issue form whose field ids (`environment`, `report`) the in-app Report a Problem fills.	`MUST`
`branding/`	Source SVG of the icon; generated PNG/ICNS/ICO live in `packaging/`.	`MUST`



Editora's repository root: every file on this page, with the homepage, Readme, MIT license and Contributing links GitHub derives from them.

README badges (in this order)

```
[![CI](https://github.com/adriandeleon/<App>/actions/workflows/ci.yml/badge.svg)](…/actions/workflows/ci.yml)
[![Release](https://img.shields.io/github/v/release/adriandeleon/<App>)](…/releases/latest)
[![License: MIT](https://img.shields.io/github/license/adriandeleon/<App>)](LICENSE)
![Java](https://img.shields.io/badge/Java-25%2B-orange?logo=openjdk&logoColor=white)
![JavaFX](https://img.shields.io/badge/JavaFX-27%2B-1e90ff)
![Platforms](https://img.shields.io/badge/platform-Windows%20%7C%20macOS%20%7C%20Linux-lightgrey)
[![Stars](https://img.shields.io/github/stars/adriandeleon/<App>?style=flat)](…/stargazers)
```

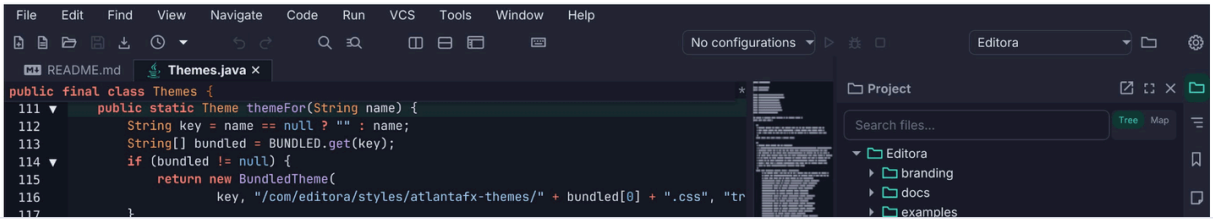
README Contributing MIT license

Editora

CI passing release v0.27.0 license MIT Java 25 JavaFX 27 platform Windows | macOS | Linux stars 7

A keyboard-driven, cross-platform programmer's text editor built with **JDK 25**, **JavaFX 27**, **RichTextFX** and **Maven**. Every action is a registered command, reachable from an Emacs-style keymap or a fuzzy command palette.

🌐 **editora-project.dev** — features, user docs, screenshots, every command, keybindings, plugins, what's new, roadmap and the blog. The site's source is adriandeleon/editora-website.



The top of Editora's README: logo, badges in this order, a two-sentence pitch, links to the website, then the hero screenshot.

The hero screenshot is a `<picture>` with a dark `srcset` under `prefers-color-scheme: dark`, so GitHub shows the screenshot that matches the reader's theme. Screenshots are taken at 2× by a headless harness so they can be retaken after any UI change. The README states that the app is built with the help of AI coding tools.

Documentation

An app ships with its documentation. There are three readers and three places: the person using the app reads the **website's docs**; the person changing the app reads `docs/`; a coding agent reads `AGENTS.md`. The README is the landing page that points to each. None of it is optional, and none of it is written after the fact: a change that affects what a reader would learn updates the documentation in the same pull request.

Reference: Editora's `AppInfo.docsUrl()`, `WindowCommandRegistrar` (`help.documentation`), `MenuBarModel` (the Help menu), `WelcomePane` (the Help row), `CommandPalette.docsUrl` and `SettingsFrameBits.helpUrl` (deep links); the repository's `docs/README.md`, `docs/conventions.md#docs-in-the-same-pr` and `CONTRIBUTING.md`; and the editora-website repository's `src/content/docs/` and `scripts/`.

User documentation

Rule	Level	Reference
Every app has user documentation. A *Getting started* page (install, first launch, the first thing to do), one page per feature area, the keyboard reference, the Settings, and an FAQ. Written for the person using the app, in their words, with screenshots from the real window. The README's 200 lines are not it.	`MUST`	editora-website `src/content/docs/` (43 pages in nine categories, from *Getting started* to *Help*); torrenta-website (19), termina-website (16), mirada-website (11)
It lives on the app's website, under `/docs/`, in the app's `<app>-website` repository, beside the feature pages, the screenshots, the news posts and the blog. Each docs page is a Markdown file whose frontmatter gives its `category` and `order`; the sidebar, the sitemap and the RSS feed follow. The app's repository never carries a second copy.	`MUST`	editora-website `README.md` (*Content*); Editora's `AGENTS.md`: "User-facing feature documentation goes to the separate website repository"
Deep links from the app open the running version's docs. Pages live under `/docs/v-<version>/…`; bare `/docs` and every unversioned slug redirect to the current version, so a link in a page never carries a version and a bump just regenerates `src/lib/version.ts`. The app builds the base from its own version with `-SNAPSHOT` dropped, so a development build points at the release it is working toward instead of a path that would always 404. A release that pointed at another release's docs would claim what is not true (*Never claim what is not true*), so this becomes a `MUST` the day the app links to a specific page.	`SHOULD`	`AppInfo.docsUrl()`, `releaseVersion()`; editora-website `astro.config.mjs`, `README.md` (*Versioned docs*). The other three sites carry a comment to port these routes when their app first deep-links
Generated where the app is the source. The Commands and Keybindings pages are generated from the app's `MainController`, `messages.properties` and keymaps; the Roadmap and What's new from `TODO.md` and `CHANGELOG.md`; the version from `pom.xml`. The output is committed, so the site builds without the app checked out. A check fails the site when a docs page quotes a command id that does not exist or a chord that is bound to a different command. One fact, one source (*One source of truth for each fact*).	`SHOULD`	`scripts/GenCommands.java`, `GenRoadmap.java`, `GenFeatures.java`; `scripts/check-doc-commands.mjs` (`npm run check:docs`)
Release notes a user can read. Each `CHANGELOG.md` entry opens with a bold one-line summary written for users (Repository documents); the GitHub release carries the section, the Welcome page's *What's New* opens this build's release, and the website's What's new is generated from the same file.	`MUST`	`AppInfo.releaseNotesUrl()`, `WelcomePane` (`welcome.whatsNew`); editora-website `GenRoadmap.java`

Reachable from the app

The documentation is one command away, so nobody has to know the website's address.

Rule	Level	Reference
Help → Documentation opens the user docs in the browser. It is a command (`help.documentation`), so it is in the palette, bindable and in the menu through the registry like every other row; it is the first row of the Help menu after the Command Palette, and the Welcome page's Help row repeats it.	`MUST`	`WindowCommandRegistrar` (`help.documentation` → `MainController.openDocumentation()`), `MenuBarModel` (`menubar.help`), `WelcomePane` (`welcome.documentation`)
The Help menu, in this order: Command Palette · Documentation · Check for Updates · Message Log · Debug Log · Report a Problem · About. Simple mode keeps Command Palette, Documentation and About. An app without a feature drops its row; nothing moves.	`SHOULD`	`MenuBarModel`, both `MenuSpec("menubar.help", …)`
A keyboard reference in Help: a searchable window of every key, grouped as the menus are, or the keybinding editor.	`SHOULD`	Termina's and Mirada's `ShortcutsWindow`; Editora's keybinding editor (Keyboard first)
A one-line description for every command (`command.<id>.desc`), shown under the title in the palette. It is the in-app documentation of the command and the parity test holds every command to it.	`MUST`	`messages.properties`, `MessagesTest` (Code quality gates)
Per-command help from the palette. With a command highlighted, `C-h` opens its page, `/docs/v-<version>/commands/<id>`; Search Everywhere shares the URL builder so the two pickers can never disagree.	`MAY`	`CommandPalette.docsUrl`, `setDocsOpener`
Help from a Settings page. Each page of the Settings window has a *Help for <page>* link to its docs page, from one map of page to slug.	`MAY`	`SettingsWindow`, `SettingsFrameBits.helpUrl`, `settings.help`

Strings

Key	English
`command.help.documentation`	Documentation
`command.help.documentation.desc`	Open the <App> documentation site for this version in your browser.
`welcome.documentation` / `welcome.whatsNew`	Documentation / What's New
`settings.help`	Help for {0}

Developer documentation

Rule	Level	Reference
`docs/` **with an index.** `docs/README.md` says who the folder is for (contributors; user docs live on the website), states that the code wins when the two disagree, and lists every guide with one line. The set is the one Repository documents names: `architecture.md`, `conventions.md`, `testing.md`, `gotchas.md`, `release.md`, `building-and-packaging.md` and `decisions/`.	`MUST`	Editora's `docs/README.md`; the template's `docs/`
A decision record for every choice a newcomer would question: a fork, a rejected dependency, a storage format, an escape hatch. One numbered Markdown file, `docs/decisions/NNNN-slug.md`, with the context, the decision and what it costs.	`SHOULD`	Editora's `docs/decisions/` (18 records, from `0001-richtextfx-fork.md`)
Three files, three jobs. `AGENTS.md` is the operational guide, kept short (commands, worktree-per-task, the change checklist). `CONTRIBUTING.md` is the short version for a person, pointing into `docs/`. `docs/` holds the durable explanations. Exhaustive history goes to `docs/reference/`, not into any of the three.	`MUST`	Editora's `AGENTS.md` (*Documentation map*), `CONTRIBUTING.md`, `docs/reference/README.md`
Javadoc sits on its member. A doc comment above another doc comment documents nothing; a test fails the build on one. Comments say why, not what.	`SHOULD`	`StackedJavadocTest`

Kept in step

Rule	Level	Reference
Docs in the same PR. A change that affects behavior updates `CHANGELOG.md`, `TODO.md`, the contributor guides it touches and the website's user docs in the same change, never as a follow-up. A reviewer treats a feature PR with no docs change as incomplete.	`MUST`	`docs/conventions.md#docs-in-the-same-pr`, `CONTRIBUTING.md` (step 4), `AGENTS.md`
When documentation and code disagree, the code wins, and the documentation is fixed in the same change. The same rule this guide applies to itself.	`MUST`	Editora's `AGENTS.md`, `docs/README.md`
Screenshots are retaken, not described. The capture hook renders the real window at 2× after any UI change, for the README (its hero in light and dark), the website and bug reports; a stale screenshot claims what is not true.	`SHOULD`	`DevCapture` (Shared features); Editora's headless harness
The README defers. About 200 lines: pitch, hero, highlights, install, build, command line, config folder, license, and links into the website's docs. It carries no feature catalog, so it changes only when those facts do.	`MUST`	Repository documents; `docs/conventions.md#docs-in-the-same-pr`

Code quality gates

- **NullAway is an error, not a warning.** Put `@Nullable` on what may be null and fix the report; do not suppress it. A new package starts with its own `package-info.java` carrying `@NullMarked`.
- **Coverage floors are a regression net, not a target.** The build-wide floor is 0.90 lines and methods. Each package of 300+ lines gets its own floor about three points below its measured level; when a package's coverage rises, ratchet its floor up. A test (`BuildHygieneTest`) fails when the documented floor table and the pom disagree.
- **Extract the decision, then test it** (principle 12). **Prefer a pure decision helper plus unit tests.** Use the headless FX harness only when controller or toolkit behavior is the point.
- **Tag FX tests** `@Tag("fx")`, so the pure suite runs alone with `-DexcludedGroups=fx`.
- **Guard tests that pay for themselves across the family:**
  - *Command sweep* runs every registered command in a real headless window (`CommandSweepFxTest`).
  - *Message parity*: every locale has every key; every `command.<id>` has a `.desc` (`MessagesTest`).
  - *Settings round trip*: every switch, number and choice survives a restart (`SettingsRoundTripFxTest`).
  - *Contrast*: text tokens keep their ratios in every theme (`StyleContrastTest`).
  - *Release supply chain*: signing and attestation steps stay where they are (`ReleaseSupplyChainTest`).
- **No sleeps in tests.** Own workers and FX callbacks with a scope (`AsyncTestScope`) and use latches.
- **A test window writes only inside its temp folders**, never in the real home directory. The command sweep checks it after running every command (`CommandSweepFxTest.leftInTheHomeDirectory`).
- **Verify UI work in the running app** (principle 13): a capture hook or headless screenshot of the real window, with real key and mouse events, before a UI change is called done.
- **A guard that cannot fail is decoration.** When you add a drift or parity test, break the thing it guards once and watch it fail. *(Modula's `DependenciesTest`.)*

Performance

A desktop app is judged by how it feels under the hand: a key that lags or a window that hangs while it reads a file is a defect, not a slow feature. These rules come from Editora's `docs/performance.md`, the contract every change on a hot path there must honor.

The UI thread

Rule	Level	Editora reference
Never block the JavaFX application thread. No file or network I/O, no process start or wait, no parsing, searching, hashing or JSON of any size on it. No `Thread.sleep`, `Future.get()`, `join()` or `Process.waitFor()` there.	`MUST`	`docs/performance.md` §1
Touch the scene graph only on the FX thread. Background work hands its result back with `Platform.runLater`; nothing else creates, changes or reads live nodes.	`MUST`	every service
Drop stale results. A background result is applied under a generation guard, so an answer to an old request never overwrites a newer one.	`MUST`	`GitService`, `SearchService`, `LatestOnly`
Name your threads and make them daemons, so a thread dump says what is running and a stray worker never keeps the app alive after its last window closes.	`SHOULD`	`daemon("my-feature")`
Leave the common ForkJoinPool alone for blocking work. It has one worker fewer than the machine has cores; a few blocked waits (`CompletableFuture.supplyAsync` of a `waitFor`) starve every other user of it. A `Process` subclass overrides `onExit()`.	`MUST`	`ExitWatch`, `docs/gotchas.md`

The canonical shape, used by every Editora service:

```
private final ExecutorService exec = Executors.newSingleThreadExecutor(daemon("my-feature"));
private final AtomicLong gen = new AtomicLong();

void request(String input, Consumer<Result> onResult) {
    long mine = gen.incrementAndGet();
    exec.submit(() → {
        Result r = computeOffThread(input);        // heavy work, off the FX thread
        if (mine == gen.get()) {                   // superseded? drop it
            Platform.runLater(() → {
                if (mine == gen.get()) onResult.accept(r);
            });
        }
    });
}
```

For one-off blocking calls (reading a file, stat-ing a path, starting a process), a virtual thread (`Thread.ofVirtual().start(…)`) is enough; the result still comes back through `Platform.runLater`.

Hot paths

Each app names its hot paths: what runs on every keystroke, scroll pulse or progress tick. In Editora they are typing, scrolling, highlighting, the overlays and the gutter. In Torrenta they are the transfer table's refresh, in Termina the terminal grid, in Mirada stepping between images.

Rule	Level	Editora reference
Debounce and coalesce. At most one redraw per pulse, with a `pending` flag; text-driven work waits for typing to pause. Streams (process output, logs, progress) are delivered in batches, at least 16 ms apart, never one `runLater` per line.	`MUST`	`FxThrottle`, `OutputPump`, `ConsoleAppender`
Work only on what is visible, and only on what changed. No O(document) or O(list) work per keystroke, scroll or tick; re-process the edited range or the visible rows. Use virtualized controls (`ListView`, `TableView`, a fixed cell size) for anything that can grow.	`MUST`	`HighlightPass`, `MinimapRows`
Allocate little on a hot path. Reuse buffers and arrays; cache data a repaint only reads.	`SHOULD`	minimap stripes
Keep CSS cheap. No `applyCss()` on a large subtree, stable style classes, and spans of the same style merged before they are applied.	`SHOULD`	`SpanMerger`, `docs/gotchas.md`

Memory and size

Rule	Level	Editora reference
Bound everything that grows: logs, histories, consoles, caches, undo. Keep the newest, keep a note of what was cut.	`MUST`	`MessageLog` (200), `DebugLog`, `OutputCap`

Rule	Level	Editora reference
Guard large inputs. Decide by size and shape before loading: turn off expensive features past a threshold, open read-only or capped past another, refuse what will not fit the free heap.	MUST	5 MB / 50 MB guards, `LoadHeapGuard`
Release what a closed window held: listeners, timers, executors, file watchers, native resources. A test opens and closes windows and checks nothing is left.	MUST	`FxWindowFixture.dispose`, `ClosedLspManagers`

Start-up

Rule	Level	Editora reference
The first frame first. Build only what the first window shows; tool windows, panels and pages build on first open.	MUST	lazy `ToolWindow` suppliers, `LazyToolWindowsFxTest`
Defer what nobody waits for: the update check, extra fonts, indexes and caches start after the first frame has painted.	MUST	`UpdateCheckScheduler`, `Fonts.loadRemainingAfterNextPaint`
Read config off the FX thread, before the toolkit needs it.	SHOULD	the `config-prefetch` thread
Ship precompiled stylesheets (`.bss`) and an AOT cache trained on a real session.	SHOULD	`css2bss`, `aot_build.java`; Termina's and Mirada's `aot-train.sh`

Measure, don't guess

- **Every change on a hot path says what it costs** in its pull request: added FX-thread work, allocation per keystroke or scroll, extra layout passes, memory. "Negligible" is an answer; no answer is not.
- **Measure start-up** with a script that takes the median of several cold launches (`scripts/measure-startup.sh` in Editora and Mirada) and phase marks in the app (`perf/Startup`).
- **Sample the FX thread** when something stalls (`EDITORA_PERF_SAMPLE=1`), and keep a headless test for each hot path that once regressed.
- A suspected regression gets a number before and after, from temporary `System.nanoTime` instrumentation if nothing better exists.

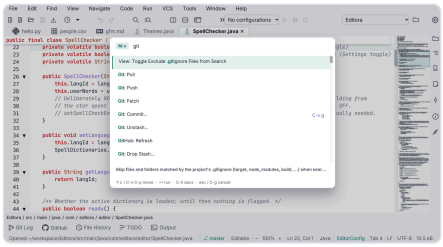
Keyboard first

Commands

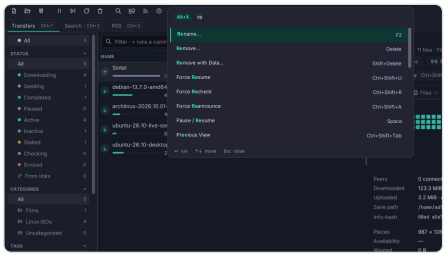
- Every user-facing action is a `Command` (id, title, runnable) registered in one `CommandRegistry`.
- Command ids are dotted and grouped by area: `file.open`, `view.toggleStatusBar`, `help.reportProblem`.
- Titles come from `command.<id>` and palette descriptions from `command.<id>.desc` in the message catalogs, so a registered command is localized and discoverable without extra work.
- A command that cannot run in the current context is shown grayed with a tooltip that says why, not hidden (`ui/Chrome.java`: `paletteEnabled` and `disabledReason`).

Command Palette

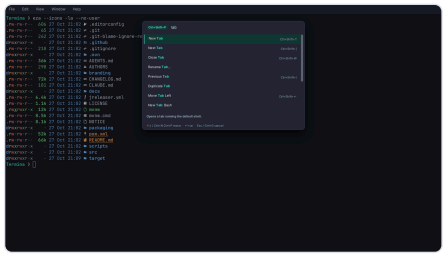
- `MUST` Opened by `M-x` in Emacs keymaps and `Ctrl/Cmd+Shift+P` in the others.
- Fuzzy matching over title, id and description; a query that names a command's title outranks a fuzzy hit. Each row shows the active key binding. See the Command Palette template.
- It is an **in-scene overlay** in the window's root `StackPane`, not a `javafx.stage.Popup`. On Windows a Popup does not reliably take OS keyboard focus and the whole app can stop receiving keystrokes.
- The same picker (`QuickOpen`) serves every other list choice: recent files, themes, keymaps, settings values.



Editora: M-x over the editor; typing git lists every Git command.



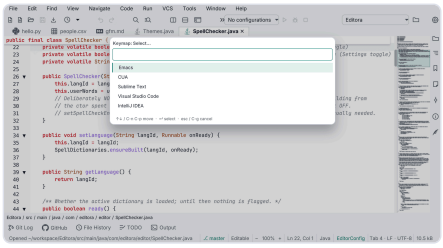
Torrenta: the same palette, filtered to torrent actions with their shortcuts.



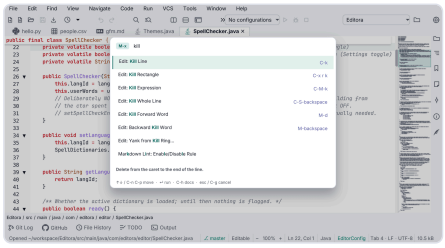
Termina: the palette over a shell session, filtered to tab commands.

Keymaps

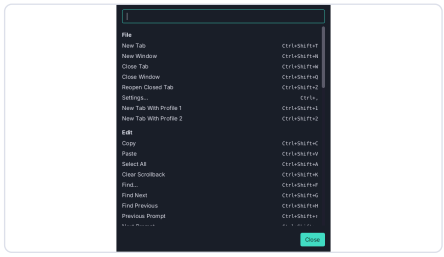
- Ship presets as JSON resources: Emacs, CUA, IntelliJ, VS Code and Sublime, with `.mac.json` variants where Cmd replaces Ctrl.
- Layer order: base keymap → user overrides → plugin keymaps → the user's own bindings again, so the user always has the last word (`KeymapLayers`).
- A keymap editor in Settings lists every command with its chord, searchable, with conflict warnings (`ui/SettingsKeymapsPage.java`; conflicts from `KeybindingEdits.takenDefaults`).
- Never bake a chord into translated text. Derive the label from the active keymap (`command/ChordFormat.java`, the one place a binding is rendered).
- Multi-key prefixes (`C-x …`) show a which-key style hint listing the possible next keys. Editora echoes only the pending prefix (`C-x -`) in the status bar (`KeyDispatcher`); no app lists the next keys yet.



Keymap: Select… offers Emacs, CUA, Sublime Text, Visual Studio Code and IntelliJ IDEA.



M-x kill: each command shows its chord from the active keymap.

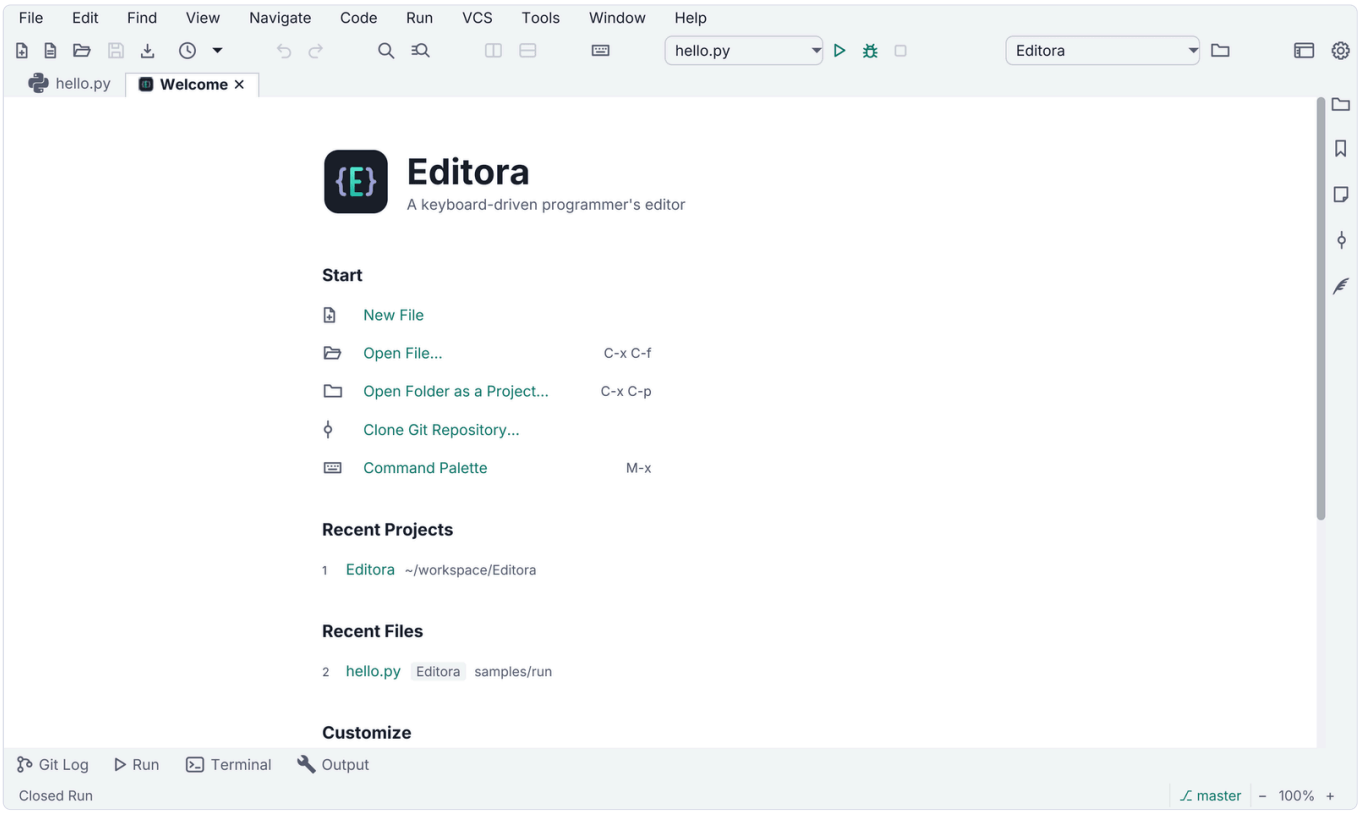


Termina's shortcuts reference: every key, grouped by menu.

Focus and accessibility

- Every interactive control is reachable with Tab and operable with Space/Enter. Checked in the running app; `FocusRingFxTest` holds that every control family draws the focus ring when Tab reaches it.
- Keyboard focus is drawn by one `:focus-visible` block at the end of `app.css`.
- Icon-only buttons are built by a helper (`Icons.button`) that sets both the tooltip and the accessible text.
- On/off rows use a keyboard-operable switch (`SettingSwitch`, `TOGGLE_BUTTON` role). A bare AtlantaFX `ToggleSwitch` is mouse-only.
- Readable text uses `-color-fg-muted` or stronger; `-color-fg-subtle` (about 3:1) is for disabled or decorative text only.
- Rows wrap instead of truncating to "…". Check German at 1024 px wide, by eye: no test catches a cut row.

Window shell

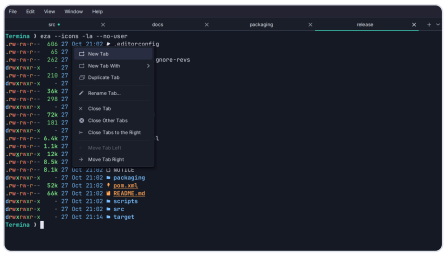


Editora's Welcome page: Start actions with their chords, Recent Projects, Recent Files and Customize.

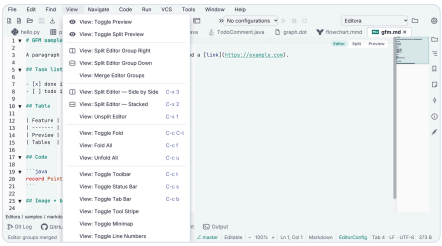
- **Status bar**, always present (toggleable). Its left side is the *echo area*: one line of feedback for the last action. Errors are red, warnings amber. Clicking it opens the Message Log. Editora's `ui/StatusBar.java` (`status-echo-error` , `status-echo-warn` ; a click runs `view.messageLog`) and the Message Log template.
- **Themes**: light and dark AtlantaFX themes plus a "follow the system" choice, switchable at run time without a restart. Semantic colors: amber = not durable yet (unsaved, read-only), red = broken, green = verified or added. Editora's `ui/Themes.java` (`applyAppTheme`) switches at run time and `ui/WindowColorScheme.java` tells every window's frame which scheme it is in; Mirada's `ui/Theme.java` (`ThemeChoice.SYSTEM`) follows the system's color scheme as it changes.
- **Title bar**: `SHOULD` use JavaFX 27's `HeaderBar` with `StageStyle.EXTENDED` to put the menu or tabs in the title bar, with a decorated-window fallback, and `useSystemMenuBar` on macOS. Editora's `ui/ExtendedWindow.java` .
- **Native dialogs** go through one helper (`Dialogs.styled`) because a dialog is its own scene and does not inherit the app stylesheet or the UI font.
- **A destructive action asks first, and says what it does.** Removing, deleting, discarding or overwriting something the user made goes through one confirm helper (Torrenta's `RemoveConfirm` , Editora's save decisions). The question names the thing ("Remove 3 torrents?"), the confirm button names the action (Remove, Discard, Overwrite), never OK or Yes, and Escape cancels. Where the action can be undone instead (Torrenta's `UndoStack` , Termina's and Editora's reopen-closed-tab), prefer the undo over the question.
- **Welcome page** when there is nothing to restore: Start actions (which run commands), Recent items, a Customize block (keymap and theme), Help links. Editora's `ui/WelcomePane.java` ; see the Welcome template.
- **About** shows version, build, runtime (Java and JavaFX versions), license, links, and the result of the last update check. Editora's `ui/AboutDialog.java` ; see the About template.
- **Window state**: size, position, maximized state and open items are restored on the next launch; a `--no-session` flag skips the restore. Editora's `config/WorkspaceState.java` (`windowX` … `windowMaximized`), `ui/WindowSessionCoordinator.java` , and `App.noSessionFlag` .
- **Zoom**: UI and content zoom from the keyboard (`Ctrl/Cmd +` , `-` , `0`), persisted. Editora zooms content: `view.textZoomIn` , `view.textZoomOut` and `view.textZoomReset` , kept in `Settings.fontZoom` .
- **Modes** `MAY` : Zen (no chrome), Simple (fewer menus for new users), Expert. Editora's `ui/WindowChromeCoordinator.java` (`toggleZen` , `applySimpleMode`) over the truth table in `ui/Chrome.java` .



Mirada's HeaderBar title bar carries the file name and folder; the info panel and thumbnails are commands too.



Termina puts its tabs in the title bar with JavaFX 27's HeaderBar.



Every menu item is a command and shows its chord.

Icons and color

One typeface pair, one palette, one color vocabulary and one way of drawing icons, so a tab, a status segment, a tree row and a Settings row say the same thing in the same way in every app. Reference: Editora's `styles/ui-font.css` and `ui/Fonts.java` (type), `styles/atlantafx-themes/editora-light.css` and `editora-dark.css` (the Caret & Ink palette, which Torrenta and Termina vendor verbatim), the `.root` block at the top of `styles/app.css` (the state vocabulary), `ui/Themes.java` and `ui/EditorThemes.java` (themes), `ui/Icons.java`, `ui/FileIcons.java`, `ui/MenuBarIcons.java` and `editor/MenuIcons.java` (icons), and `StyleContrastTest`, which pins the color rules.

Type

Inter speaks, JetBrains Mono works. Inter is the chrome: menus, buttons, settings, pickers, status bar. The monospace face is for text that is code or output: the editor, a terminal, the environment block in About, log timestamps.

Property	Value	Reference
UI family	Inter, in four faces, bundled in the app's resources and registered before any CSS is applied	`Fonts.UI`, `Fonts.load()`, `resources/com/editora/fonts/inter/`
Root size	14 px, the AtlantaFX theme's root size; popups pin absolute pixels where an owner's font would leak (see Menus)	`MainMenuBar.UI_FONT_SIZE`
How it is attached	A scene stylesheet (`ui-font.css`, one rule: `.root { -fx-font-family: "Inter" }`) on every window's scene, dialogs and popups included	`App.installUiFont`
Monospace	JetBrains Mono by default; Cascadia Code, Fira Code, IBM Plex Mono and Source Code Pro are bundled as editor choices	`Fonts.DEFAULT`, `Fonts.FAMILIES`

- `MUST` Attach Inter with a *scene* stylesheet, not a rule in the app stylesheet. Dialogs, the Settings window, the palette, context menus and tooltips each live in a scene of their own that does not share `app.css`, and a scene sheet is re-applied cleanly when the theme switches at run time (`setUserAgentStylesheet`). AtlantaFX neither bundles nor names Inter, and JavaFX does not fall through a font-family list, so without the sheet the whole interface renders in the platform's "System" face.
- `MUST` Bundle the font. Do not map the platform's own UI font (an earlier macOS-only sheet used San Francisco, which JavaFX faux-bolds; Inter has a real bold on every platform). Editora's four faces are in `src/main/resources/com/editora/fonts/inter/`.
- Termina and Torrenta carry the same sheet and the same four Inter faces. Modula is the stated exception: IBM Plex Mono everywhere, because every reading on a radio is a number (see *Before adding a setting* in Settings).
- A line of UI text is `-color-fg-default`; secondary text `-color-fg-muted`. `-color-fg-subtle` is about 3:1 and is for disabled or decorative text only (`StyleContrastTest.textThatMustBeReadUsesTheMutedRoleNotSubtle`).

The palette: Caret & Ink

The flagship theme pair, "Editora Light" and "Editora Dark", is authored (not vendored) and derived from Editora's icon: a **caret** teal accent, **ink** navy as the text in light and the surface in dark, **porcelain** and **abyss** grounds, and **brace** periwinkle reserved for the keyboard. Amber, red and green are semantic and never decorate.

	LIGHT	DARK	TOKEN
Ground	`#ffffff`	`#1b1e2a`	`-color-bg-default`
Porcelain / Abyss	`#f4f5f9`	`#12141d`	`-color-bg-subtle / -color-dark`
Ink	`#1b1e2a`	`#e8eaf3`	`-color-fg-default`
Muted ink	`#575d6e`	`#9ba1b5`	`-color-fg-muted`
Subtle ink	`#8b91a3`	`#6c7288`	`-color-fg-subtle`
Caret	`#0e8577`	`#43dec5`	`-color-accent-emphasis`
Caret, as text	`#0d786b`	`#43dec5`	`-color-accent-fg`
Amber	`#8d6309`	`#e3a93c`	`-color-warning-fg`
Red	`#c03434`	`#ee7a6e`	`-color-danger-fg`
Green	`#2c784c`	`#5bc98a`	`-color-success-fg`
Brace	`#3c4380`	`#b8bee5`	`-state-key (a ladder of the ground)`

- The theme files follow the vendored AtlantaFX community themes: a `.root` block of `-color-*` tokens, then component rules that reference only those tokens.
- `SHOULD` A new app ships the pair as its default theme. Torrenta and Termina copy `editora-light.css` and `editora-dark.css` unchanged into their own `styles/` ; the palette is the family's, not the editor's.
- Brace has no AtlantaFX role, so it is a literal: a `ladder()` over `-color-bg-default` that picks a deep ink on a light ground and a lighter periwinkle the darker the ground gets, and reads at 4.5:1 or more on every bundled theme's default and subtle surfaces (`StyleContrastTest.theKeyboardPeriwinkleReadsOnEveryBundledAppTheme`).

State colors

A color is a meaning, named once on `.root` and referenced everywhere. The same state wears the same color in a tab, a status segment, a tree row, a picker and a Settings row.

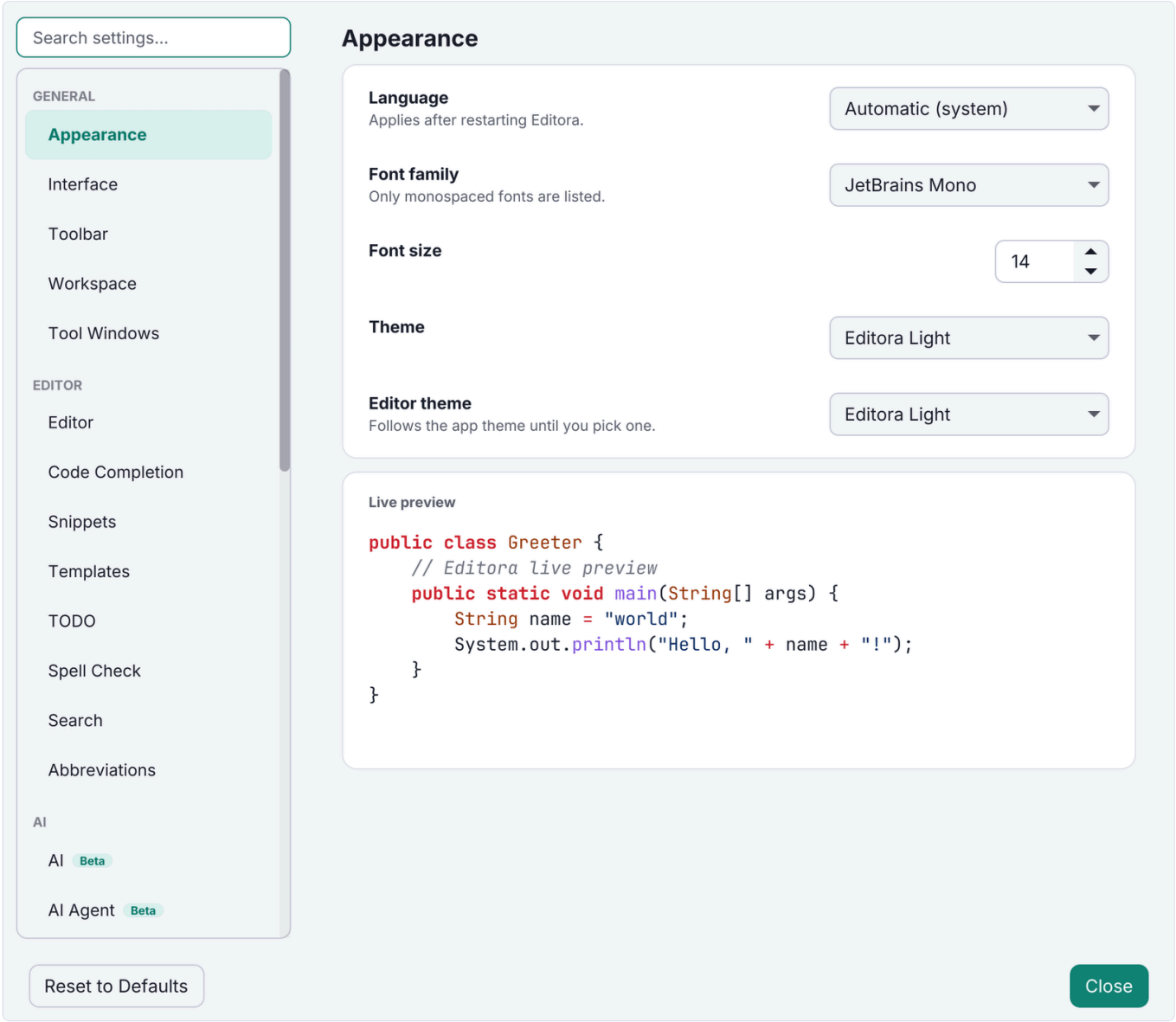
Token	Chains to	Meaning	Where it appears
`-state-amber`	`-color-warning-fg`	Not durable yet	An unsaved tab, a read-only file, a narrowed view, a snapshot build
`-state-red`	`-color-danger-fg`	Broken	Errors in the echo area and the Message Log, breakpoints, a recording
`-state-green`	`-color-success-fg`	Verified	Found, added, passing, running
`-state-accent`	`-color-accent-fg`	The app is offering something	An update, a link, the current branch, a selected row's bar
`-state-olive` , `-state-violet`	A ladder of the ground	Git only: untracked, renamed	The Project tree and tabs, the Map's structure columns
`-state-key`	A ladder of the ground	The keyboard, and nothing else	Chord chips, the menu chord column, brace matching
`-state-sel`	`-color-accent-subtle`	The selected overlay row	The palette, pickers, the Message Log (about a 13 % accent wash)

Each of the first four also has a `-bg` (the role's `-subtle`) and a `-hover` (the role's `-muted`) variant, so a pill, a row tint and a hover state come from the same place. `.state-pill` is the shape (bold, radius 999, padding 1 × 8); the color variant beside it carries the meaning.

- `MUST` No fixed hex in a component rule. A hex suits one theme; the roles track every theme for free. A severity or debug color that is not a state token fails `StyleContrastTest.severityAndDebugColoursComeFromTheStateTokens` .
- `MUST` Readable text uses `-color-fg-muted` or stronger, which is 4.5:1 on the default, subtle and overlay grounds in both flagship themes. Tinted text (`-color-<role>-fg`) reaches 4.5:1 on its own subtle fill and on both grounds.
- `MUST` Ink on an emphasis fill is `-color-fg-emphasis` , never white: white on the dark accent is 1.68:1.
- `MUST` One focus ring, `-focus-ring: -color-accent-emphasis` , drawn by the `:focus-visible` block at the end of `app.css` , at 3:1 or better on every surface (see Keyboard first).
- A brand hue the theme has no role for (periwinkle, olive, violet) is a `ladder()` of `-color-bg-default` , never a fixed pair: the old fixed light hexes read at 3.2:1 on Editora Dark.
- `SHOULD` A pure test reads the stylesheets and does the WCAG arithmetic, as `StyleContrastTest` does; a contrast regression is invisible in a code diff. Mirada's `AccentTest` checks that the accent reads as text on both grounds; Termina's `ThemeTest` keeps every chromatic color of its terminal schemes legible.

Themes

- `MUST` Light and dark, a "follow the system" choice, and a live switch without a restart (see Window shell). The first-run default is **Editora Light** (`Themes.DEFAULT`), which is also the fallback for a theme name the app does not know.
- Editora lists 28 app themes: the Caret & Ink pair, AtlantaFX's Primer, Nord, Cupertino and Dracula, and 21 community themes (Army, Autumn, Blacky, Blue, Browny, Fall, Navy, News, Spring, Summer, Winter, Yacht, with light and dark variants) vendored unmodified from the MIT-licensed dlsc `atlantafx-themes` collection. User themes under `<config>/themes/` join the list (`Themes.names()`).
- **Editor themes are separate** (`EditorThemes`). The flagship pair carries its own editor stylesheet; every community theme is served by one `adaptive` editor theme whose syntax palette is expressed in the theme's own `-color-*` tokens, so it follows whatever ground the theme has.
- Semantic colors never decorate: a theme may change the teal, the ground and the ink, but amber still means "not durable yet".



Settings: the theme picker with the Caret & Ink pair first, and the UI and editor fonts.

Icons

Every icon is an SVG path drawn in code, colored by the stylesheet so it follows the theme, and returned as a fresh node per call (a JavaFX node has one parent). There are two families, mid-migration to the UI Kit, and they are not interchangeable.



Property	Line family	Filled family	Reference
Source	The UI Kit's 16-unit outline glyphs, drawn for the kit	Material Design 24 dp single-path glyphs (Apache-2.0), plus Simple Icons brand marks (CC0)	`Icons.line`, `Icons.of`, `NOTICE`
Style class	`icon-line`	`toolbar-icon`	`app.css`
Color	`-fx-stroke: -color-fg-muted`, stroke width 1.3, round caps and joins, fill transparent	`-fx-fill: -color-fg-muted`	`.icon-line`, `.toolbar-icon`

Property	Line family	Filled family	Reference
Hover, disabled	`-color-fg-default` on hover, `-color-fg-subtle` disabled; every context mirrors the matching rule of the other family	The same	`.button:hover .icon-line`, `.button:hover .toolbar-icon`
Drawn size	16 × 1.2 = 19.2 px (`LINE_SCALE` = `ICON_SCALE` × 24 ÷ 16)	24 × 0.8 = 19.2 px (`ICON_SCALE`)	`Icons`
Used for	The chrome the kit draws: toolbar, tool stripe, tabs, and every action that has a kit glyph	Everything the kit specifies no glyph for: file types, context-menu rows, build-tool and browser brand marks	`Icons` , `FileIcons`

- `MUST` One class per family on a node, never both. A line glyph is colored by its stroke, and any `X .toolbar-icon` fill rule (there are dozens) would paint its outline solid. A glyph that moves family moves its CSS with it.
- `MUST` The same action wears the same glyph wherever it is reached: Save is one floppy on the toolbar, in the File menu and in a tab's menu. `MenuBarIcons` reuses `Icons` ; `editor/MenuIcons` (the editor package may not depend on `ui`) copies each twin's family and path data verbatim.
- `MUST` An icon-only control has a name: build it through `Icons.button` or `Icons.toolbarButton` , which set the tooltip and the accessible text together (see Keyboard first).
- `SHOULD` No invented glyph. A row gets an icon only where an existing glyph genuinely depicts it; a picture that does not mean the thing beside it has to be read and then discounted. The column stays reserved so a menu with a mix stays aligned (see Menus).
- **File icons** (`FileIcons`): a pure `iconKeyFor(name)` resolves a file name to a key (unit-tested without the toolkit), `forFileName` turns the key into a node. Brand logos from Simple Icons, category glyphs (image, archive, PDF, table, lock, JSON, settings) from Material, and the generic document glyph for anything unknown, so a new type never regresses the look.
- **Licenses.** Every glyph set and font is credited in `NOTICE` : Material Design Icons (Apache-2.0), Simple Icons (CC0; the logos stay their owners' trademarks), Octicons and Seti UI (MIT), Inter and the monospace faces (OFL).

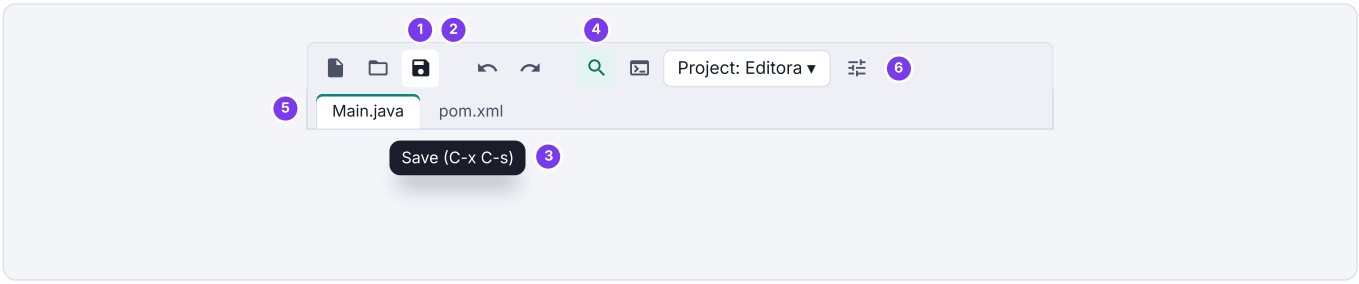
In the apps

App	Icons	Size
Editora	Both families, `ui/Icons` , `FileIcons` , `MenuBarIcons` , `editor/MenuIcons`	19.2 px on the toolbar and in menus
Torrenta	Both families, copied from Editora as `Icons.Glyph` , which knows its box and resizes with `at(px)` ; the toolbar is all line glyphs (stroke 1.5, `-torrenta-icon`)	17 px toolbar, 16 px menu and tab (`MENU_SIZE`), 13 px sidebar
Termina	Material filled, `ui/MenuIcons` , colored by `.menu-icon` (`-color-fg-muted` ; `-color-fg-default` focused; subtle and half opacity disabled)	24 × 0.7 = 16.8 px, menus only
Mirada	Hand-drawn stroked paths on a 24-unit grid, two-unit strokes, round caps, in GNOME's symbolic style, so nothing carries a license; `.icon` strokes `-color-fg-default`	16 px, centered in a 16 px box
Modula	`Glyphs` : the transport symbols (tune, seek, star) as shapes, because the characters are missing from most faces and render differently on every platform	11 px
nux	No in-window icons; the tray icon is rendered as a bitmap by severity (`TrayIconRenderer`)	Tray only

Toolbars and tooltips

A toolbar is the fast path to commands that are also in the menus and the Command Palette: nothing lives only on it. Every button is a command, so its icon, title and chord come from the registry and the active keymap. Reference: Editora's `ui/Icons.java` (`button` , `toolbarButton` , `name`), `ui/ToolbarCoordinator.java` (the customizable bar), `ui/WindowChromeCoordinator.java` (`collapseToolbarSeparators`), `ui/ToolWindowManager.java` (the tool window stripes) and the `.tool-bar` , `.toolbar-button` , `.tool-stripe-button` and `.tooltip` rules in `app.css` .

Anatomy



1. **Icon button**, 28 px: padding 3 × 5, radius 4, no border, the glyph in `-color-fg-muted` and `-color-fg-default` under the pointer. It has a name even though it shows none: the same string is its tooltip and its accessible text.
2. **Group gap.** A vertical `Separator` node whose line is transparent: its padding is what separates the groups. No drawn rule.
3. **Tooltip.** The command's title and its chord in the active keymap, "Save (C-x C-s)". A popup window with the menus' radius 8.
4. **Open toggle.** A button whose overlay is showing (Find, the palette, a split) wears `:open` : an accent-subtle fill and the glyph in the accent color. Simple mode's own button does the same while the mode is on.
5. **One band.** The menu bar, the toolbar and the tab strip sit on `-color-bg-subtle` with no hairline between them; spacing does the grouping.
6. **Pinned tail.** The project picker, About and Quit live in their own container beside the `ToolBar` , so a narrow window overflows the icons (which are all in the menus too) and never the tail.

Spec

Property	Value	Reference
Button	28 px high: padding 3 × 5, radius 4, style classes `button-icon flat toolbar-button` ; every control on the bar is pinned to 28 px, combos and menu buttons included	`.toolbar-button` , `.tool-bar .combo-box-base`
Bar	Padding 5 1 2 6 (asymmetric: the tab strip below adds its own inset); 18 px font for the bar's own labels, controls inside it at 14 px; `-color-bg-subtle` , no rule above or below	`.tool-bar` , `.menu-bar, .tool-bar`
Icon	The command's glyph, 17 px drawn (Torrenta) or a kit glyph at the toolbar scale (Editora); `-color-fg-muted` , `-color-fg-default` on hover, `-color-accent-fg` while `:open`	`Icons` , `.toolbar-icon` , `.icon-line` , `.button:open`
Group gap	A vertical `Separator` with its line transparent; its 0.75 em padding is the gap	`.tool-bar .separator:vertical > .line`
Tail	Its own container (`.toolbar-tail`), same insets and font as the bar, never overflowed	`ToolbarOverflowFxTest`
Tooltip	`<title> (<chord>)` , the chord from `KeymapManager.displayChord` ; only the title when unbound; radius 8 like the menus	`ToolbarCoordinator.buildExtraButton` , `ToolWindowManager.tooltipFor` , `.tooltip`
Stripe button	A toolbar button on the tool window rail: min 24 px, padding 9 × 5, radius 6, 8 px apart on a 30 px rail; the same muted, hover and `:open` colors; the bottom stripe adds a 14 px label	`.tool-stripe-button` , `ToolWindowManager.register`
Customize mode	The bar tinted `-color-accent-subtle` , a move cursor, the dragged button at 0.4 opacity, a 2 px accent insertion line on the drop target's edge	`.tool-bar.toolbar-customizing` , `.toolbar-drop-before`

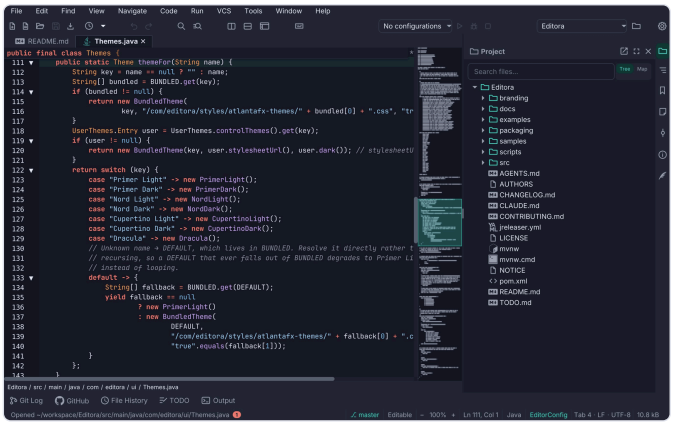
Rules

- `MUST` **. Every icon-only button has a name.** Build it through one helper that sets the tooltip and the accessible text from the same string (`Icons.button` , `Icons.name` ; Torrenta's copy of both). A hand-rolled button gets the tooltip and forgets the name, which is how Editora once had seven copies of the helper drifting apart, and how a screen reader comes to announce "button". nux sets both by hand on its "?" button for the same reason.
- `MUST` **. Toolbar buttons stay out of the Tab order.** `Icons.toolbarButton` sets `focusTraversable` to false: the command, the palette and the menus reach the action, and Tab is for the content. Termina's search bar buttons do the same. The keyboard page has the general focus rules.
- `MUST` **. The tooltip names the command and its chord**, read from the active keymap, never typed into a string: "Save (C-x C-s)". Editora's catalog buttons (`ToolbarCoordinator`), its stripe buttons (`ToolWindowManager`), Torrenta's `toolbar.withChord` and Mirada's header buttons all format it the same way.

- `SHOULD` **. Groups by spacing, not rules.** Keep the `Separator` nodes (their padding is the gap) and make the line transparent. Hiding a button group (Simple mode, a build tool that is not detected) runs `collapseToolbarSeparators`, which hides leading, trailing and doubled separators so no gap is left standing on its own, and shows them all again when nothing is hidden.
- `SHOULD` **. One chrome band.** The menu bar, the toolbar and the tab strip share `-color-bg-subtle` and draw no hairline between them; the breadcrumb, the bottom stripe and the status bar do the same at the bottom. Three rules within forty pixels read as clutter, not structure.
- `MUST` **. A toggle looks on while it is on.** A button that opens an overlay (Find, the palette, a split) and the Simple mode button carry the `:open` pseudo-class while their state holds, so the way back out does not look like the way in. Torrenta uses `:on` for Simple mode and `:active` for the alternative speed limit.
- `MUST` **. The toolbar never holds the only way to an action.** Every icon in the customizable cluster is also in the menus and the palette; that is what makes it safe to hide icons in a narrow window and to let the user remove them. Editora's `toolbar/ToolbarCatalog.java` names each item by the id of the command it runs.
- `MAY` **. A customizable toolbar.** Editora's cluster is a list of catalog ids in Settings (`toolbarLayout`), sanitized on read and shared by every window. The bar's right-click menu offers *Customize Toolbar…* (or *Done*), *Configure…* (the Settings page) and *Restore Default*. In customize mode the bar tints, buttons drag to reorder with an accent insertion line and drag off the bar to remove; Escape leaves the mode. Pinned by `ToolbarCustomizeModeFxTest` and `ToolbarCustomizationFxTest`.
- **The bar's right-click menu is one menu**, reused across right-clicks, so showing it again moves it, and `ContextMenuDismissal` closes it on a press anywhere, the toolbar included (`ToolbarContextMenuDismissFxTest`). The bar runs at 18 px, and a popup inherits the font of the node it was shown from; the `.context-menu` rule that pins menus to 14 px is what keeps this menu the size of every other (`ToolbarMenuFontFxTest`). See Menus.
- **Tool window stripes are toolbars.** A stripe button is built like a toolbar button: tooltip with the chord, accessible text bound to the window's title, a right-click menu with *Open in Split* and *Hide*. The right stripe's first icon lines up with the editor's first line of text, not with the tab strip (`ToolWindowStripeAlignmentFxTest`).

In the apps

- **Editora**: the customizable cluster, the pinned tail, the stripes.
- **Torrenta**: the same `Icons.toolbarButton` and `Icons.name`, 32 px buttons, Simple mode as `:on`; its separators still draw a line.
- **Mirada**: a `HeaderBar` title bar with flat icon buttons around the file name, stroked symbolic icons in the text color, tooltips of the form "label (chord)"; no accessible names on the buttons.
- **Termina**: no main toolbar (its tabs are in the title bar); the search bar's buttons are 24 × 22 flat icons with tooltips, out of the Tab order.



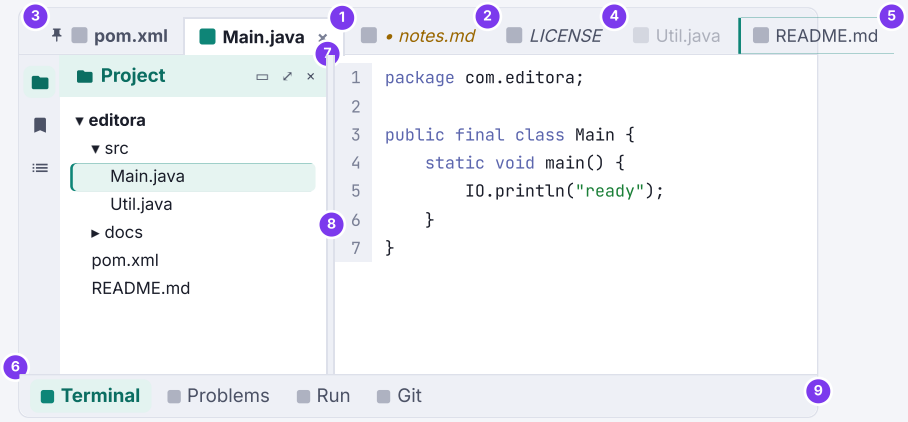
Editora's toolbar: grouped icons on the chrome band, the project picker and Quit pinned at the right.



Mirada's HeaderBar: flat icon buttons around the file name.

Tabs and tool panels

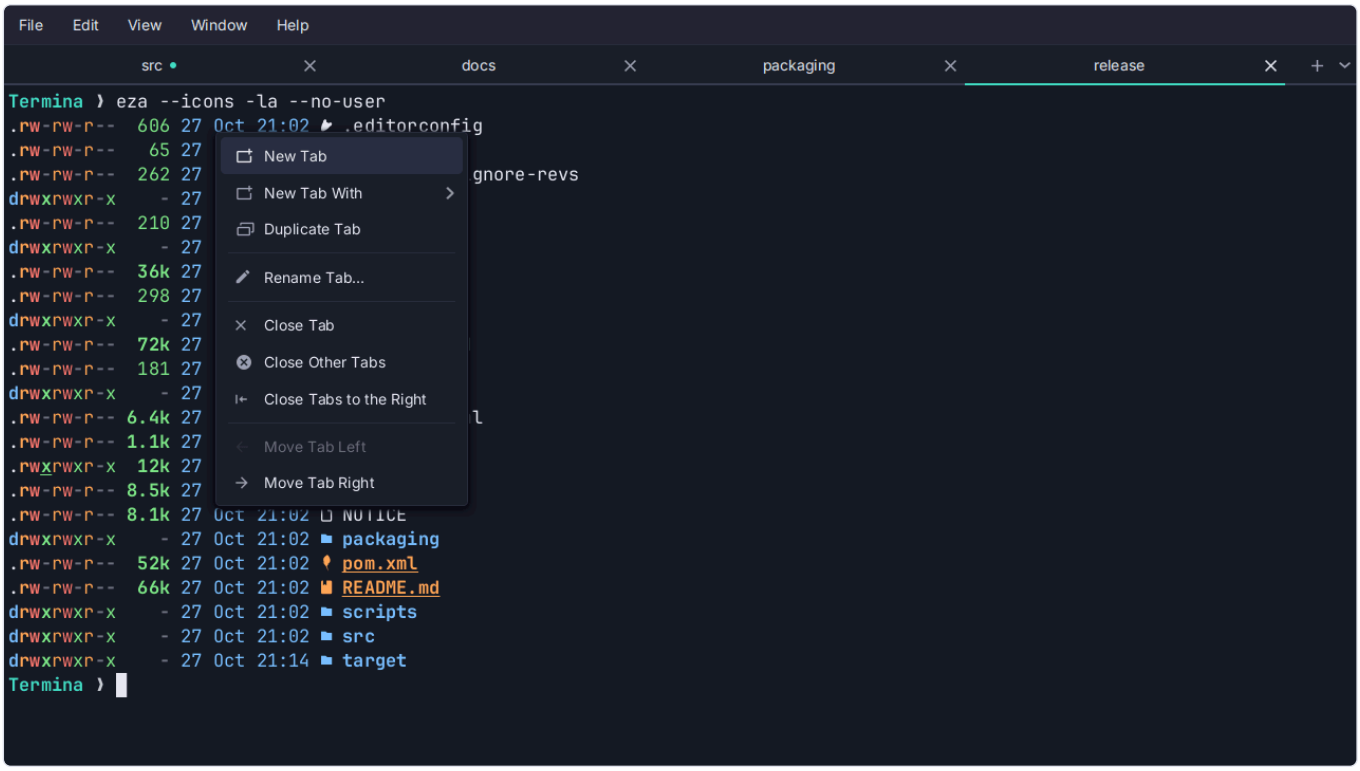
A window shows its documents in a **tab strip** and its tools in **panels** docked beside or under them. Both are driven by commands, both remember their state, and both draw their chrome as one band with the toolbar, not as boxes inside boxes. Reference: Editora's `ui/TabHeaders.java` (the tab header), `ui/ClosedTabs.java`, `ui/Switcher.java`, `ui/ToolWindowManager.java`, `ui/ToolWindowPanel.java` and the `.tab-pane`, `.tool-stripe` and `.tool-window` rules in `app.css`.



1. **Active tab.** Its fill is the editor's own surface (`-editor-surface`) with a border on three sides and none at the bottom, so the tab opens into the editor instead of sitting on it. Title bold in `-color-fg-default`; the file-type glyph tints accent.
2. **Unsaved tab.** A `•` before the name, amber (`-state-amber`, "not durable yet") and italic. Amber outranks the Git-status color: "you have not saved this" matters more than "this differs from HEAD".
3. **Pinned tab.** A pin glyph and a bold title, kept at the front of the strip in pin order. Every bulk close (Others, All, Unmodified, Left, Right) skips it.
4. **Read-only tab.** Muted and italic, echoing the status bar's Read-Only segment.
5. **Drag to reorder.** The header is the drag handle and a snapshot of the whole tab follows the pointer; the dragged tab dims to 40 % and a 2 px accent line marks the edge it would land on (left half of a tab: before it; right half: after).
6. **Tool stripe.** A 30 px rail of icon buttons, one per tool window, 8 px apart, on the subtle ground with no drawn edge. The open window's button is tinted; the one holding keyboard focus is tinted stronger. The first icon lines up with the editor's first line of text, not with the tab strip.
7. **Panel header.** The window's glyph and title (14 px bold, muted) with float, maximize and close; the same size and glyph as its stripe button, so the two name the same thing. On the subtle ground, with no rule under it; when the panel holds focus the band tints accent-subtle and the title turns accent.
8. **The divider is the edge.** The panel draws no frame: the SplitPane divider between it and the editor is the real, grabbable edge, and a hairline beside it would only double it. Its position is remembered.
9. **Bottom stripe.** Its buttons carry a 14 px label beside the glyph, side by side; the open one is accent and bold.

Tabs

- `MUST` Every tab operation is a command, so it has a chord, a palette entry and a menu row: close, close others, close all, close unmodified, close to the left and right, pin, reopen closed, next and previous. The tab's right-click menu lists them (`TabContextMenu`).
- `MUST` **. The header is the title.** The Tab's own text stays empty; a graphic header carries the file-type glyph, the markers and the title in a `.tab-title` label, which is what makes it a drag handle and lets each state set its own color (`TabHeaders`). The tooltip is the full path.
- `MUST` **. Bulk close skips pinned tabs.** `closeOtherTabs`, `closeAllTabs`, `closeUnmodifiedTabs` and `eligibleToLeft` / `eligibleToRight` all filter on the pinned set; a pinned tab cannot be closed by accident. Pinning moves the tab to the end of the pinned group, unpinning to just after it (`togglePin`, `moveToPinBoundary`).
- `MUST` **. Reopen Closed Tab.** `ClosedTabs` keeps the files whose tabs this window closed, newest first, bounded to twenty, a file listed once; reopening restores the caret. Termina brings back the *tab* (its shells, their directories, the pane arrangement, the names the user gave them, its place in the strip) and not the scrollback, because the process is gone and pretending otherwise would be worse than saying so. `C-S-t` in the CUA keymap; Termina `Ctrl+Shift+Z`, `Cmd+Z` on macOS.
- **Drag to reorder** with the feedback above (`TabHeaders.enableTabDrag`; Termina `installTabDrag` and the pure `TabReorder.insertIndex`). In Termina a double-click on a tab renames it, and a pane dragged onto a tab joins it (the whole header outlined, `tab-drop-into`).
- **Git status on the tab** in the Project tree's colors: modified accent, added green, untracked olive, renamed violet, deleted muted, conflict red (`TabGitStatus`). A read-only tab still shows it; an unsaved one shows amber.
- **Switcher.** `C-x C-b` (Emacs) opens an IntelliJ-style list of the open files in tab order: `C-n` / `C-p` or the arrows move, Enter or releasing Ctrl activates, `C-g` or Esc cancels, `C-d` or Backspace closes the highlighted file. Its width is fixed to the longest path on each open so it never resizes while you move (`Switcher`).
- **The window title** names the file, the project and the app, with a `•` prefix for an unsaved file.
- **Tabs in the title bar** `SHOULD` through JavaFX 27's `HeaderBar`; see Window shell. The tab bar can be hidden by a command (`view.toggleTabBar`, `.tab-pane.no-tab-header`).

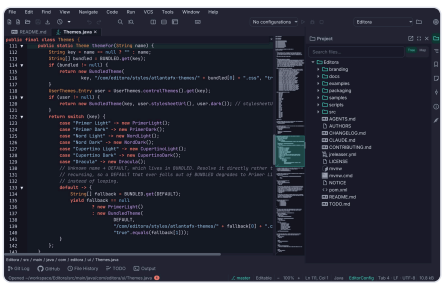


Termina's tabs in the title bar, with a tab's right-click menu.

Tool panels

- `MUST` **. A command toggles each panel** (`tool.<id>` : `M-1` … `M-0` in the Emacs keymap for ten of them), and the stripe button is the same command with a glyph. The panel's content is built on first use.
- `MUST` **. State is restored.** Which windows are open, on which side, in what order, hidden or shown, floating, and every divider position go to `workspace-state.json` and come back on the next launch (`ToolWindowManager`).
- **Three sides, two windows each.** Left, right and bottom stripes (`ToolWindow.Side`); a side holds at most two windows stacked (Project over Structure, two consoles along the bottom), because "past that a side stops being a dock and becomes a list of slivers" (`MAX_PER_SIDE`).
- **Re-dock by dragging the stripe button** to another stripe, with the same dim-and-accent-line feedback as a tab; an empty stripe appears as a dashed accent drop zone for the duration of the drag (`ToolWindowDock`).
- **Float, maximize, resize and close from the keyboard**: `view.maximizeToolWindow` (`C-x f`), `view.closeFocusedToolWindow` (`C-x 0`), the resize commands (`C-x S-.` , `C-x S-,` , `C-x S-6`). At most one window is maximized at a time.
- **Focus comes back.** Closing the panel that holds keyboard focus returns it to where it was in the editor (`focusEditor` , a weak reference to the last editor focus owner).
- **Hidden is not unavailable.** The user's show/hide choice for a stripe button is kept apart from a context-driven hide (the Commit window outside a Git repository), so the second never clobbers the first (`ToolWindowVisibility`).
- **A bottom window that is unusable at the default height asks for more on its first open** (`setFirstOpenShare`); the user's own size wins afterwards. A stripe button badges a spinner while its window has something under way (`busy`).
- **Panels follow the active document where they should**: the Git UI follows the active tab's repository (`GitPathScope`), the status bar's environment segment the active `.http` file, Local History the file whose tab menu opened it.

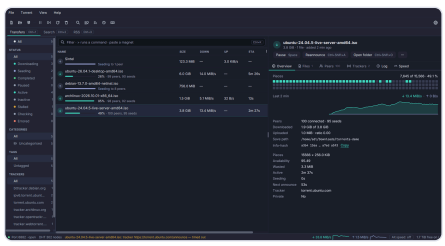
The other apps dock their panels the same way without a stripe. Torrenta's filter **sidebar** sits down the left or the right of the list or nowhere, kept as two settings keys so hiding it keeps its side (`SidebarPlace`); its inner edge drags to resize within readable bounds and a double-click resets it (`SidebarWidth`); the **detail pane** docks under the list in two columns or beside it in one, from the View menu (`DetailPane`). Mirada's info panel opens on the right and its thumbnail strip along the bottom or down the left side, each closed until its command asks for it (`ThumbnailStrip.place`). Termina splits a tab into **panes** instead: a pane is picked up with a drag while `Ctrl+Alt+Shift` (`Cmd+Option+Shift` on macOS) is held, since every smaller chord is spoken for (`PaneDrag`), and a maximized pane wears a "Pane 2 of 3" badge so the others are known to still be there.



Editora: the Project tool window docked left of the editor, its stripe on the window's edge.



Mirada: the info panel on the right and the thumbnail strip along the bottom, both commands.



Torrenta: the filter sidebar beside the list and the detail pane under it.

Spec

Property	Value	Reference
Tab strip	`-color-bg-subtle`, padding 3 10 0 10, tabs bottom-aligned so the active one touches the editor; no bottom rule	`.tab-pane > .tab-header-area`
Tab	Transparent, square corners, padding 2 11 3 11; hover `-color-bg-inset`	`.headers-region > .tab`
Active tab	Two background layers: `-color-border-default` outside, `-editor-surface` inset 1 1 0 1; title bold `-color-fg-default`; glyph `-color-accent-fg`	`.tab:selected`
Title	Inter 14 px, `-color-fg-muted`, in a graphic label; the full path as tooltip	`.tab .tab-title`, `TabHeaders.tooltip`
Unsaved	`•` prefix, `-state-amber`, italic	`.tab-header.dirty .tab-title`
Pinned	`Icons.pin()` in the header, bold; first in the strip	`.tab-header.pinned`, `togglePin`
Read-only	`-color-fg-muted`, italic	`.tab-header.read-only`
Drag	Dragged header at 0.4 opacity; 2 px `-color-accent-emphasis` on the landing edge	`.tab-dragging`, `.tab-drop-before`, `.tab-drop-after`
Closed tabs	Twenty, newest first, one entry per file, with the caret	`ClosedTabs.DEFAULT_CAPACITY`
Switcher	26 px rows, 12 visible, width fixed per open	`Switcher.CELL_HEIGHT`, `MAX_VISIBLE`
Tool stripe	30 px wide, `-color-bg-subtle`, 8 px spacing, no edge; buttons min 24, padding 9 5, radius 6	`.tool-stripe`, `.tool-stripe-button`
Stripe states	Open: `-color-accent-subtle` fill, accent glyph. Focused window: `-state-accent-hover`	`:open`, `:tw-active`
Bottom stripe	Label 14 px beside the glyph, padding 3 8; open = accent, bold	`.tool-stripe-horizontal .tool-stripe-button`
Panel	`-color-bg-subtle` ground, radius 0 0 8 8, no border	`.tool-window`
Panel header	Padding 6 6 6 12, radius 7 7 0 0, no rule under it; title 14 px bold muted with the window's glyph; active: `-color-accent-subtle` band, accent title	`.tool-window-header`, `.tool-window:active`
Panel content	`-color-bg-default`, padding 8; its tree or list draws no frame of its own	`.tool-window-content`
Per side	Two windows at most, stacked in an inner SplitPane	`MAX_PER_SIDE`
Persisted	Open state, sides, order, visibility, floating windows, divider positions	`WorkspaceState` in `workspace-state.json`

Menus

Two surfaces show the same rows: the **menu bar**, a browsable map of every command, and **right-click menus**, what can be done with the thing under the pointer. Both are built from the command registry, so a row's title, icon, chord and enabled state come from the command and never drift from the palette or the toolbar. Reference: Editora's `ui/MainMenuBar.java` (menu bar), `ui/ContextMenuDismissal.java`, `ui/LazyContextMenu.java`, `ui/RowContextMenu.java` and `ui/TabContextMenu.java` (right-click menus), and the `.menu-item` and `.context-menu` rules in `app.css`.

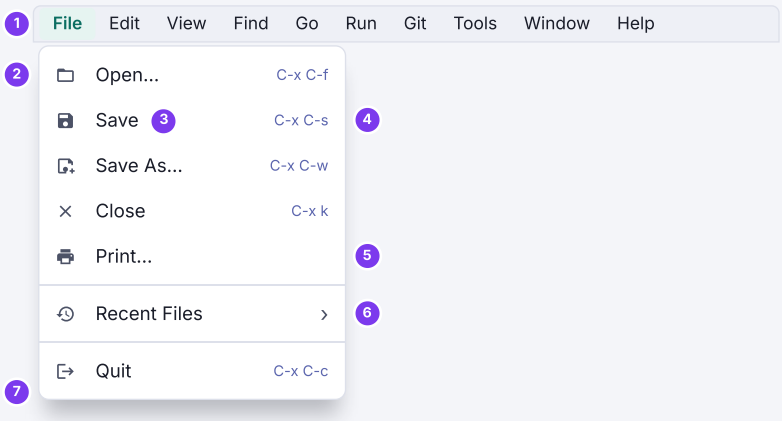
Look and feel

Both surfaces draw their rows the same way. Absolute pixels are deliberate: a popup inherits the font of the node it was opened from, and `em` would resolve against a file tree's or an 18 px toolbar's size (Editora's toolbar menu once rendered a third larger than every other menu in the app).

Property	Value	Reference
Font	Inter 14 px, regular, in every row and label	`.context-menu`, `.context-menu .menu-item .label` in `app.css`; `MainMenuBar.UI_FONT_SIZE`
Row	34 px high: padding 8 × 12, the same in every theme	`.menu-item { -fx-padding: 8px 12px }`
Icon column	22 px, reserved even when the row has no glyph, so a menu with a mix stays aligned; 8 px to the title	`MainMenuBar.ICON_COLUMN`, `rowContent`
Icon	The command's glyph, the same one as on the toolbar and in the palette (Save is the same floppy everywhere), filled `-color-fg-muted`	`Icons`, `MenuBarIcons`, `.toolbar-icon`
Title	`-color-fg-default`	`.menu-item-title`
Chord	Right-aligned in its own column, 11 px, in the keyboard color (`-state-key`, brace periwinkle); at least 18 px from the title column	`.menu-item-chord`, `alignChordColumns`
Submenu row	Laid out like a command row, with an invisible chord as wide as the menu's widest, so the arrow lands at the right edge	`nestedMenu`, `.menu-submenu`
Card	Radius 8; shadow `rgba(27,30,42,0.28)`, radius 16, **offset Y 16**	`.context-menu`
Danger item	Text in `-color-danger-fg`	`.menu-item.danger`
Disabled item	The theme's dimmed label, and the icon dimmed with it	Termina's `.menu-item:disabled .menu-icon`
Separators	Never two in a row and none at either end: when items hide, their orphaned separators hide too	`LazyContextMenu.collapseSeparators`, Torrenta's `MenuSeparators`

The shadow's offset is not cosmetic. A menu is a native window sized to its root *including the effect*, so a shadow that reaches above the menu is invisible window over the bar the menu hangs from, and the pointer lands on it instead of the next menu button. The offset must be at least the radius; Editora pins it with `MenuPopupBoundsFxTest`.

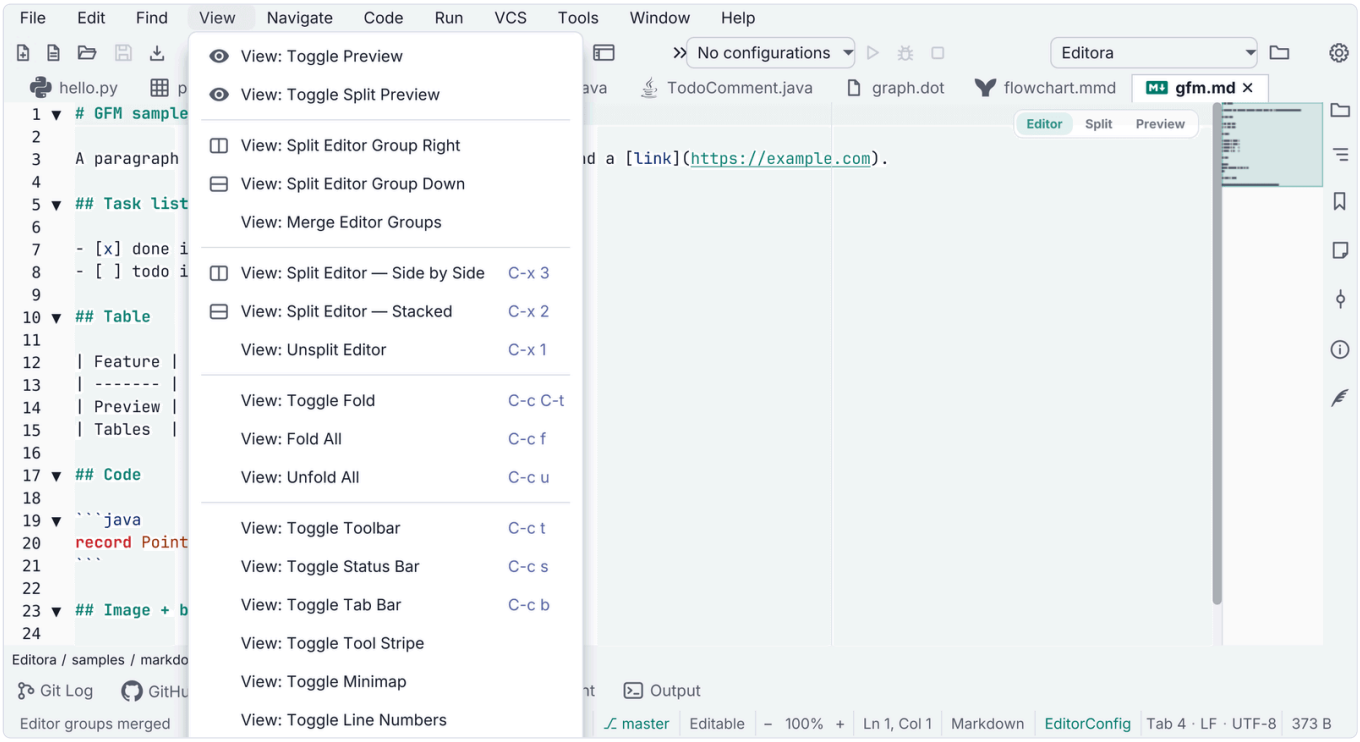
The menu bar



1. **Menu button.** Padding 3 × 10 on a bar with 4 px side insets; the bar and the toolbar share one `-color-bg-subtle` band with no rule under it.
2. **Icon column**, 22 px. A glyph only where an existing icon genuinely depicts the command; about half of Editora's rows have one. An invented glyph is worse than none, so the rest keep an empty column of the same width.
3. **Title**, 14 px Inter, the command's title with the menu's own name dropped ("Save", not "File: Save") unless two rows would then read the same.

4. **Chord column.** 11 px, periwinkle, right-aligned into one column per menu (not app-wide), read from the active keymap. Drawn by the app, not a JavaFX accelerator: Emacs chords are multi-key, which a `KeyCombination` cannot express, and on macOS an accelerator would bypass the key dispatcher.
5. **Unbound command.** An empty chord cell, never a wrong chord. A live keymap switch rewrites every cell.
6. **Submenu row.** Icon, title and an invisible chord as wide as the menu's widest, so its arrow sits at the right edge instead of halfway across.
7. **Card.** Radius 8; the shadow falls below the menu (offset ≥ radius), so nothing invisible covers the bar.

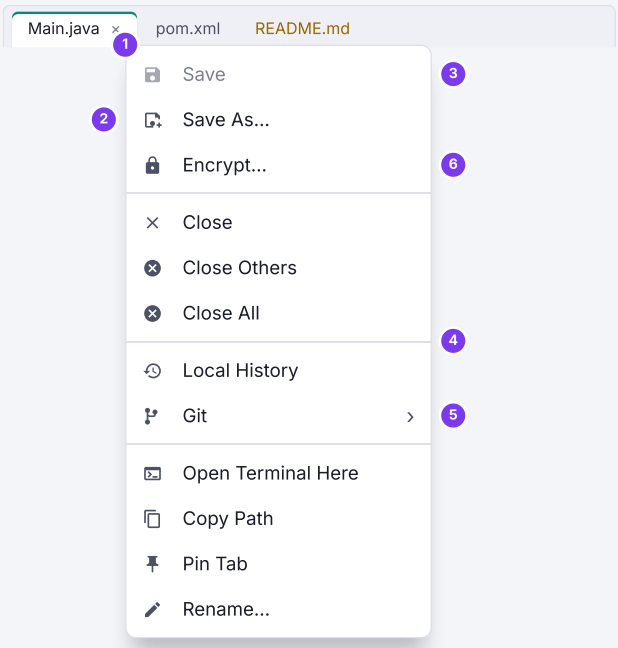
- `MUST` Every item is a command from the registry, with its title from `command.<id>`, its chord from the active keymap and its enabled state from the command's gate. The bar assembles and refreshes; it holds no behavior of its own (`MenuBarModel` over `CommandRegistry`).
- `MUST` No `KeyCombination` accelerators. In the in-window bar each row is a `CustomMenuItem` laying out icon, title, spacer and chord. On the **macOS system menu bar** (`useSystemMenuBar`) the native menu draws text only and no graphics, so there the chord is appended to the title and rows carry no icon, which is also the platform's convention.
- **Enablement is re-read on every open** (`refreshEnablement`), titles and chords on a keymap or locale change. A menu built before Git finished detecting a repository must not stay grayed out.
- **Simple mode** shows a shorter menu, rebuilt only when the mode actually changes (`MainMenuBar.setSimple`, `MenuBarModel.menus(simple)`).



Editora's View menu: icons where a glyph fits, chords in one periwinkle column. An earlier build: rows now drop the menu's own name from their titles.

Right-click menus

`MUST` on every list row, tree row, tab, editor and read-only console: the actions for *that* thing, each one a command. The mockup is an editor tab's menu, shortened; Editora's has twenty-five rows.



1. **Anchored at the pointer.** The menu's top-left corner is the click. From the keyboard (Menu key or `Shift+F10`) it opens under the selected row, a quarter of the way in and never off a narrow one, or just below the caret in an editor; never in the middle of the control.
2. **Every row has an icon**, the command's own. The one exception is a `CheckMenuItem`, whose tick lives in the graphic column.
3. **Grayed, not hidden.** An action that does not apply right now (Save on an unchanged file, Reveal on an unsaved one) stays in place, dimmed. Only a row that cannot apply to this thing at all is hidden (the Git submenu on an untitled buffer).
4. **Separators** group the rows; one that hiding has left alone hides too.
5. **Submenu** for a family of actions (Git: stage, unstage, revert, compare, history), with its own icon.
6. **Labels follow state.** Encrypt... becomes Lock File on an encrypted buffer, Pin Tab becomes Unpin Tab. Decided on every showing, not when the menu was built.

Behavior

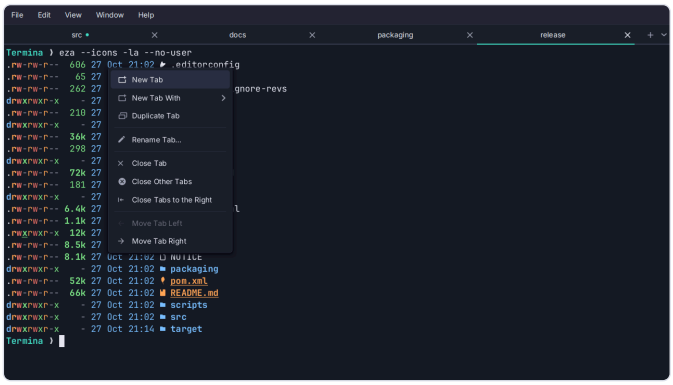
- `MUST` **. One menu at a time.** A right-click while a menu is open replaces it; the first one closes before the second shows. JavaFX stacks a second menu beside the first when each row owns its own `ContextMenu`, and the stale one is then no longer dismissible. Editora's `ContextMenuDismissal` closes every open right-click menu on the press that opens the next; Modula's `RadioPane` hides the previous menu before showing a new one; Termina keeps one menu per view and shows it again.
- `MUST` **. A press anywhere outside the menu closes it**, including inside the control the menu was opened on. JavaFX's auto-hide deliberately ignores a press on the menu's *owner node*, so a stock menu on a tree or a text area stays open when the user clicks another row or the text. The press is not consumed: a click in a list still selects the row, and a right-click on another row opens that row's menu straight away. Install the fix once, at startup, for every current and future window (`ContextMenuDismissal.install()` in `App`); drop-downs opened by a plain click (menu buttons, choice boxes, the menu bar) keep the stock behavior, and a menu that turned auto-hide off is left alone. Pinned by `ContextMenuDismissalFxTest` .
- `MUST` **. Escape closes it** and focus goes back where it was. In a terminal the Escape must not reach the shell (Termina's `onKeyPressed` filter hides the menu and consumes the key).
- `MUST` **. Reachable from the keyboard.** The Menu key and `Shift+F10` arrive at the focused control, not at a row's cell, so a menu installed on cells never saw them. `RowContextMenu.install(tree)` re-fires the request at the selected row's cell, scrolled into view first, so every row action a mouse can reach the keyboard can too.
- **Right-click selects first.** In an editor, a right-click outside every selection moves the caret to the click (so Paste lands there); inside a selection it keeps carets and selections (so Cut and Copy act on what was clicked). From the keyboard the target is the caret (`ContextMenuTarget`). In a list, the menu is the clicked row's.
- **Built when asked for.** A menu attached to something that exists once per open file is built for every file and opened for almost none, so `LazyContextMenu.of(build)` builds the items on the first showing, and an `onShowing` handler sets the labels, enablement and visibility for *this* showing, then collapses the separators. A read-only console's menu is just Select All and Copy, both disabled while it is empty (`ConsoleContextMenu`).
- **Destructive rows** carry the `danger` class and follow the window shell rule: the action asks first with a button that names it, or is undoable.
- **Terminal views** `MAY` **.** A program that grabbed the mouse owns the right-click; `Shift+`right-click always opens the menu, and the keyboard request is never contested (Termina's `shouldShowMenu` , with `ContextMenuContentionTest`).

Spec

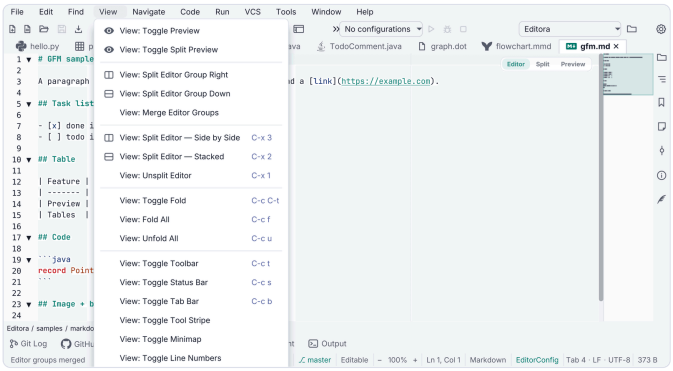
Property	Value	Reference
Kind	`javafx.scene.control.ContextMenu` , auto-hide on, hide on Escape on: a popup window, unlike the palette	`TabContextMenu` , `ConsoleContextMenu`

Property	Value	Reference
Opens at	The pointer; from the keyboard, the selected row's bottom edge at min(width ÷ 4, 48) px in, or under the caret	`RowContextMenu.anchorX`, `ContextMenuTarget`
Font	Inter 14 px, pinned on `.context-menu`, whatever the owner's font	`app.css`
Rows	34 px (padding 8 × 12); icon column 22 px; glyph scaled 0.8 from a 24 dp Material path, or a 16-unit line glyph	`Icons.ICON_SCALE`, `Icons.line`
Card	Radius 8, shadow radius 16 at offset Y 16	`.context-menu`
Dismissal	Any press outside the menu, in any window; the next right-click; Escape	`ContextMenuDismissal`, `ContextMenuDismissalFxTest`
Marker	A showing menu opened by a right-click or the Menu key carries the window property `editora.rightClickMenu`	`ContextMenuDismissal.RIGHT_CLICK_MENU`

In the apps



Termina's tab menu: an icon on every row, following Editora's convention.

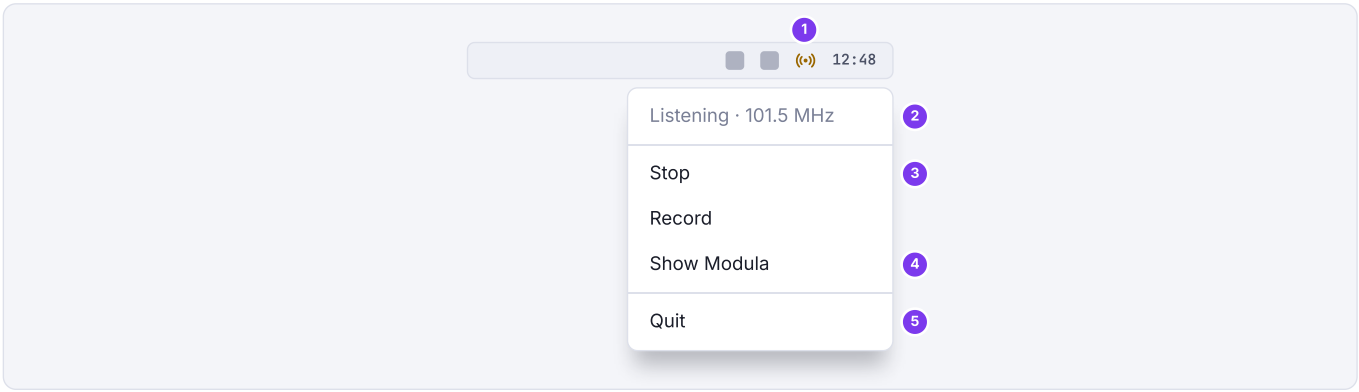


Editora's in-window menu bar with the chord column (an earlier build, before the titles lost their menu prefix).

Tray and background

An app whose work goes on without its window (Torrenta's transfers, Modula's radio, nux's monitoring) lives behind a tray icon. For such an app everything here is a `MUST`; for the others (Editora, Termina, Mirada) a tray is `MAY`, and none has one. Reference: Torrenta's `tray/` package (`Trays`, `TorrentaTray`, `SniTray`, `AwtTray`, `TrayIconRenderer`, `SniMenuModel`), `os/Autostart` with `os/AutostartSpec`, and `automation/Notifiers`. Modula's and nux's `tray/` packages are the same code with their own menus; nux's `TrayController` holds the close policy.

The icon and its menu



1. **The icon** is the app's own mark, recolored by state, not a badge and never the number it stands for ("101.5" at 22 px is unreadable and lies by rounding). Torrenta's arrow is teal while something downloads, muted when nothing does, red on an error; Modula's broadcast glyph amber while listening, ink when stopped, coral on a fault; nux shows the charge number colored by severity, gray when disconnected.
2. **Status line**, disabled, first. It is also the tooltip, because GNOME's AppIndicator extension renders a StatusNotifierItem's menu but not its tooltip, so the menu is the one place the live figures are sure to appear.
3. **The main actions** the window would offer, so the common jobs need no window: Listen becomes Stop, Record is grayed (not hidden) until the receiver runs; Torrenta's menu has none; nux offers Connect, Switch UPS and the instant commands, which still go through the app's confirmation.
4. **Show *App*** brings the window back, as a left click on the icon does.
5. **Quit**, after a separator. The only way to end the program apart from File → Quit and the system's Quit.

Tray menus are text only: AWT's `PopupMenu` and dbusmenu show no item icons, so the menu rules about icons and chords do not apply here. The order does.

Spec

Property	Value	Reference
Linux backend	A StatusNotifierItem on the session bus, found through `org.kde.StatusNotifierWatcher`: KDE natively, GNOME with the AppIndicator extension, X11 and Wayland alike. No AWT fallback: the XEmbed tray GNOME dropped in 3.26 and Wayland never had	`SniTray`, `StatusNotifierWatcher`, `Trays.create`
Windows and macOS	AWT `SystemTray`, every mutation on the EDT, a multi-resolution image so a HiDPI tray picks a crisp variant	`AwtTray`
Created	Off the FX thread, after the first frame (D-Bus and AWT can block); the outcome is a tray or the one-line reason there is none	`Trays.Attempt`, `MainWindow.installTray`
Icon	Pure Java2D from the branding SVG's geometry, at each size the backend asks for, so the panel entry and the dock entry are one mark	`TrayIconRenderer`, `TrayDisplay`
Attention	A state that needs a person sets the SNI status `NeedsAttention`; otherwise `Active`	`SniTray.update`
Left click	Shows the window: SNI `Activate` and `SecondaryActivate`, a single primary click on the AWT icon	`SniTray.Item`, `AwtTray`
Right click	The menu; the host pulls it from the item's `Menu` property	`SniMenu`, `SniMenuModel`
Updates	Only when the value-equal snapshot changed: a repaint crosses a process boundary on Linux. Torrenta polls once a second, Modula nine times	`TrayDisplay`, `MainWindow.pushTray`
Package	Free of JavaFX: it takes callbacks, and the window marshals them onto the FX thread	`TorrentaTray`

Behavior

- `MUST` **. Fully usable without a tray.** No icon is a normal outcome (stock GNOME has no StatusNotifier host), not an error. The reason is said where the person will look, because the bug they actually hit is silence: the box says close to tray, and closing quits. Torrenta's status bar says "No tray icon (…); closing the window quits Torrenta"; Modula's says closing the window will quit it (`RadioPane.reportNoTray`).
- `MUST` **. Closing the window hides it only while an icon is actually up and the setting says so.** Both are read at close time, not at startup: the checkbox can change while the window is open, and hiding a window that leaves no icon behind is how a program becomes unreachable. Otherwise a close quits.

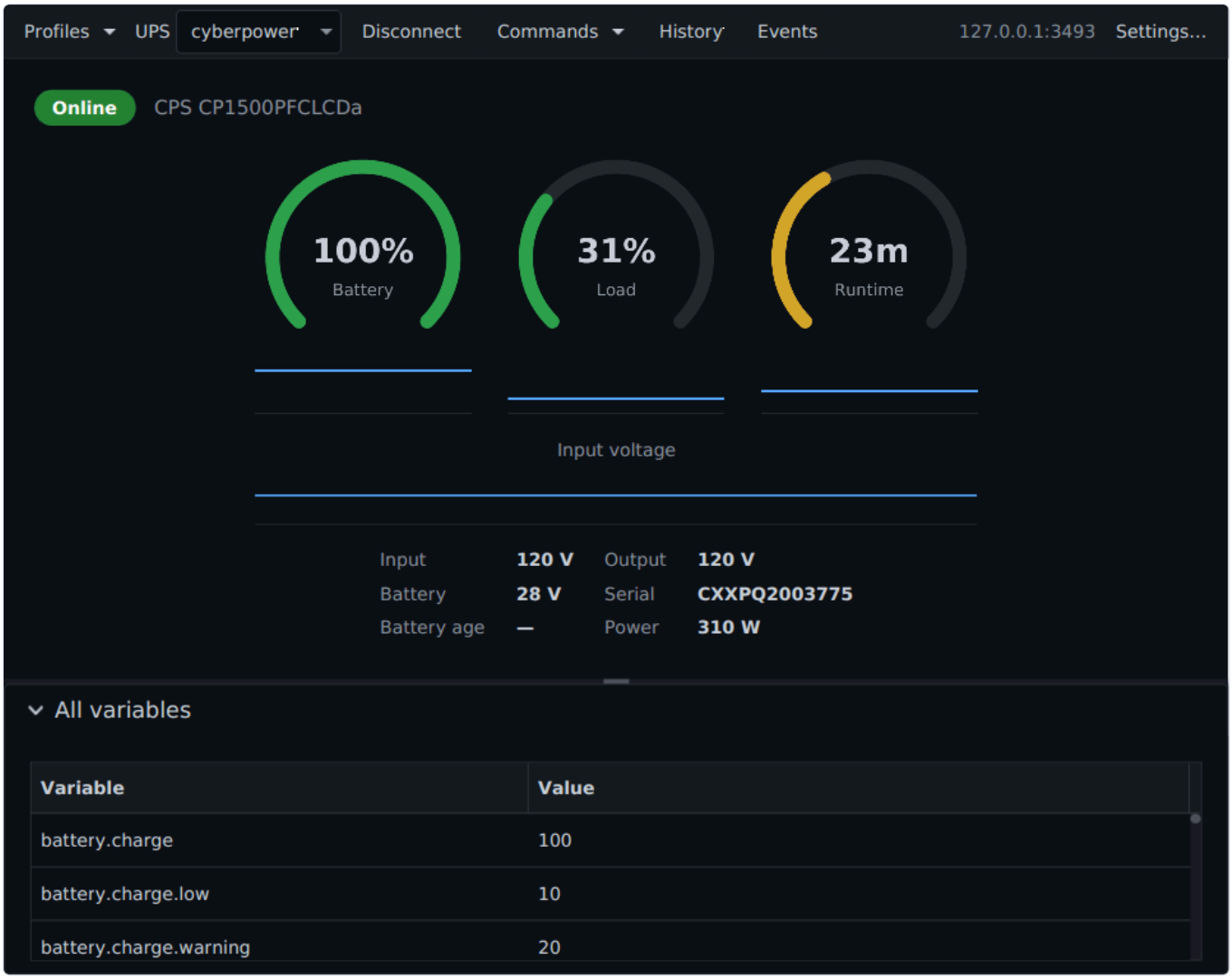
`Platform.setImplicitExit(false)` is set only once a tray exists, and turning the tray off brings a hidden window back (`MainWindow.setOnCloseRequest` , Modula's `installCloseBehaviour` , nux's `TrayController.closeHidesWindow` and `updateImplicitExit`).

- `MUST` **. Quit is explicit** and one path: the tray's Quit, File → Quit and the system's Quit all run the same method, which closes open dialogs first (quitting from inside a nested event loop aborted the JVM) and saves the window geometry (`MainWindow.quit`).
- `--minimized` **comes up behind the icon.** The window is never shown until the tray attempt has answered; if no icon comes, the window is shown after all, so a start at login can never leave a program with no way back. With the tray setting off, Torrenta starts minimized instead and nux ignores the flag (`MainWindow.show` , nux's `App`).
- **Start at login writes the OS's own entry, and nothing else.** An XDG autostart entry in `~/.config/autostart` (quoted `Exec` , `StartupWMClass` matching the installed launcher, `X-KDE-autostart-after=panel` so a tray host is up first), a LaunchAgent in `~/Library/LaunchAgents` , the `HKCU\…\Run` value on Windows (read together with `StartupApproved` , where Task Manager keeps the person's "off"). The entry is the one truth: turned off in GNOME Tweaks, KDE's Autostart page or Task Manager, the box shows unticked. The installers never do it, since they run for every account. Only an installed package has anything to start (`jpackage.app-path`); a run from source says so. The one setting beside it is *start behind the tray icon*, written into the entry as `--minimized` . Every call may block on the file system or `reg.exe` : never on the FX thread. The exact bytes are a pure, tested spec (`Autostart` , `AutostartSpec` ; nux's design).
- **Desktop notifications go through one** `Notifier` : `notify-send` on Linux (nux and Termina fall back to `gdbus` against the freedesktop service), `osascript` with `display notification` on macOS, a PowerShell toast on Windows (Torrenta ships `toast.ps1` ; nux raises it through the tray icon, which the `Notifier` keeps as the last resort on every platform). Each is a short program run with the title and message as arguments, never as part of a script, so text from a torrent name or a remote shell cannot run anything. Off the FX thread, bounded to a few seconds; a failure is logged and never breaks the app (`Notifiers` , `ProcessNotifier` , nux's `service/Notifier` , Termina's `DesktopNotifier`).
- **Notify for what happened while the window was away**, one switch per event, few on by default: Torrenta notifies on added, finished, seeding done, error and updated, with only *finished* on at first, and a Test button in Settings shows one now; nux notifies on power events with a rule per UPS; Termina and Editora relay a program's own OSC 9 and OSC 777 notifications. The in-app status already shows the state, so a notification is never the only place it is said.
- **A dock badge** `MAY` **.** Torrenta puts the download rate on the macOS dock icon through AWT's `Taskbar` and a count on a Linux launcher that honors Unity's `LauncherEntry` signal (KDE, Dash to Dock); nothing on Windows, whose taskbar badges only AWT's own windows. Cleared when nothing downloads; its own switch (`os/Badges` , `BadgeState`).
- **A second launch brings the hidden window forward.** The single-instance hand-off shows a window hidden to the tray before it opens what it was given (`MainWindow.showFromTray`).
- **AWT stays headless unless the tray needs it.** Linux never needs it; Windows and macOS lift the guard only when the tray setting is on, decided in `main` (`Trays.usesAwt` , `os.AwtUse`). A tray turned on later says "restart to show the tray icon" rather than "unsupported".

Settings copy

Key	English
`ui.tray`	A tray (menu-bar) icon with the rates and a menu; on by default
`ui.closeToTray`	Closing the window keeps Torrenta in the tray. *The torrents go on; the tray icon's Show Torrenta brings the window back, its Quit ends the program, as does Quit in the File menu. Without a tray, closing quits.* On by default; nux grays it while the tray is off
(the OS entry)	Start Torrenta when you log in. *Adds Torrenta to your login items: an entry in ~/.config/autostart on Linux, a LaunchAgent on macOS, the Run key on Windows. Turned off in the system's own settings, it shows unticked here. Only for an installed Torrenta, not a run from source.*
`os.autostartInTray`	Start behind the tray icon. *At login Torrenta comes up behind its tray icon instead of as a window, and minimized where there is no tray.* `torrenta --minimized` *does the same by hand.*
`notify.<event>`	One switch per event; `notify.finished` on by default
`status.noTray`	No tray icon ({reason}); closing the window quits {app}

In the apps

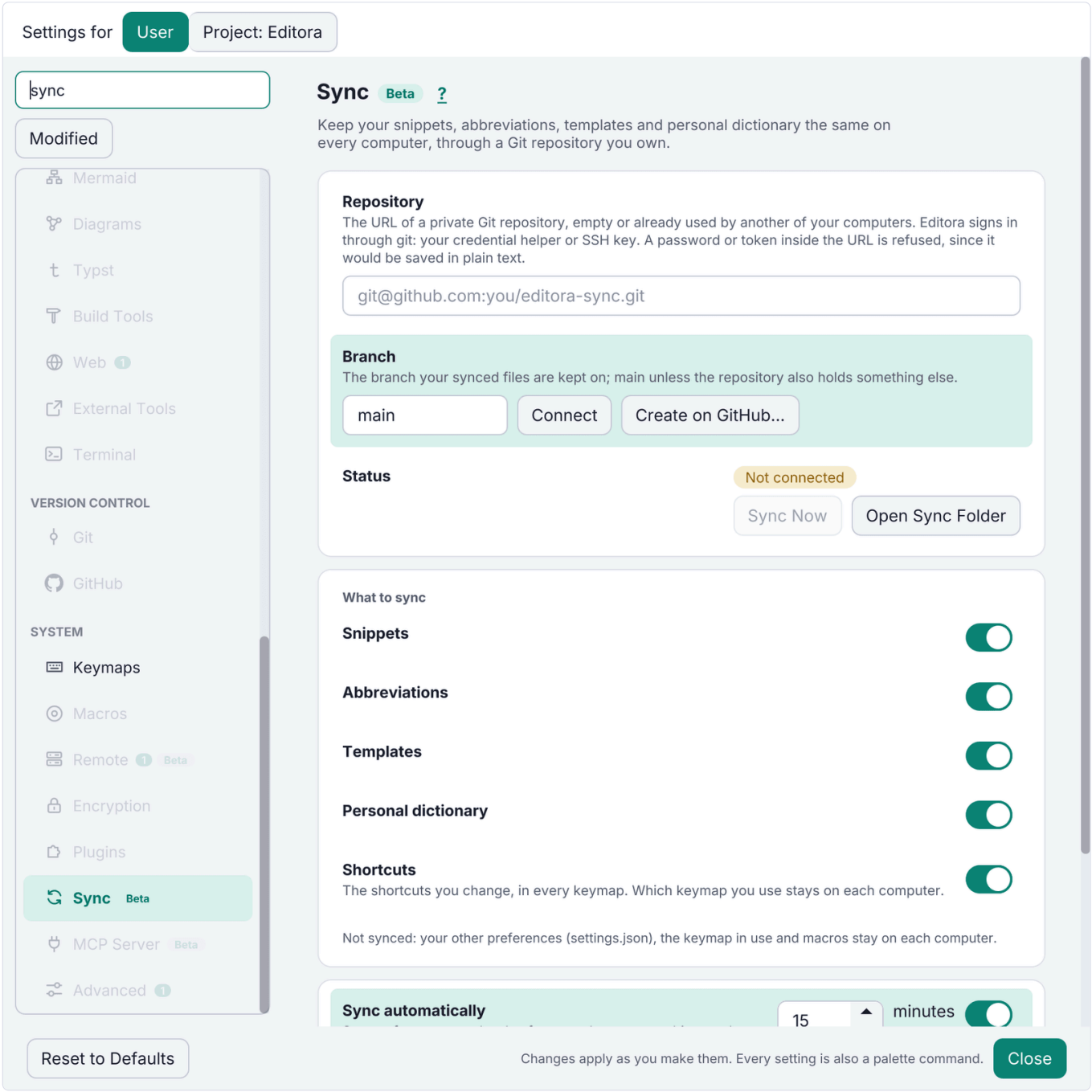


nux lives in the tray: its icon carries the charge number, and this dashboard is one left click away.

Settings

The Settings window

- `MUST` Opened by `Ctrl/Cmd+,` and a palette command. A sidebar of pages with icons on the left, the page on the right, and a **search field on top**.
- **Search** finds a row by its title, description, visible text, its card or section heading, its page name and English keywords. Title hits rank above description hits; whole words above word starts. The page with the best match opens, and every non-matching row, heading and card is hidden (`SettingsSearch`).
- **Live apply.** No Save button. Text fields commit on Enter or focus loss (`SettingsWindow.commitOnEnterOrBlur`); invalid values are marked and never committed (`SettingsSyncPage` 's branch field).
- A change made in one window reaches every other open window: every window holds the same `SharedConfig` , which announces each change (`onSettingsChanged`).



Typing sync in Settings: pages without a match gray out and the best page opens.

Before adding a setting

A setting is a question the user has to answer. When the question has one right answer, choose it, make it the behavior, and say why in the code and the docs; do not ship a knob for it. Modula's test: *would a car radio have it?* A preset button, yes; a "decimation stages" spinner, no. So Modula has no gain, bandwidth or

squelch control: each has one right value for a listener, and the app supplies it. Add a setting when people really differ (a keymap, a theme, a language), not when the developer could not decide.

Adding a setting touches five places

1. **Model** field, default and accessors, with a schema bump and migration.
2. **Control**, built once, writing the field and applying it.
3. **Row** on its page with search keywords.
4. **Load** line that sets the control from the model, safe to run at any time.
5. **Palette command** that flips or prompts the same field, plus strings in every catalog.

Config folder and schema

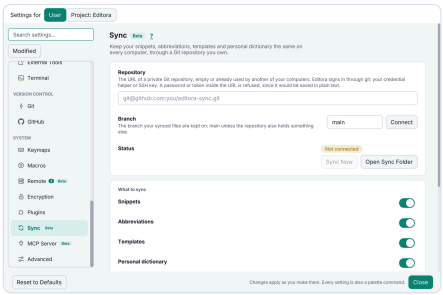
- Config lives in `~/.<app>` (`~/.<app>-dev` for `--dev`, any folder via `--config-dir` or `<APP>_CONFIG_DIR`).
- `settings.json` for preferences, `workspace-state.json` for session state, other stores in their own JSON files.
- Every versioned file has a `SCHEMA_VERSION` and a `v → v+1` migration step. A purely additive field uses an identity step.
- A file newer than the running build is backed up to `<name>.v<n>.bak` and defaults load: an older build never clobbers a newer config.
- **Never trust a config file.** Config is plain JSON (or properties) and people edit it by hand. Clamp every value to its valid range when it is **read**, not only when it is written: a hand-edited two-million-line scrollback must not exhaust memory at the next launch (Termina). An unknown value falls back to its default; a file that cannot be decoded is kept as `<name>.corrupt.bak` and defaults load. A bad file never stops the app from starting.
- **Export configuration** writes a timestamped `.zip` of the config folder (`ConfigExporter`). Import restores from one.

Secrets stay out of files

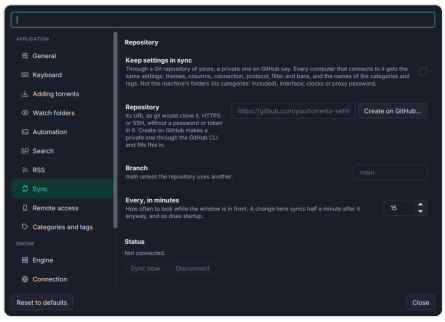
- **No secret in a config file.** API keys and tokens go to the operating system's keychain: macOS Keychain, Windows Credential Manager, the Secret Service on Linux (Editora's `secrets/SecretStore`). Calls may block on a prompt, so they run off the FX thread.
- **Or keep it for the session only.** A credential the app can ask for again (nux's UPS login) lives in memory and is never written. Work that needs it waits until the user has signed in this session; do not "fix" that by storing the password.
- **Refuse a secret in the wrong place.** A repository URL with a password or token in it is refused, with the fix (Settings Sync); a problem report redacts tokens, hosts and home paths before anyone sees it (Diagnostics).
- **No UI for a dangerous escape hatch.** An option that weakens security, such as nux's trust-all TLS (`tlsTrustAll`), exists only as a key in the config file, so nobody turns it on by clicking.

Settings Sync (Git / GitHub)

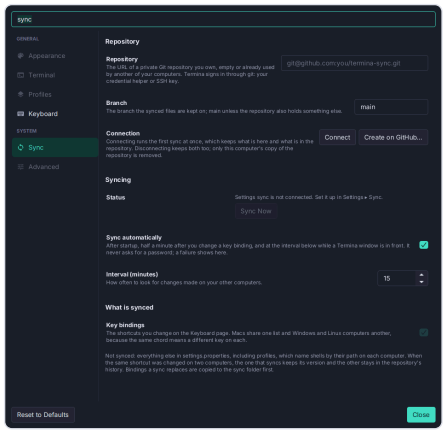
- `SHOULD` Optional sync of settings, keybindings and user data through a Git repository the user owns.
- Setup by pasting a repo URL, or **Create on GitHub** through the `gh` CLI. The account that can read the repo is pinned per clone, so machines with two GitHub accounts work.
- Automatic (on change and at start) and manual triggers. Machine-local keys (window size, paths) never sync.
- Conflict rule: on a same-entry conflict *this machine wins*. A backup is taken before any sync that replaces local files.
- Status shows as a pill in Settings and as echo messages; failures go to the Message Log.
- Reference: `sync/SyncEngine`, `sync/SyncMerge`, `sync/SyncAccount`, `ui/SettingsSync`, `docs/subsystems/settings-sync.md`.



Editora's Sync page: repository, branch, Create on GitHub... and what to sync.



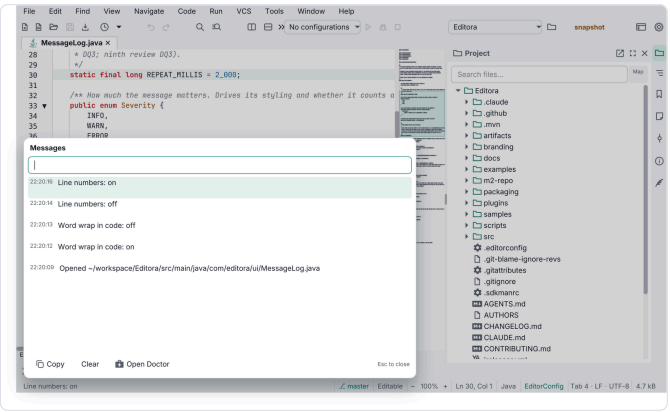
Torrenta's Sync page.



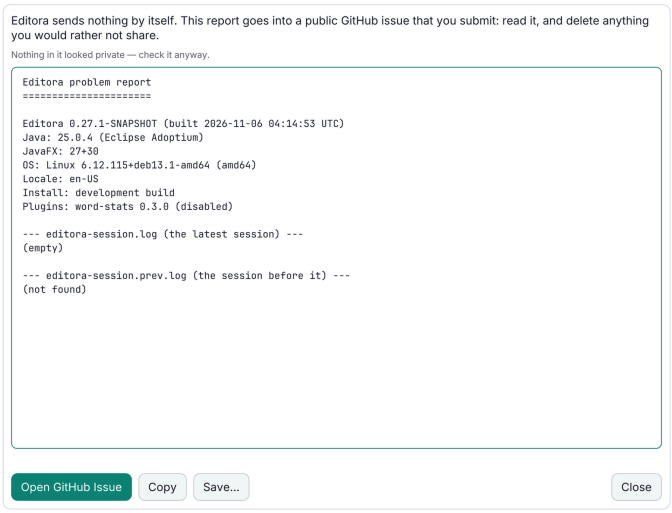
Termina's Sync page.

Diagnostics and support

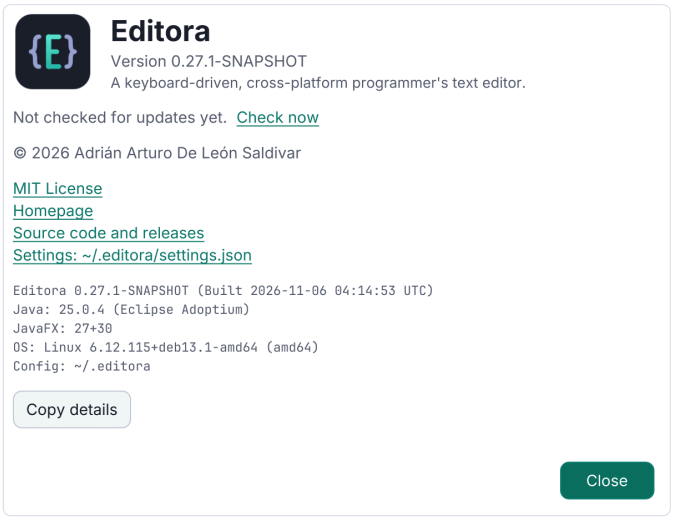
Feature	Behavior	Level	Editora reference
Message Log	Session-only history of every echo-area message with `HH:mm:ss` times, newest first, bounded. Opened by clicking the status message; Esc closes; Clear empties it.	`MUST`	`ui/MessageLog`, `ui/MessageLogPopup`
Debug Log	Captures all `java.util.logging` records and uncaught exceptions in memory (bounded) **and** mirrors them to `<app>-session.log` in the config folder, keeping the previous one as `<app>-session.prev.log` so a crash is still visible next launch. A modeless window with Refresh, Copy, Clear, Export and Report a Problem. The only way to see logs in a packaged build.	`MUST`	`ui/DebugLog`, `ui/DebugLogWindow`
Uncaught exception handler	Installed first thing in `main`; routes to the Debug Log and an echo message. Never a stack trace dialog.	`MUST`	`DebugLog.install()`
Update notification	Reads GitHub `/releases/latest`. Once per app session (not per window), at most once a day, a few seconds after the first frame. A newer version shows a status-bar notice and a line in About; "Check for Updates" runs it by hand. Off switch in Settings. Never downloads or installs by itself.	`MUST`	`update/UpdateCheck`, `ui/UpdateCheckScheduler`, `ui/UpdateCoordinator`
Report a Problem	Help → Report a Problem builds a report (version, OS, runtime, settings summary, recent Debug Log), **redacts** it (home paths, user names, tokens, hosts), shows it editable, copies it to the clipboard and opens a prefilled GitHub issue from `bug_report.yml`. Over about 7,500 characters the link falls back to "paste from the clipboard". `<app> --report` does the same from the command line for when the app cannot start. Nothing is uploaded.	`MUST`	`report/ProblemReport`, `report/Redactor`, `report/ReportCommand`, `ui/ReportProblemWindow`
Doctor	A page that checks the environment (external tools, versions, permissions, paths) with a status per row and a fix or link for each failure.	`SHOULD`	`doctor/*`, `ui/DoctorPane`
Crash recovery	Unsaved user work is snapshotted off the FX thread and offered back after a crash.	`SHOULD` (apps with user documents)	`recovery/RecoveryService`



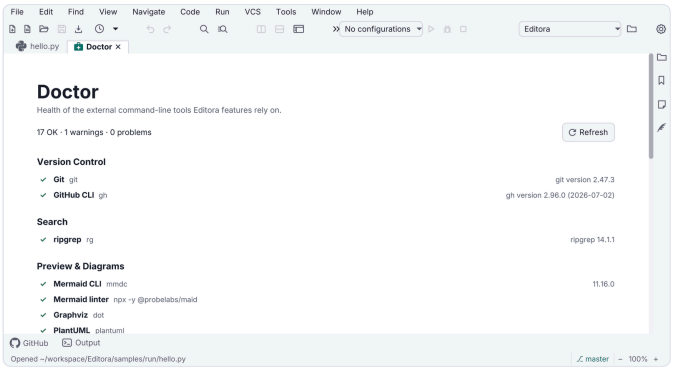
Clicking the status message opens the Message Log: this session's messages, newest first.



Report a Problem: a redacted, editable report that opens as a prefilled GitHub issue.



About: version, runtime, license, links and the update status.



Doctor: each external tool with its version, or what to install.

Secrets never reach a log, a report or a config file: see Secrets stay out of files.

Command line and launch

Every app accepts at least:

Flag	Meaning
`--help`, `--version`	Printed before the GUI starts; plain English, not translated.
`--config-dir <dir>`, `--dev`	Use another config folder (`~/.<app>-dev` for `--dev`).
`--new-instance`	Start a second process instead of handing off to the running one.
`--no-session`	Do not restore the last session.
`--report`	Write a redacted problem report and open the GitHub issue.
`[files…]`	Open these items.

- **Single instance per config folder.** A second launch hands its arguments to the running app over a local endpoint and exits (`ipc/SingleInstance`). On macOS, `openFiles` Apple Events reach the running app (`MacOpenFiles`). An advisory lock tells two processes on one folder about each other (`config/InstanceLock`).
- The headless-AWT guard (`java.awt.headless=true`) is set at the top of `main`, before any AWT class loads (`App.main`).
- A `.app` launched from Finder inherits a stripped `PATH`: resolve external tools through the login shell's `PATH` (`process/ProcessRunner.augmentedPath`).

Internationalization

Reference: Editora's `i18n/Messages.java` (`tr` , and `available()` behind the Settings language picker), the catalogs in `src/main/resources/com/editora/i18n/` , and `i18n/MessagesTest.java` , which holds them in step.

Languages

Every app ships **at least these six**. They are a floor, not the set: an app `SHOULD` add languages as people ask for them, and a new app can start with more.

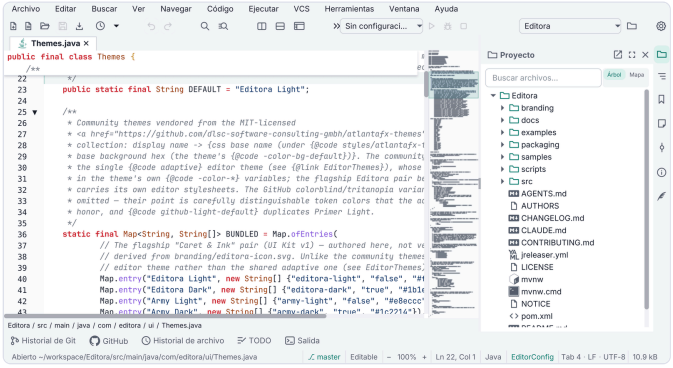
Language	Catalog
English (base)	`messages.properties`
Spanish	`messages_es.properties`
Portuguese	`messages_pt.properties`
French	`messages_fr.properties`
Italian	`messages_it.properties`
German	`messages_de.properties`

Adding a language:

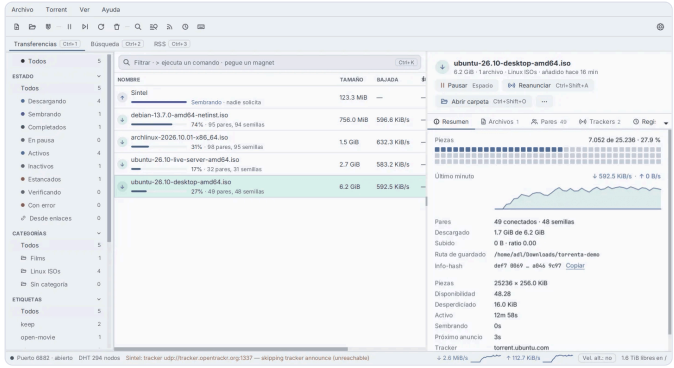
- Add `messages_<code>.properties` with every key, and the language to the Settings language picker. The parity test (`translationCatalogsHaveExactlyTheBaseKeySet`) then holds it to the same keys as the others.
- Translate for real; do not ship English values in another language's catalog. No test catches an English value, so this one is checked in review.
- Check the layout in the new language at 1024 px wide. German is today's longest; rows must wrap, never cut. Also checked by eye, in the running app (see Quality).
- Scripts beyond Latin need more than strings. Chinese, Japanese and Korean need a font fallback, because Inter has no CJK glyphs. Arabic and Hebrew need `NodeOrientation.RIGHT_TO_LEFT` on every scene. No app in the family ships such a language yet, so there is no code to copy.

Rules

- Never pass a raw English literal to a control: `tr("some.key")` from one `Messages` class. A test (`everyLiteralKeyUsedInTheSourcesExists`) fails when a `tr` key is missing from the base catalog.
- Every language has a real translation of every key. A test fails the build when a key is missing in any catalog (`translationCatalogsHaveExactlyTheBaseKeySet` , `translationsAreNonEmptyAndDistinctFromKeys`).
- `MessageFormat` rules: in a value with a placeholder, double literal apostrophes; identifiers such as line numbers and ports use `{0,number,#}` so they are not digit-grouped. Held by `aValueWithAPlaceholderNeverHasALoneApostrophe` , `aValueWithoutAPlaceholderNeverDoublesAnApostrophe` and `identifierArgumentsAreNeverDigitGrouped` .
- Untranslated on purpose: technical identifiers, `--help` and `--version` text (`App.java` 's usage text).



Editora in Spanish: menus, tool windows and the status bar all come from the catalogs.

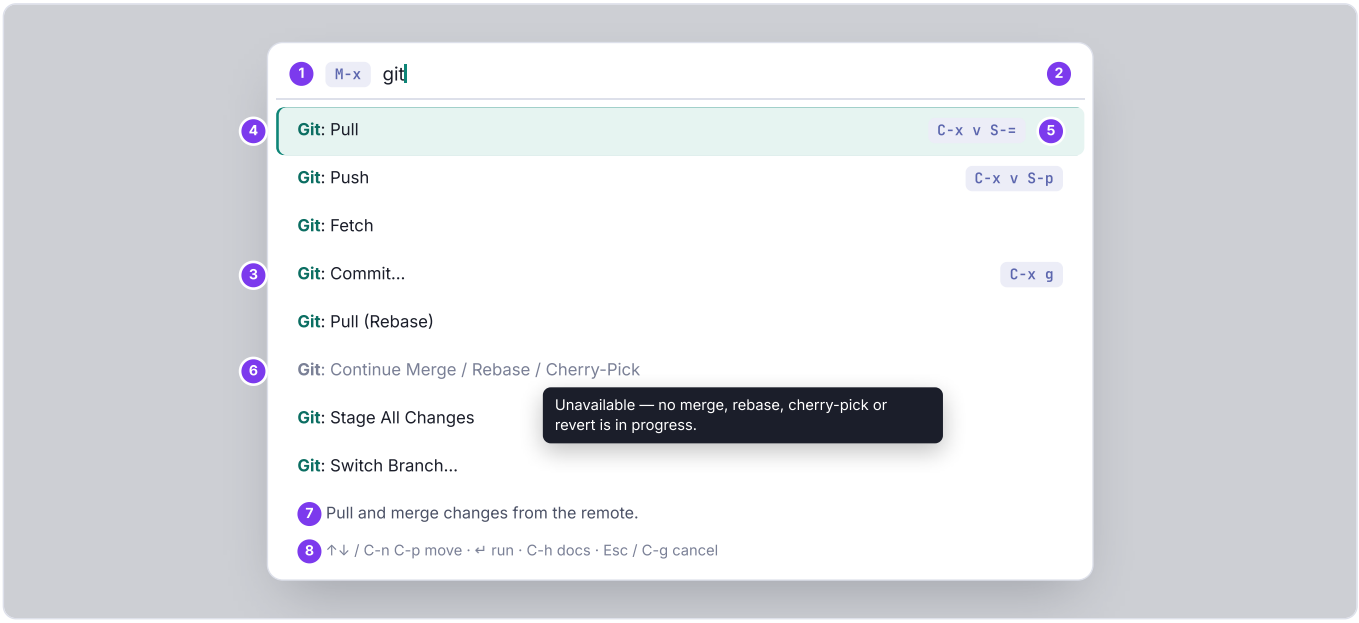


Torrenta in Spanish.

Command Palette

`MUST` in every app. One palette per window, opened by `M-x` in the Emacs keymaps and `Ctrl/Cmd+Shift+P` in the others. It lists every registered command, so a command that exists is a command people can find. Reference: Editora's `ui/CommandPalette.java`; the pure parts (`FuzzyMatch`, `MatchText`, `PaletteKeys` and the caret-key helper, `TextInputKeymap` in Editora, `TextInputKeys` in the copies) are shared with Termina and Mirada. Torrenta shares `MatchText` and ranks with its own `CommandMatcher`.

Anatomy



1. **Opener chip.** The chord that opened the palette, read from the active keymap. No chip when the user has unbound it, never a wrong one.
2. **Query field.** The first row is the input; there is no title. Prompt "Run command...". Emacs caret keys work inside it.
3. **Match highlight.** The characters the fuzzy matcher scored, bold in the accent color. The highlight and the ranking come from the same call, so they never disagree.
4. **Selected row.** Tinted background plus a 2 px accent bar on its inside edge. The bar is what makes the selection readable in a long list.
5. **Key chip.** The command's binding in the active keymap, in the keyboard color (brace periwinkle). Absent when unbound.
6. **Unavailable command.** Listed, grayed, never selected and inert to clicks. Its tooltip says why and what to do ("Unavailable — no merge, rebase, cherry-pick or revert is in progress.").
7. **Description.** The selected command's `command.<id>.desc`, one line with an ellipsis, always one line tall so the card never jumps.
8. **Key legend.** The palette's own keys, rendered from the active keymap.

Spec

Property	Value	Reference
Kind	In-scene overlay in the window's root `StackPane`, over a faint scrim. **Not** a `javafx.stage.Popup`: on Windows a popup does not reliably take keyboard focus.	`OverlayHost`
Width	620 px	`content.setPrefWidth(620)`
Rows	36 px high, 8 visible before the list scrolls	`ROW_HEIGHT`, `VISIBLE_ROWS`
Card	`-color-bg-overlay`, 1 px `-color-border-default`, radius 12, padding 6, one wide drop shadow	`.command-palette` in `app.css`
Row text	12.5 px; padding 7 × 16	`.command-palette .list-cell`
Scrim	`rgba(0,0,0,0.18)`; a click on it closes the palette	`.overlay-backdrop`
Content	Every registered command: title from `command.<id>`, description from `command.<id>.desc`	`CommandRegistry`, `MessagesTest`
Ranking	Fuzzy score over title, id and description; a query that names a title outranks a fuzzy hit	`FuzzyMatch`, `titlePhraseScore`

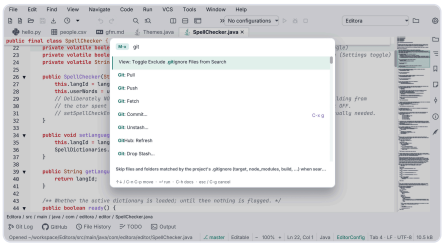
Behavior

- **Open:** the `palette.show` command, from its chord or a menu item. The query starts empty and lists every command.
- **Type** to filter; the first *enabled* match is selected.
- **Move:** ↑ ↓, `C-n` `C-p`, Page Up/Down. **Run:** Enter, or a click on an enabled row. The palette closes first, then the command runs.
- **Cancel:** Esc, `C-g`, or a click on the scrim. Focus returns to where it was.
- **Docs:** `C-h` (or the first free chord of `C-h`, `F1`, `Shift+F1` in the active keymap) opens the selected command's page on the website, for the running version.
- A live keymap switch re-renders the chips, the opener and the legend.

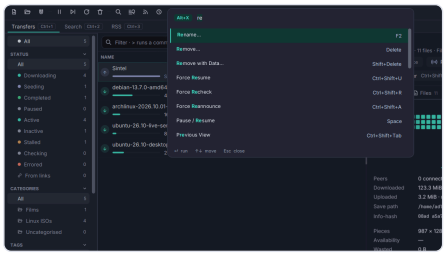
Copy

Key	English
`palette.prompt`	Run command…
`palette.disabled.<reason>`	Unavailable — *what is missing. What to do first.*

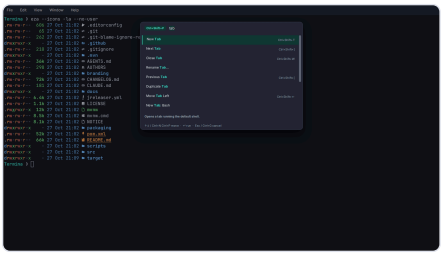
In the apps



Editora.



Torrenta.



Termina.

QuickOpen picker

`SHOULD` serve every "choose one from a list" moment: recent files, themes, keymaps, a setting's values, a branch to compare with. It is the Command Palette card with a title over the field and a prompt in it, so a list choice is made the way a command is run: type a few letters, Enter. Reference: Editora's `ui/QuickOpen.java` (one generic class, instantiated about sixty times), driven by `PickerKeys` and shown by `OverlayHost`. The keyboard page states the rule; this page is the shape.

Anatomy



1. **Title.** The command that opened the picker ("Jump to Recent File", "Set App Theme"), bold 12 px in the muted color. This is what tells the picker apart from the palette, which has no title.
2. **Query field** with a prompt per picker: "Type to filter recent files...", "Type to filter themes...". The Emacs caret keys work in it (`TextInputKeymap.installShared`).
3. **Match highlight.** The characters the fuzzy matcher scored, bold in the accent color; the same `MatchText.runs` as the palette, so the highlight and the rank never disagree.
4. **Selected row.** The first match, or with nothing typed the value in force when the picker knows it (`setCurrentItem`), so a settings picker opens on the current value.
5. **Detail**, right-aligned and muted: the file's folder, a branch's upstream, "current". It gives way with an ellipsis; the title keeps its width and never wraps.
6. **Icon** (optional, `setItemIcon`): a file-type glyph from `FileIcons` . A per-row class marks state (an unsaved buffer in amber).
7. **Key legend.** "↵ select" plus any row action's key, rebuilt from the active keymap on every showing.

Spec

Property	Value	Reference
Kind	The palette's card in the shared in-scene `OverlayHost` , over the same scrim; never a `javafx.stage.Popup`	`QuickOpen.setOverlayHost` , `OverlayHost`
Width	620 px; a picker may ask for its own width and a minimum number of rows (a registry browser with one entry still gets a roomy box)	`DEFAULT_WIDTH` , `setPreferredSize`
Rows	26 px fixed cells; the list hugs its match count, at least 1 row, 12 before it scrolls	`CELL_HEIGHT` , `MAX_VISIBLE` , `resizeList`
Title	Bold 12 px, `-color-fg-muted` , over the field	`.command-palette .palette-title`
Row	Title as matched runs, an optional icon before it, the detail in the palette's muted right-detail style	`ItemCell` , `.keybinding`
Legend	11 px muted under the list	`.command-palette .palette-hint`
Card	`-color-bg-overlay` , radius 12, 1 px `-color-border-default` , padding 6: the palette's	`.command-palette`
Ranking	`FuzzyMatch` over the search key (the label unless the picker names another, a note's body for instance); equal scores keep the supplier's order	`filter`

Behavior

- **Open** from a command. The items are read fresh on every showing and snapshotted, so typing filters a stable list; a source that answers late (an outline the language server has not returned yet) calls `reload()` .
- **Nothing typed:** the supplier's own order, which is the meaningful one (recency for recent files, document order for symbols), with the current value pre-selected when there is one. **Typing** ranks by fuzzy score; ties keep their order so rows do not shuffle under the cursor between keystrokes.
- **Move:** the active keymap's line and page chords (`nav.lineDown` , `nav.pageUp` , ...) plus ↑ ↓, Page Up/Down, Home and End, which always work. **Choose:** Enter, or a click on a row. The card hides first, then the choice runs, so focus is already back where it belongs.

- **Cancel:** Esc, `C-g`, or a click on the scrim. The host restores focus. A picker that previewed something undoes it.
- **Preview** `MAY`. `setPreview(onPreview, onCancelled)` previews the highlighted row as the selection moves and undoes it on cancel: Recent Locations shows each place and returns to the origin on Esc. A preview without an undo is worse than none. Nothing is previewed for the row the picker opens on, only for one the user moved to. The theme and keymap pickers do not preview; they apply on Enter.
- **Row actions** `MAY`. `addRowAction(key, legend, action)` acts on the highlighted row and keeps the picker open (the breakpoints picker enables or removes one), re-reading the list afterwards and keeping the cursor in place.
- **Empty list:** a placeholder sentence in the list (`setPlaceholder`), never a blank card.
- **One card at a time.** The host shows one card; opening a picker replaces whatever card was up.
- **Relatives.** The Open Files `Switcher` lists tabs in tab order, opens on the current one and closes a file with `C-d` or Backspace. `FileFinder` is the Emacs `find-file` field with Tab completion. `MultiSelectPicker` ticks several rows with Space where exactly one would be wrong. `BranchPopup` and `GitStashPopup` are sectioned pickers with their own actions. All of them use `PickerKeys`, so they move, accept and cancel alike.

Uses

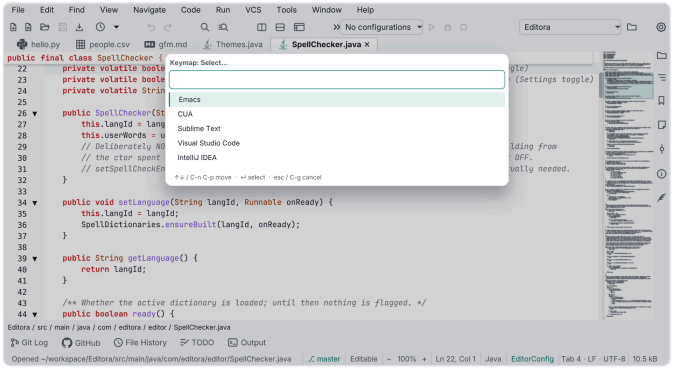
App	Picked with it	Reference
Editora	Recent files (with file icons), recent locations (previewed), bookmarks, the structure outline, tool windows, open files, projects, keymap, app and editor theme, spell language, a setting's values (opened on the current value), branches, tags and commits to compare with, remotes, commit message history, the kill ring, Occur lines, external tools, macros, the debug main class, breakpoints, agent models	`NavigationCoordinator`, `EditorSettingsCoordinator`, `DiffCoordinator`, `DebugCoordinator`, `BreakpointsPicker`
Mirada	Go to bookmark..., Open recent folder...: the palette shown over a list of names with a detail per row and an "open" legend	`CommandPalette.Choices`, `ViewerWindow`
Termina, Torrenta, Modula, nux	Commands only; profiles and other lists are menus or combo boxes today	`CommandPalette` in each

Copy

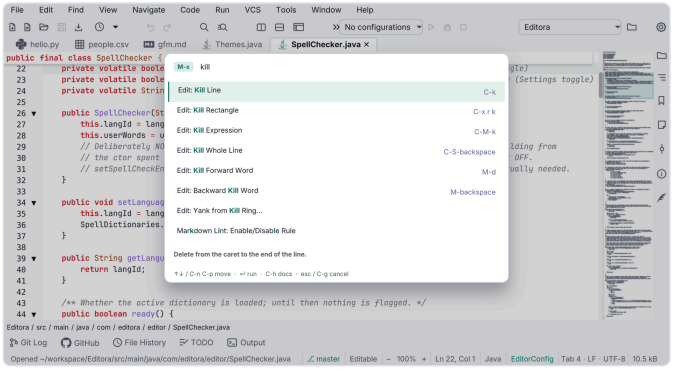
Key	English
`nav.recentFiles.title` / `.prompt`	Jump to Recent File / Type to filter recent files...
`palette.theme.appTitle`, `palette.theme.editorTitle` / `palette.theme.prompt`	Set App Theme, Set Editor Theme / Type to filter themes...
`command.keymap.select` / `palette.keymap.prompt`	Keymap: Select... / Type to filter keymaps...
`palette.setting.pick`	Type to filter...
`pickerKeys.select`, `pickerKeys.open`, `pickerKeys.cancel`	{0} select, {0} open, {0} cancel
`switcher.header`	Open Files

A title names the command; a prompt starts with "Type to filter" and ends with an ellipsis. A placeholder says what is missing and what to do ("No breakpoints set. Click a line number to add one.").

In the apps



Editora: Keymap: Select... over the editor, the title above the field.

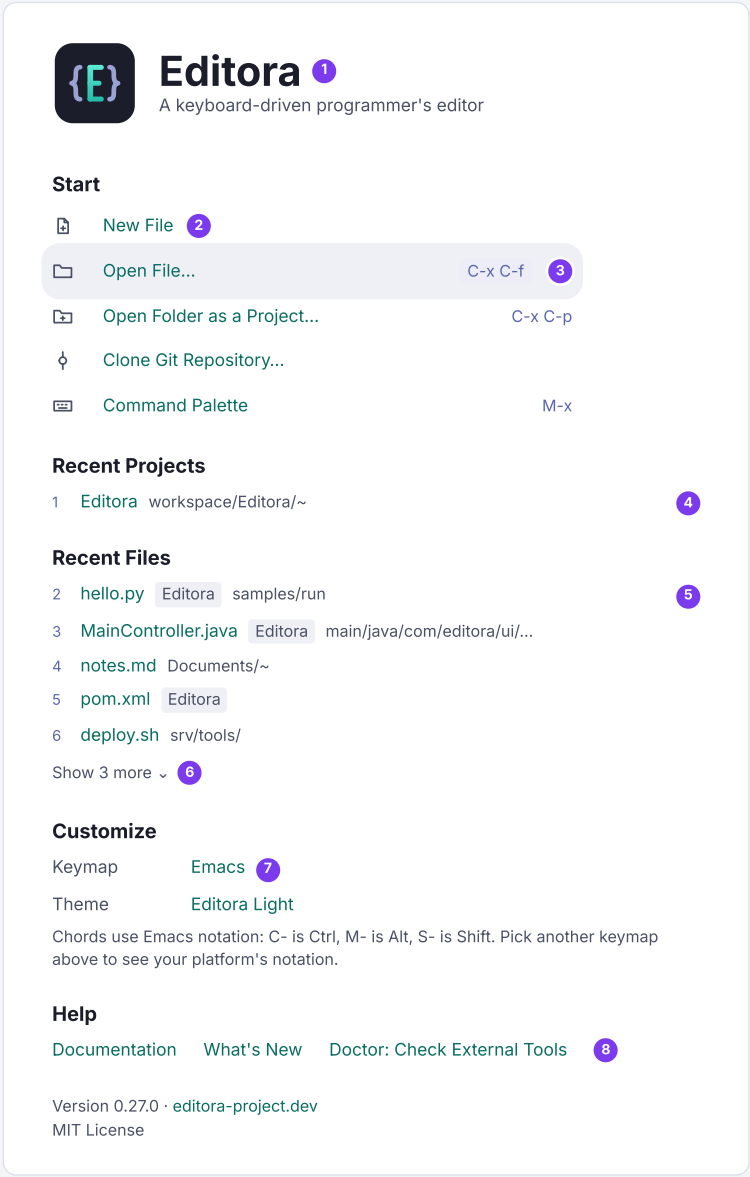


The palette the picker is built from.

Welcome page

`SHOULD` in an app that opens files or projects: the page shown when there is no session to restore, and the answer to "what do I do now?". An app whose window *is* its content shows its own empty state instead, with the same ingredients (Torrenta's drop zone, below). Reference: Editora's `ui/WelcomePane.java`, with the text rules in `ui/WelcomeText.java` and the strings under `welcome.*`.

Anatomy



1. **Identity.** The app icon at 64 px beside the name (2.4 em, bold) and the README's one-line tagline, muted. One centered column, at most 640 px wide and never narrower than 340; a longer window scrolls.
2. **Start actions**, each an existing command: New File, Open File, Open Folder as a Project (only with Projects on), Clone Git Repository (only with Git on), and the Command Palette always last, the gateway to everything else. Accent links with no box; the icon sits in a 26 px column so every label starts at the same x, and the whole row answers a click.
3. **Chord**, right-aligned past a 32 px gap, in the keyboard color, from the active keymap. Unbound: no chord. Open File shows find-file's chord in the Emacs keymap, where that is the key to learn. The hovered row tints and the chord becomes a chip.
4. **Entry number.** The first nine recent entries, projects first, are numbered in the keyboard color; typing that digit opens the entry. Projects come in the order of their most recent file.
5. **Recent file row.** The name keeps its width; a chip names the project (or the host of a remote file); the folder is muted, relative to the project root, with `~` for home, and elides from its start so the part that differs stays visible. The full path is the row's tooltip; right-click offers Copy Path and Remove from recent files.
6. **Disclosure.** Five rows per list; the rest wait behind "Show N more", which becomes "Show fewer".

7. **Customize.** The active keymap and app theme by name; each link runs the picker command. Under an Emacs-style keymap a caption explains the notation. On a first run, with nothing recent, this block moves up under Start.
8. **Help and footer.** Documentation, What's New (the release notes) and the Doctor. Then the version, the home page and the license; a dev build adds its commit on its own line once the lookup, which runs `git` off the FX thread, has answered.

Spec

Property	Value	Reference
Kind	A real tab content in its own closeable "Welcome" tab, so the tab strip handles switching and closing; not an overlay	`WelcomePane implements TabContent`, `MainController.addWelcomeTab`
Column	640 px max, 340 px min, 48 px margin, centered in a scroll pane; below the minimum the page scrolls sideways	`COLUMN_WIDTH`, `MIN_COLUMN_WIDTH`, `MARGIN`
Zoom	Every px size is multiplied by the text zoom and the fonts are `em`, so zoom scales the layout, not only the words	`setFontScale`, `.welcome-*`
Surface	`-color-bg-default`, the UI surface, not the editor's; its tab takes the same fill	`.welcome-pane`, `.ui-surface-tab`
Header	Icon 64 px; title 2.4 em bold; tagline `-color-fg-muted`	`.welcome-title`, `.welcome-caption`
Section heading	1.15 em bold, padding 28 0 8 0	`.welcome-section`
Links	`Hyperlink` in `-color-accent-fg`, no border, underlined only on hover or keyboard focus; visited stays the accent	`.welcome-action`, `.welcome-link`
Action row	Icon column 26 px, gap 12 to the label, at least 32 px to the chord; hover fills `-color-bg-subtle` with 6 px radius, insets 8 px past the row	`WelcomePane.action`, `.welcome-action-row`
Chord	`-state-key`, 0.9 em; on hover a `-state-key-bg` chip, radius 4	`.welcome-key`
Recent row	Number `-state-key` 0.8 em; folder `-color-fg-muted` 0.9 em with a leading ellipsis; project chip muted on `-color-bg-subtle`, radius 3	`.welcome-entry-number`, `.welcome-recent-dir`, `.recent-project-name`
Lists	5 rows, then a flat disclosure button	`COLLAPSED_RECENT_LIMIT`, `addCollapsed`
Focus	The first recent entry, or New File when there is none, with no focus ring; Enter and Space open it	`requestFocus`, `focusPage`

Behavior

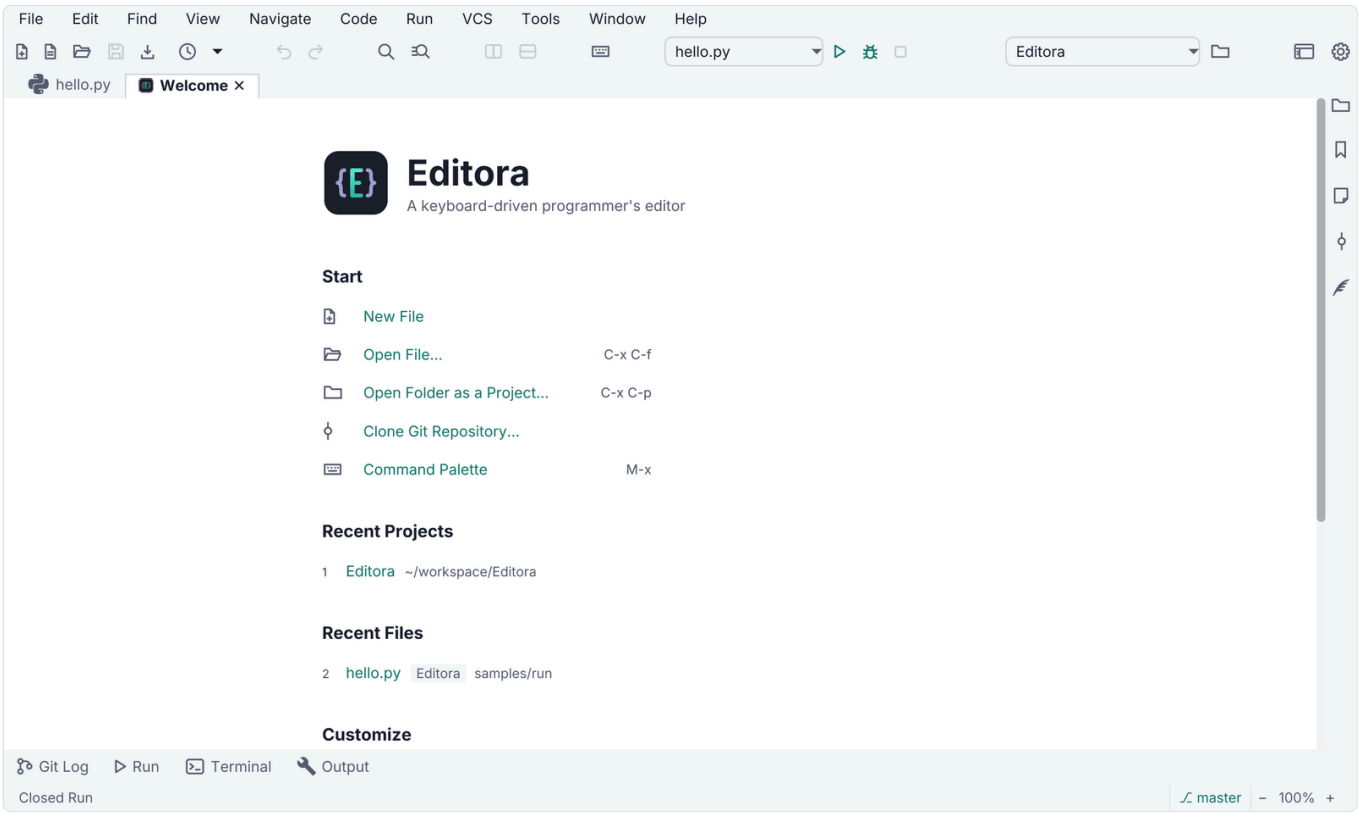
- **Shown** at startup when there is no session to restore (so always with `--no-session`), by the `view.welcome` command (Help → Welcome Page), and when the tab strip is empty after a project swap (`showWelcomeIfNoTabs`). Standalone diff startup suppresses it while its worker reads the files.
- **Closing the last file tab does not bring it back.** The editor area shows its own quiet hint instead: Open File, Open Folder, Command Palette and Welcome Page, each with its chord (`EditorEmptyState`), and takes focus so chords still reach the window.
- **Rebuilt on every show** (`refresh`), so the recents, the Projects and Git toggles, the keymap and the theme are current. A refresh keeps the focus where it was.
- **Recent lists tell the truth.** A file that no longer exists is not offered (`RecentFiles.showable`, the same filter as the toolbar's dropdown); remote files are kept unchecked. A project whose root is gone is dropped. Remove from recent files asks first, through the caller, then refreshes.
- **Digits 1 to 9** open the numbered entries: typed characters only, with no modifier, so chords and Enter are untouched (`onKeyTyped`).
- **Remote Sites** appear as their own section when saved SFTP sites exist; a click opens the connection form prefilled.
- **Customize** runs `keymap.select` and `theme.setAppTheme`; a change made there re-renders the page, chords included.
- **The Welcome tab is not a document**: its menu is Close only, no Save, Rename or Pin, and the title bar shows the app or project name, not a file.

Copy

Key	English
`welcome.tab`	Welcome
`welcome.tagline`	A keyboard-driven programmer's editor
`welcome.start`	Start
`welcome.newFile` / `welcome.openFile`	New File / Open File...
`welcome.openFolder` / `welcome.clone`	Open Folder as a Project... / Clone Git Repository...
`welcome.recentProjects` / `welcome.recentFiles` / `welcome.remote`	Recent Projects / Recent Files / Remote Sites
`welcome.noRecent`	No recent files yet. Open a file or a folder to start, or run editora FILE in a terminal.
`welcome.recent.showMore` / `welcome.recent.showFewer`	Show {0} more / Show fewer

Key	English
`welcome.entryNumber`	Press {0} to open *(accessible help)*
`welcome.customize` / `welcome.keymap` / `welcome.theme`	Customize / Keymap / Theme
`welcome.notation.emacs`	Chords use Emacs notation: C- is Ctrl, M- is Alt, S- is Shift. Pick another keymap above to see your platform's notation.
`welcome.help` / `welcome.documentation` / `welcome.whatsNew` / `welcome.doctor`	Help / Documentation / What's New / Doctor: Check External Tools
`welcome.version`	Version {0}
`welcome.page`	Welcome Page *(the menu item)*

In the apps



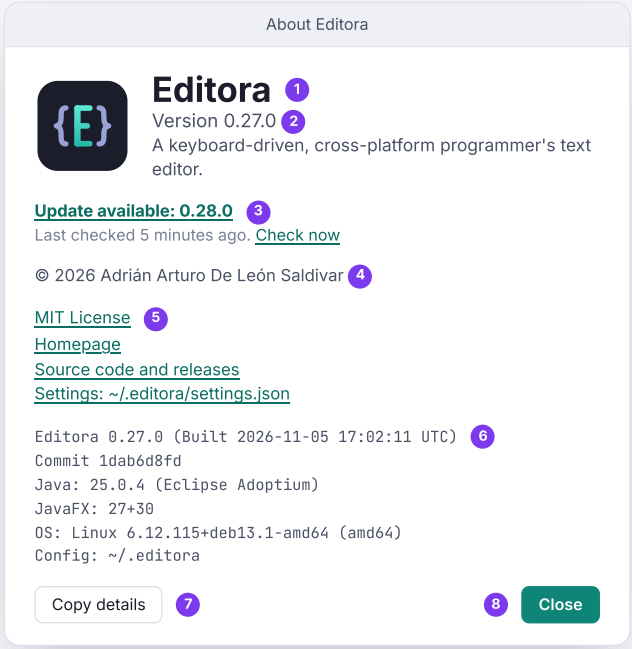
Editora: Start actions with their chords, Recent Projects and Recent Files numbered 1 and 2, Customize below.

Torrenta has no documents to come back to, so its welcome is an **empty state over the table** until the first torrent (`ui/Welcome.java` , bound to the list being empty): a drop zone that says "Drop a .torrent, a magnet or a link anywhere", the four keys that matter with their chords (open a .torrent file, add what is on the clipboard, filter or run a command, Settings), the clipboard's magnet or link offered with an *Add it* button when there is one, and where downloads go with a *change* link. Nothing on it takes focus: the keys are the way in. Termina, Mirada, Modula and nux open straight into their content and have neither.

About window

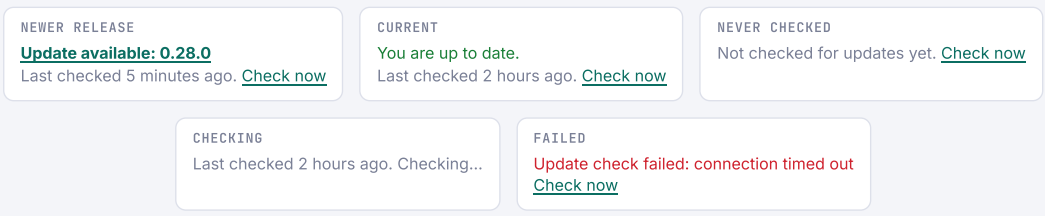
`MUST` in every app. Help → About, the `help.about` command, and on macOS the application menu's About item. It answers "what exactly am I running?" in one place, and its details are what a bug report needs. Reference: Editora's `ui/AboutDialog.java`; Torrenta (`AboutDialog`), Termina, Modula, Mirada and nux (`AboutWindow`) follow the same order.

Anatomy



1. **Identity.** The app icon at 72 px beside the name (26 px, bold). No alert header band, no info graphic: the panel reads as one block, like a native About panel.
2. **Version and tagline.** Version 14 px muted; a snapshot build appends its Git branch, "0.28.0-SNAPSHOT (feat/map)". The tagline is the README's one-line pitch, 13 px muted.
3. **Update status.** What the last update check found (see the states below), when it ran, and *Check now*. Nothing is claimed before a check has answered.
4. **Copyright**, muted.
5. **Links.** License, homepage, source code and releases, and the settings file, which opens in the app (or in the file manager for apps with no editor).
6. **Environment.** Monospace 11.5 px, muted: version and build time, commit, Java and JavaFX versions with vendor, OS with architecture, config folder. Home folders shown as `~`.
7. **Copy details** copies the environment block and stays open; its label becomes "Copied". It sits in the content, not the button bar, so pressing it does not close the panel.
8. **Close**, the one button, and the default: Enter and Esc both close.

Update status states



"Update available" links to that release's page. An update check that is switched off must never read as "up to date": with no answer, the panel says it has not checked.

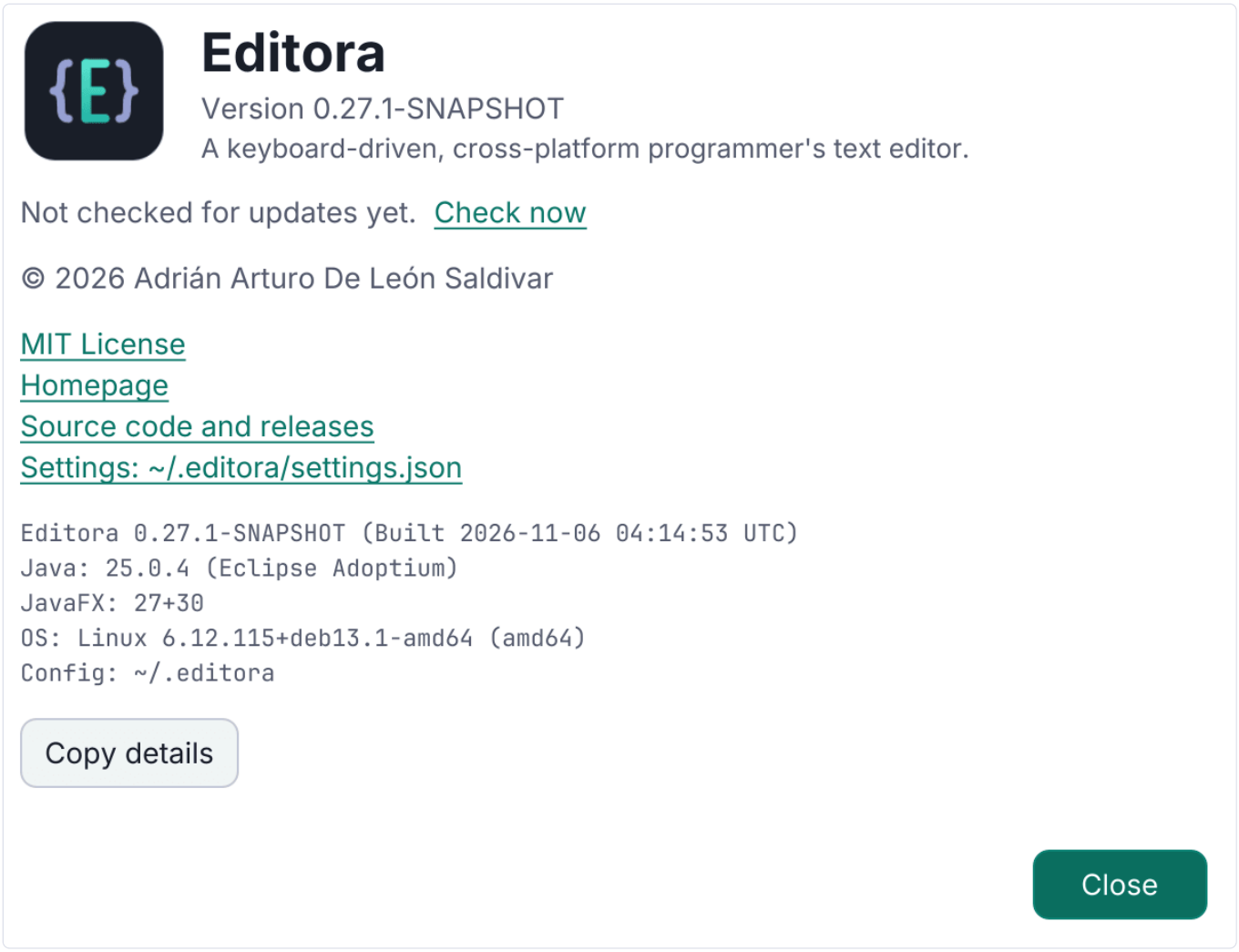
Spec

Property	Value	Reference
Kind	A modal dialog owned by the window, styled through `Dialogs.styled` so it gets `app.css` and the UI font	`AboutDialog.show`
Layout	One column, 14 px between blocks; content padding 6 8 2 8	`.about-content`
Icon	The app icon PNG at 72 px	`icon-128.png`
Name / version / tagline	26 px bold / 14 px muted / 13 px muted	`.about-name`, `.about-version`, `.about-tagline`
Update colors	Available: accent, bold. Up to date: green. Failed: red. Last checked: muted	`.about-update*`
Environment	JetBrains Mono 11.5 px, muted	`.about-env`
Source of the version	Maven-filtered `build-info.properties`, read by `AppInfo`: the same text as `--version` and a problem report	`AppInfo`

Copy

Key	English
`dialog.about.title`	About {0}
`about.version`	Version {0}
`about.updateAvailable`	Update available:
`about.upToDate`	You are up to date.
`about.lastChecked`	Last checked {0}.
`about.notChecked`	Not checked for updates yet.
`about.checkNow` / `about.checking`	Check now / Checking…
`about.homepage` / `about.releases`	Homepage / Source code and releases
`about.copyDetails` / `about.copied`	Copy details / Copied

In the apps

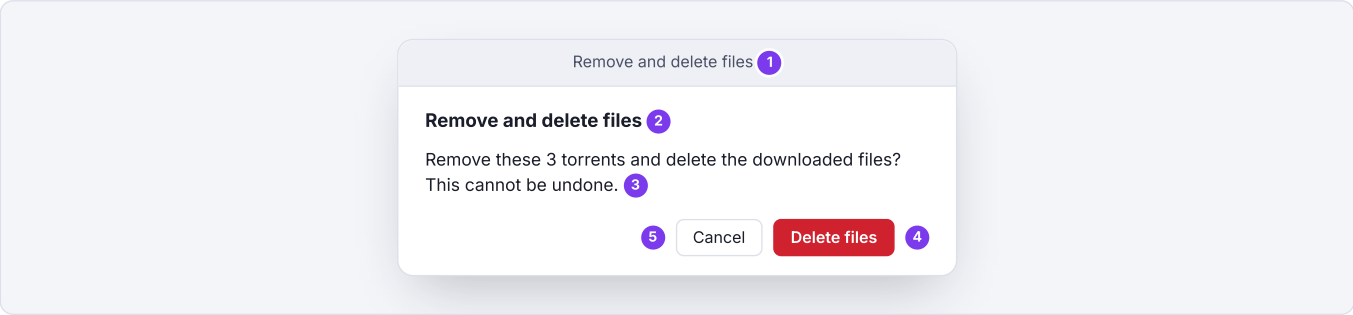


Editora.

Confirm dialog

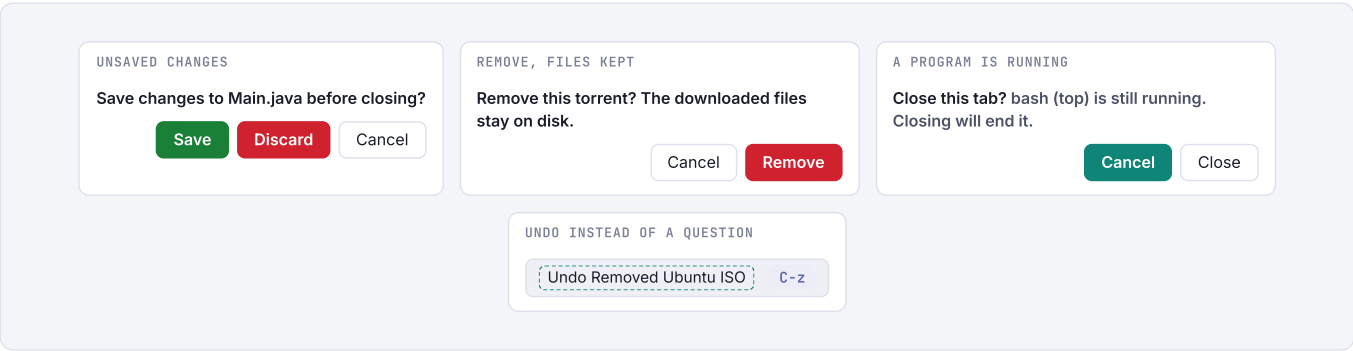
`MUST` before anything the user made is removed, deleted, discarded or overwritten, unless the action can be taken back instead (see Window shell). One question, asked the same way everywhere: the text names the thing and what happens to it, the confirming button is the action's own verb in red, and Cancel is the way out. Reference: Editora's `ui/CloseCoordinator.java` (unsaved changes, Quit) and `GitCoordinator.confirmDestructive`; Torrenta's `ui/RemoveConfirm.java` asks it for the torrent list, the RSS tab and every list in Settings.

Anatomy



1. **Window title**: the header's words again ("Remove feed", "Unsaved changes"). A plain confirmation, no icon in the title and no app name.
2. **Header**, when the question is long enough to need one. Editora's Git confirmations set none and put the whole message in the content; Torrenta's header names the action and the content asks the question.
3. **The question names the thing and the consequence.** How many ("these 3 torrents", a `{0,choice,…}` plural), which one ("the feed “Linux ISOs” and its articles"), what stays ("The downloaded files stay on disk."), and "This cannot be undone." only when it is true. Termina's multi-line paste shows the text itself in an expandable area, so the decision is made on what will be sent.
4. **The action button** carries the action's verb (Delete files, Remove, Discard, Quit, Close), never OK or Yes, and the `danger` style: red, not the accent. It is the dialog's one colored button.
5. **Cancel**, in the catalog's word, with the `CANCEL_CLOSE` role: Escape and the window's close button both answer Cancel.

States



- **Three answers** when saving is an option: Save (the `success` style, green), Discard (`danger`) and Cancel, in that order. Save that fails leaves the tab open.
- **A running program** is not the user's data, so Termina words the button Close and makes Cancel the default button: dismissing the dialog by any route keeps the program alive.
- **No question at all** where the action can be taken back. Mirada moves an image to the trash at once and offers Undo; Torrenta asks before removing a torrent but also keeps the removal on its undo stack for ten minutes; closing a clean tab asks nothing and Reopen Closed Tab brings it back.

Spec

Property	Value	Reference
Kind	`Alert(CONFIRMATION)` owned by the window (`initOwner`), shown with `showAndWait`; the answer defaults to Cancel when the dialog is closed any other way	`GitCoordinator.confirmDestructive`, `RemoveConfirm.ask`
Buttons	A `ButtonType` with the action's name and the `OK_DONE` role, and Cancel with `CANCEL_CLOSE` from the catalog, never JavaFX's stock OK and Cancel, which are worded in the JVM's locale	`RemoveConfirm.ask`, Torrenta's `StockButtons`
Action button	`danger` style class: the theme's `.button.danger` fills it `-color-danger-emphasis` with emphasis text. Save in the unsaved question: `success`	`MainController.styleUnsavedChangesButtons`, `styleQuitButtonAsDanger`

Property	Value	Reference
Default button	The action, where it has the `OK_DONE` role (Editora's Git confirmations, Torrenta's Remove): Enter runs it, and red is what says so. Editora's unsaved-changes and Quit questions give their buttons no role, so Enter does nothing. Termina's close-running-program question makes Cancel the default	`confirmDestructive`, `CloseCoordinator.unsavedAlert`, Termina `TerminalWindow`
Font	The UI font sheet attached to the dialog pane, because a dialog is its own scene and inherits nothing from the window	Torrenta `Fonts.installUiFont(DialogPane)`, Editora `App.installUiFont` (every window)
Stylesheet	`Dialogs.styled` adds `app.css` to a dialog that needs the app's own rules (About); the `-color-*` tokens and `.button.danger` resolve from the theme either way	`Dialogs`
Text	The question in the content; a header only when the question needs a name over it. Counts as plurals, names in quotes, the consequence spelled out	`Actions.removeQuestion`, `confirm.remove.<kind>`
Strings	One key per kind, header and question both, tested to exist for every kind	`RemoveConfirm.Kind`, `RemoveConfirmTest`

Behavior

- **Ask before, not after.** The question comes before anything changes; the removal runs only on the action button (`RemoveConfirm.ask` returns true on that button alone).
- **Ask once per gesture.** An action on many things asks one question with the count ("Remove these 3 torrents?", Torrenta's "Apply to many torrents" for a bulk command). Closing every tab asks per unsaved buffer, selecting each tab first so the user sees what the question is about, and remembers a Discard answer for exactly the state that was shown (`CloseCoordinator.confirmSweep`).
- **Escape cancels**, as does closing the dialog. Nothing is lost on a dismissal.
- **Say what it costs.** A delete that cannot be undone says so; one that leaves the files says that instead. A dangerous alternative (Force with Lease, Delete Anyway) is offered beside the safe one (Pull, then Push), red, and only when the safe one has already failed.
- **Testable without a screen.** The question's words come from pure code (`RemoveConfirm.header`, `question`) and the capture hook answers the dialog through `answerForCapture`, so a UI test never has to press a native button.

Copy

Key	English
`dialog.unsaved.title` / `dialog.unsaved.header`	Unsaved changes / Save changes to {0} before closing?
`dialog.save` / `dialog.discard` / `dialog.cancel`	Save / Discard / Cancel
`dialog.quit.header` / `dialog.quit.button`	Quit Editora? / Quit
`dialog.discard.tracked`	Discard all changes to "{0}"? This cannot be undone.
`dialog.discardAll.action`	Discard All
`dialog.deleteBranch.force`	Delete Anyway
`confirm.remove.feed.header` / `confirm.remove.feed`	Remove feed / Remove the feed “{0}” and its articles? The torrents already added stay.
`actions.removeQuestion.delete`	Remove this torrent and delete the downloaded files? This cannot be undone.
`window.remove.button.delete` / `common.remove` / `common.cancel`	Delete files / Remove / Cancel
`dialog.closeRunning` / `dialog.closeAnyway` (Termina)	Close this tab? / Close

In the apps

Editora asks for unsaved changes, Quit with busy terminals, pinned tabs, Git discards and branch deletions, and Local History purges, all through `Alert` with named buttons. Torrenta's `RemoveConfirm` serves the torrent list, feeds, rules, watched folders, search providers, engine profiles, categories, tags and saved views with one shape. Termina asks before closing a tab with a running program and before a multi-line paste. Mirada and Modula ask nothing: Mirada's trash is undoable, and Modula's preset chips remove on one click, which this template says they should not.

Find bar

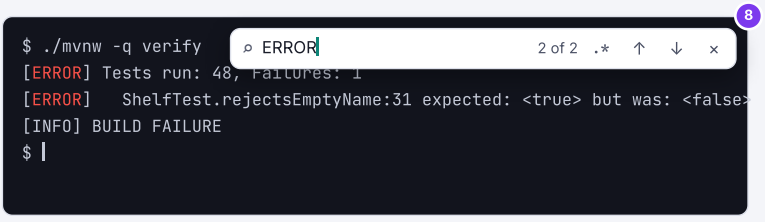
`MUST` wherever there is text to search: an editor, a read-only console, a terminal's scrollback. The bar is part of the window, never a popup, and it searches as you type. A list app filters instead of finding (Torrenta's box above the transfers), with the same field and the same Escape. Reference: Editora's `ui/FindReplaceBar.java` (the editor's find and replace), with `ui/ConsoleFindBar.java` and `ui/TerminalFindBar.java` as the same shape over a console and a terminal; Termina's `ui/SearchBar.java`; Torrenta's `Omnibox` and `FilterQuery`. Mirada has no text to search: its *Find Image…* is a picker, see the QuickOpen template.

Anatomy



1. **Query field**, focused on open with its text selected, so typing replaces the last query. A single-line selection in the editor is the first query; a multi-line one becomes the scope instead and switches *Sel* on. A clear button appears while the field has text.
2. **Count** "3 of 12", 11 px muted. Empty when there is no query, "No matches" when there are none. Wide enough that the buttons beside it do not shuffle as the digits grow.
3. **Previous and next**, 28 px flat icon buttons, the same as Shift+Enter and Enter.
4. **Options** as compact toggles with a two-letter caption (match case, regular expression, whole word, preserve case, in selection); the tooltip and the accessible name carry the full meaning. Disabled while the query is empty, except *Sel*, which is a scope.
5. **Replace** replaces the active match and steps to the next; *All* replaces every match and reports "Replaced 12 occurrence(s)" in the status line. A console or a terminal has no replace row.
6. **Close**, top right whatever the bar wraps to. Escape does the same.
7. **Matches** are all highlighted while the bar is open, the active one distinctly, and the highlights follow the text as it is edited. They go when the bar closes.

The same bar over a terminal floats at the top right of the pane it searches, in the palette's materials, so the two read as one family: a lens, the field as the card's own text, the count, a regex switch, and up, down and close.



8. **Floating card**, 380 px wide, radius 8, over the output being read. Enter steps *up* to older output and Shift+Enter down: a terminal is searched for something recent, so the search starts at the bottom (iTerm2's convention). The arrow keys do the same.

Spec

Property	Value	Reference
Kind	In-scene, never a popup. Editora docks the bar above the editor area; Termina floats it inside the `TerminalView`, so each pane of a split has its own and a search never wonders which terminal it is about	`MainController` (`topBox`), `SearchBar`
Docked bar	`-color-bg-subtle`, padding 4 8, spacing 6; icon buttons 28 × 28, flat; the groups (find, replace, options) wrap onto a second line in a narrow window instead of shrinking the fields	`.find-bar`, `.find-bar-icon`, `WrapRow`

Property	Value	Reference
Floating card	`-color-bg-overlay`, 1 px `-color-border-default`, radius 8, 380 px wide, padding 3 4 3 8, shadow 18 at offset 6; the field is the card's own text at 12.5 px	`.search-bar`, `.search-field`
Options	Toggles with a short caption; tooltip and accessible name carry the meaning; selected: `-color-accent-subtle` with `-color-accent-fg`	`OptionToggle`, `.search-toggle:selected`
Count	`{0} of {1}`, 11 px `-color-fg-muted`, right-aligned; minimum width 64 px	`find.count`, `search.count`, `COUNT_MIN_WIDTH`
Debounce	The search runs 150 ms after the last keystroke, and an edit to the text re-highlights after the same 150 ms without moving the caret	`debounce`, `editDebounce`, `TYPING_SETTLE`, `searchTyping`
Threading	A document up to 512 K characters is searched where the request is made; larger ones on one shared daemon thread, latest query only, the result applied on the FX thread. A scrollback is always scanned off the FX thread	`SYNC_SEARCH_CHARS`, `SEARCH_POOL`, `LatestOnly`, `TerminalView.searchAsync`
Bounds	A page of matches around the active one, not every match of a query that matches millions of times; a regular expression has a time budget. Termina keeps the newest 1,000 matches	`SearchMatcher.DEFAULT_PAGE`, `find.tooComplex`, `ScrollbackSearch.MAX_MATCHES`
Bad regex	Said in place, never by a dialog: the status line ("Bad regex: …") or, in Termina, the field in `-color-danger-fg` and the count "Invalid"	`find.badRegex`, `.search-field:invalid`
Focus	The bar and its buttons are out of the Tab order; in a terminal Tab belongs to the shell	`setFocusTraversable(false)`

Behavior

- **Open:** the find command, `C-s` in the Emacs keymap and `Ctrl/Cmd+F` in CUA, Sublime, VS Code and IntelliJ (`find.show`; `F3` as well in CUA and Sublime). Termina: `Ctrl+Shift+F`, `Cmd+F` on macOS, since a plain `Ctrl+letter` belongs to the shell. The command while the bar is already open finds the next match (Editora) or puts the caret back in the field (Termina).
- **Type** and the search runs: every match highlighted, the first one at or after the caret selected and scrolled to. The anchor is the caret at open time, so refining the query does not run away down the document.
- **Step:** Enter next, Shift+Enter previous, wrapping around at the ends. In a terminal the directions are up and down the scrollback. Editora: Alt+Enter turns every match into a caret.
- **Close:** Escape or the close button. The highlights go, a search still pending is canceled so nothing moves after the bar is gone, and focus returns to the editor, the console or the shell. In Termina the Escape is consumed in the field and never reaches the shell, and the terminal's own key and mouse filters leave every event aimed at the bar alone (`fromSearchBar`).
- **Keys in the field.** The keymap's caret and editing chords (`C-a`, `C-e`, `C-k`, `C-y`, `Ctrl+V/X/C/Z`) act on the field, not on the document (`TextInputKeymap.installShared`; Termina's and Mirada's `TextInputKeys`). Find's own chords stay global, so next, previous, replace and close work while the field has focus.
- **Nothing found** is one status-line message ("Not found: x") or the count "No matches". No beep, no dialog.
- **Replace** keeps case when *AB* is on; a replacement that is too slow stops with "Replace stopped: the pattern is too slow" instead of hanging the window.
- **A filter box** (Torrenta's omnibox) is the list app's version: plain words filter the names as they are typed, `key:value` terms become chips, `>` runs a command. `Ctrl+K` focuses it and is drawn in the box as a key badge. Escape peels one layer: a box with something in it empties, an empty one gives the list back. The box is out of the Tab order so a fresh launch does not land every keystroke in it, and the window's own chords (undo, `Ctrl+Alt+1…9`) do not fire while a text field has focus: `Ctrl+Z` there is the field's own undo.

Copy

Key	English
`find.prompt`	Find
`find.replacePrompt`	Replace
`find.count`	{0} of {1}
`find.notFound`	Not found: {0}
`find.badRegex`	Bad regex: {0}
`find.tooComplex`	Search stopped early: the pattern is too complex for this text
`find.replaced`	Replaced {0} occurrence(s)
`search.caseTip` · `search.regexTip` · `find.wholeWord`	Match case · Regular expression · Whole word
`search.older` · `search.newer` · `search.close` (Termina)	Previous match, up (Enter) · Next match, down (Shift+Enter) · Close (Escape)
`search.none` · `search.invalid` (Termina)	No matches · Invalid
`window.filter.prompt` (Torrenta)	Filter · > runs a command · paste a magnet

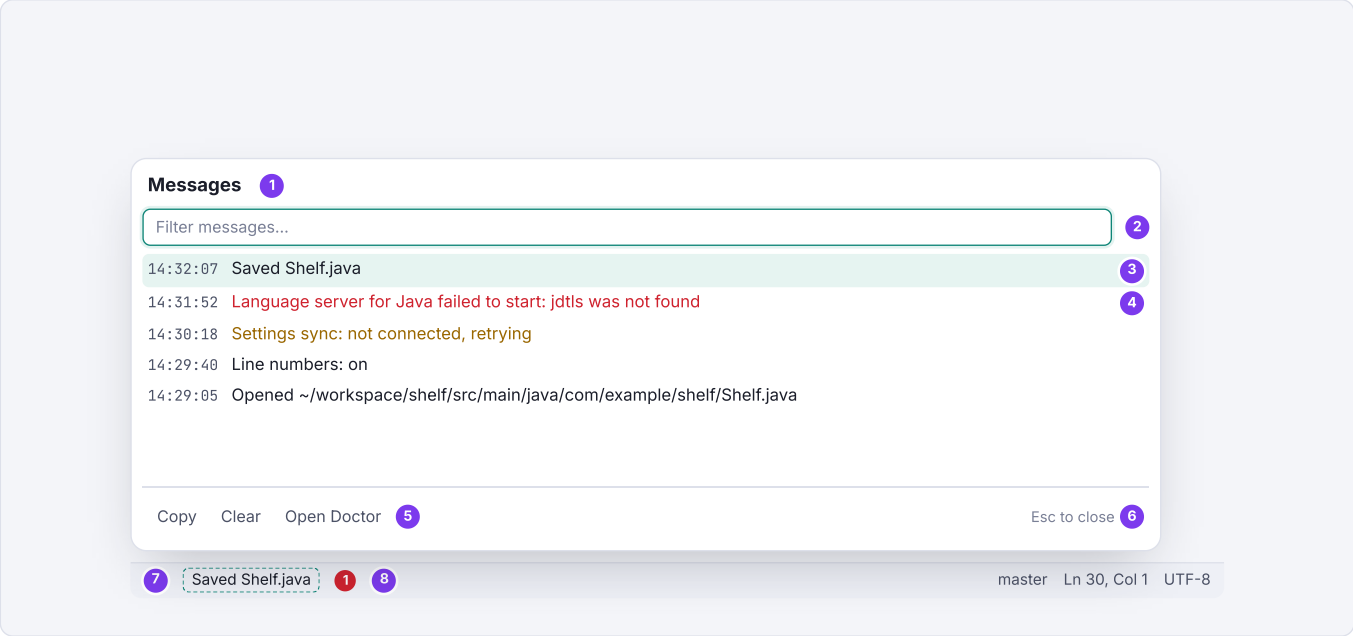
In the apps

- **Editora:** find and replace over the editor (`FindReplaceBar`), with *Find in Files* (`search.inFiles`) as a separate results window; the console and terminal tool windows carry the same bar without the replace row.
- **Termina:** the floating card over each pane, with the regex switch; matches are found across soft-wrapped lines, so a word that reached the right edge and carried on is still found.
- **Torrenta:** the omnibox above the transfers, which filters, runs commands and adds what is pasted.
- **Mirada, Modula, nux:** no free text to search; Mirada finds an image through a picker.

Message Log

`MUST` in every app with a status bar. The status bar's echo line shows one message and replaces it with the next; the Message Log keeps them, so nothing the app said is lost because it scrolled past. Reference: Editora's `ui/MessageLog.java` (the model, no JavaFX) and `ui/MessageLogPopup.java` (the view); Torrenta carries the same pair.

Anatomy



1. **Title** "Messages", 14 px bold.
2. **Filter**, focused on open. Filters as you type; ↓ or Enter moves into the results without leaving the field.
3. **Entry.** The time as `HH:mm:ss` (11 px, monospace, muted) and the message as it appeared in the echo line. Newest first.
4. **Severity** colors the message, not the row: errors red, warnings amber, information in the default color. The time stays muted.
5. **Actions:** Copy (the visible entries as text), Clear (empties the session log and its unread count), and Open Doctor: messages carry no actions of their own, and a missing tool or a server that would not start is where Doctor helps.
6. **Key hint**, muted, 11 px.
7. **Echo line.** The status bar's latest message; a click opens the log.
8. **Unread errors.** A red count beside the echo line while errors have arrived since the log was last opened; a click opens the log.

Spec

Property	Value	Reference
Kind	A popup anchored just above the status bar's echo area, over the window	`MessageLogPopup.show`
Size	773 px wide; the list 336 px high, scrolling past that	`WIDTH`, `LIST_HEIGHT`
Card	`-color-bg-overlay`, radius 12, like the palette	`.message-log`
Rows	Padding 2 4, radius 5; selected row `-state-sel`	`.message-log-list .list-cell`
Capacity	The newest 200 messages; older ones drop off	`MessageLog.MAX_ENTRIES`
Repeats	A message repeated within 2 s moves to the top as one entry, unread once	`REPEAT_MILLIS`
Lifetime	This session only, in memory. Not written to disk: the Debug Log is the persistent record	`MessageLog`
Blank messages	Not recorded (a blank message only clears the echo line)	`MessageLog.add`

Behavior

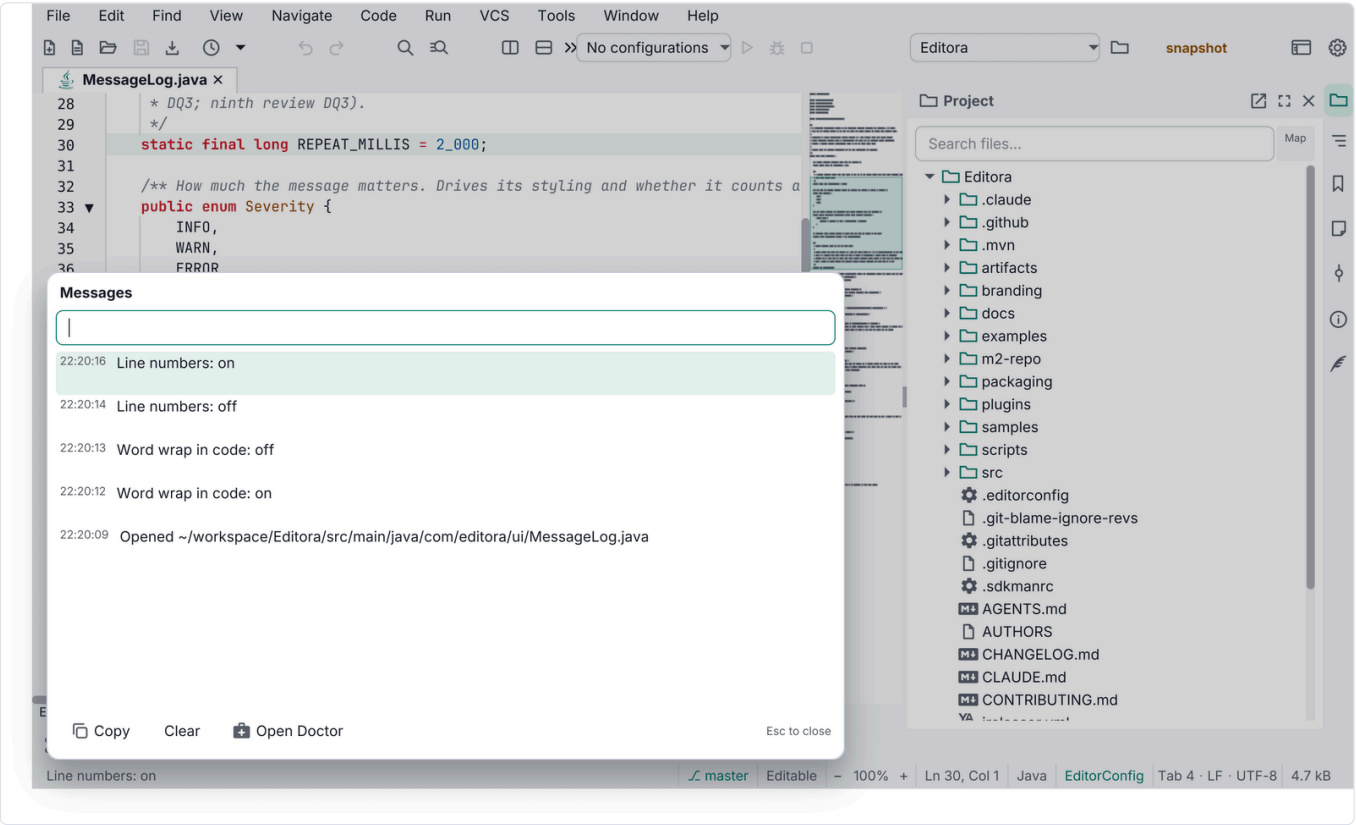
- **Open:** click the echo line or the unread-error count, or run `view.messageLog`.
- **Close:** Esc, `C-g`, or a click outside. **Copy:** `Ctrl/Cmd+C` copies the selected entries.

- **Severity comes from the message key** (`StatusSeverity`): a key ending in `Failed` or `Error` is an error, one with `cannot` or `invalid` a warning. Name new keys accordingly, or report through the error channel directly.
- Opening the log, or clearing it, resets the unread-error count. An unread error is never evicted silently: the count keeps it even when the entry drops off the end.

Copy

Key	English
`messagelog.title`	Messages
`messagelog.filter`	Filter messages...
`messagelog.empty`	No messages yet
`messagelog.copy` / `messagelog.clear`	Copy / Clear
`messagelog.doctor`	Open Doctor
`messagelog.hint`	Esc to close

In the apps

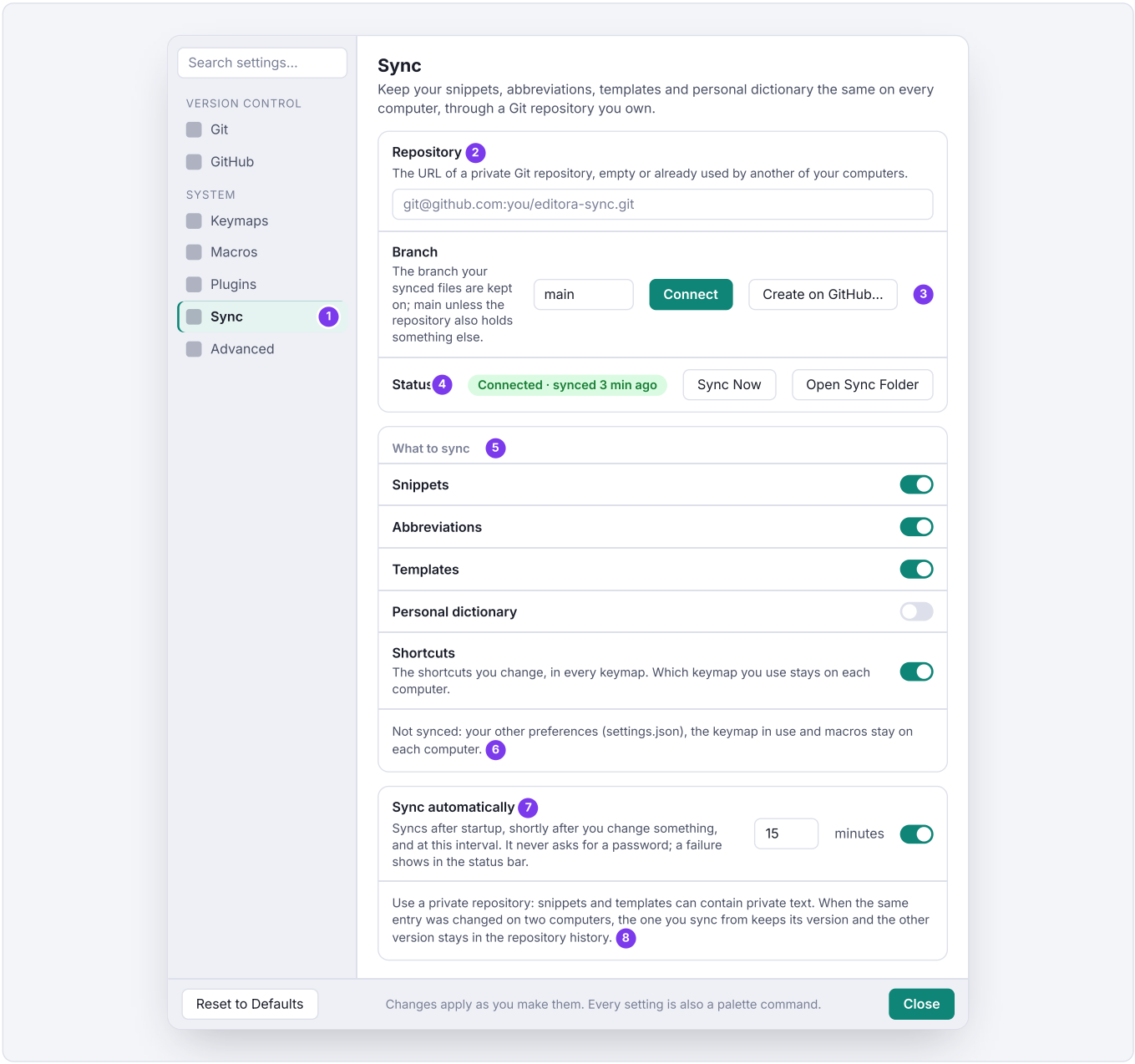


Editora.

Settings Sync

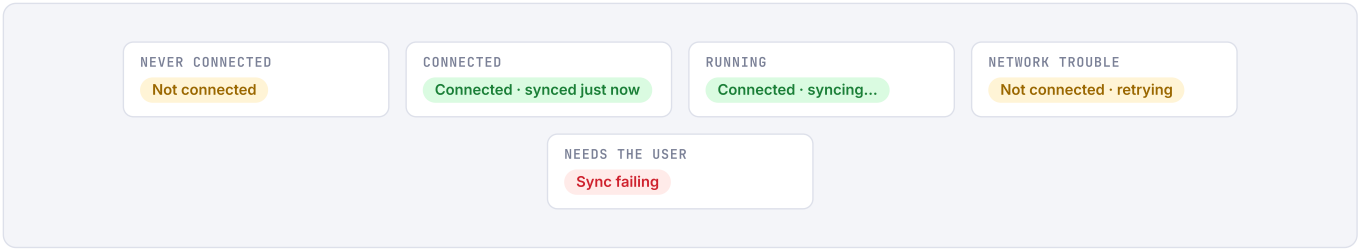
`SHOULD` in every app with user data worth carrying between computers (snippets, templates, shortcuts, profiles, saved views). A page in the Settings window, found by searching "sync". No server and no account of ours: the store is a private Git repository the user owns, reached through their own Git credentials. Reference: Editora's `sync/` package (`SyncEngine` , `SyncMerge` , `SyncAccount`) and `ui/SettingsSync.java` , with the page in `SettingsWindow` ; Torrenta and Termina carry the same package.

Anatomy



1. **Its own page** in the Settings sidebar, under System, found by the Settings search ("sync", "git", "github").
2. **Repository URL.** SSH or HTTPS. A URL with a password or token in it is refused, with the reason and the fix (a credential helper such as `gh auth setup-git` , or an SSH URL).
3. **Branch, Connect, Create on GitHub….** Connect clones or adopts the repository. Create on GitHub makes a private repository through the `gh` CLI and fills the URL in.
4. **Status pill** plus Sync Now and Open Sync Folder (see the states below).
5. **What to sync:** one switch per kind of data, each with a description when its scope is not obvious.
6. **What is not synced**, said plainly: machine-local preferences, the keymap in use, anything with paths.
7. **Automatic sync** with its interval in minutes (default 15, at least 1). Automatic runs never prompt.
8. **Privacy and conflicts**, in one note: use a private repository; on a same-entry conflict the computer you sync from wins, and the other version stays in the history.

Status pill states



Green, amber and red are the Settings status-pill colors. Amber means "will recover by itself" (offline, the remote unreachable); red means "the user must act" (Git missing, the repository written by a newer version, a sync that would remove most of a kind of data and waits for confirmation). The pill's accessible text is "Sync status: ...".

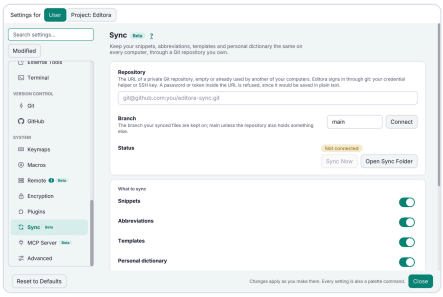
Spec

Property	Value	Reference
Store	A Git repository the user owns, cloned into `<config>/sync/`; one folder per kind of data	`SyncEngine`, `SyncTarget`
Sign-in	The user's own Git credentials (credential helper or SSH key); the GitHub account that can read the repository is pinned per clone	`SyncAccount`
Automatic runs	After startup, 30 s after a local change, and every *n* minutes (default 15)	`SettingsSync.DIRTY_DELAY_SECONDS`, `syncIntervalMinutes`
Conflicts	Per entry; this computer wins; the other version stays in history	`SyncMerge`
Safety	Files a sync replaces are copied to the sync folder first; a sync that would remove most of a kind of data stops and asks	`SyncReport`
One runner	Sync runs in the first instance only; a second instance says so	`status.sync.secondaryInstance`
Failures	To the status bar and the Message Log, said once per failure, not once per interval	`SettingsSync`
Palette	Settings Sync: Set Up..., Sync Now, and Toggle Sync Automatically	`sync.setup`, `sync.now`, `sync.toggleAuto`

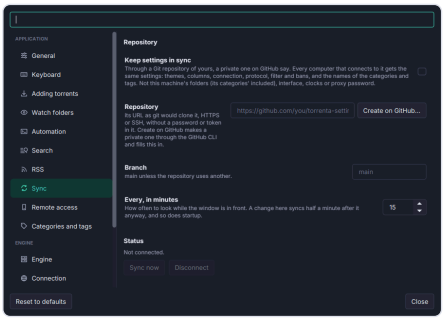
Copy

Key	English
`settings.sync.subtitle`	Keep your snippets, abbreviations, templates and personal dictionary the same on every computer, through a Git repository you own.
`settings.sync.connect` / `settings.sync.createGithub`	Connect / Create on GitHub...
`settings.sync.now` / `settings.sync.openFolder`	Sync Now / Open Sync Folder
`settings.sync.pill.*`	Connected · Not connected · Sync failing · syncing... · retrying · synced {0} min ago
`status.sync.notSetUp`	Settings sync is not connected. Set it up in Settings ▸ Sync.

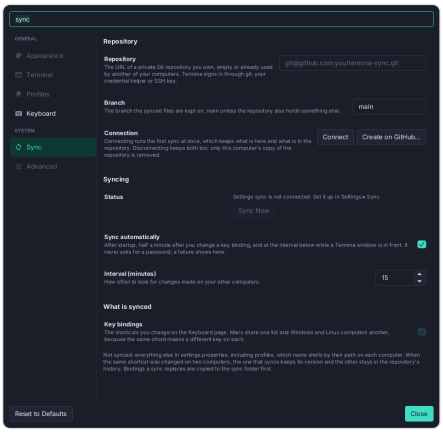
In the apps



Editora.



Torrenta.



Termina.

Packaging, signing and release

Targets

Each target builds on its own GitHub-hosted runner. jpackage and JavaFX natives are host-specific, so there is no cross-building.

Target	Runner	Downloads
Linux x64	`ubuntu-latest`	`.deb`, `.rpm`, `.AppImage`, `.tar.gz`, fat jar
Linux arm64	`ubuntu-24.04-arm`	`.tar.gz` (and `.deb` where the app's tooling allows), fat jar
macOS arm64	`macos-15`	`.dmg`
macOS x64	`macos-15-intel`	`.dmg` (the last Intel image, good to about Aug 2027)
Windows x64	`windows-latest`	`.msi`, fat jar
Windows arm64	—	**Not possible yet**: OpenJFX publishes no `win-aarch64` natives (JDK-8314064). ARM users run the x64 MSI under emulation.

- Downloads are named `<App>-<version>-<target>.<ext>`.
- jpackage builds the installers through the Maven `dist` profile. **JReleaser** (`jreleaser.yml`) only creates the GitHub release: it attaches the files, `checksums_sha256.txt` and the changelog section.
- The maven version is the single source of truth: `-SNAPSHOT` on `master`, a tag `vX.Y.Z` releases, a post-release job bumps to the next snapshot (Editora's `bump` job in `release.yml` opens that pull request).
- GitHub releases are immutable, so a **release-asset check** fails the run before publishing when an expected file is missing (Editora's "Verify the expected release assets" step in `release.yml`).
- **Linux desktop integration**: a `.desktop` file whose `StartupWMClass` is the JavaFX app class, plus AppStream metainfo `io.github.adriandeleon.<App>` (Torrenta's `packaging/linux/` holds both; Editora's `packaging/linux/Editora.desktop` has the `StartupWMClass`, but its metainfo id predates the rule).
- **AOT cache** (JDK 25 Leyden): a training run during packaging cuts start-up time; Linux legs run it under `xvfb` (Editora's `scripts/aot_build.java`).

Install methods

Channel	Platform	What the user runs	How it is published	Level
GitHub release	All	Download the `.dmg`, `.msi`, `.deb`, `.rpm`, `.AppImage` or `.tar.gz`	JReleaser attaches every file, with checksums	`MUST`
Homebrew cask	macOS (Apple Silicon)	`brew install --cask adriandeleon/tap/<app>`	JReleaser's `brew` packager writes the cask to `adriandeleon/homebrew-tap`	`SHOULD`
winget	Windows x64	`winget install AdrianDeLeon.<App>`	JReleaser's `winget` packager opens a pull request on `microsoft/winget-pkgs` from the `adriandeleon` fork	`SHOULD`

- **One tap for the family.** The official homebrew-cask repository has notability rules a new project does not meet, so the casks live in a personal tap. A cask takes exactly one artifact: it serves Apple Silicon, and Intel Macs use the `macos-x64` DMG.
- **winget is reviewed by Microsoft.** The first submission for a new publisher is checked by a person, not merged automatically. The package identifier is `AdrianDeLeon.<App>`, the moniker the app's lower-case name.
- **Both need a `PACKAGES_TOKEN` secret**, a classic personal access token with `repo` scope: the workflow's own token cannot write to the tap or open a pull request on the fork. Without it JReleaser skips both packagers and the GitHub release still publishes.
- `jreleaser package --basedir .` writes the cask and the winget manifests locally, to check them without releasing.
- **Status on 2026-11-06:** the tap exists and holds no casks yet. Torrenta and Termina have both packagers configured, waiting for the token. Mirada's first winget submission (microsoft/winget-pkgs#471599) is open.

Signing

Platform	How	Status in the family
macOS `.dmg`	Developer ID Application certificate (team 8JKSF797L8), hardened runtime with per-app entitlements, `notarytool` submission, stapled ticket, `spctl` check, then a smoke launch of the notarized app. Secrets per repo: `MACOS_CERT_P12`, `MACOS_CERT_PASSWORD`, `APPLE_API_KEY_P8`, `APPLE_API_KEY_ID`, `APPLE_API_ISSUER`. A tag without the secrets fails.	**Done** in all six
Linux `.deb` `.rpm` `.AppImage` `.tar.gz`	One shared OpenPGP release key ("Adrian De Leon (release signing)"). A separate `sign-linux` job: `rpmsign` for `.rpm`, a debsigs origin signature for `.deb`, and an armoured `.asc` beside every file. Public key committed as `packaging/linux/signing-key.asc` and attached to each release. Secrets `LINUX_GPG_KEY`, `LINUX_GPG_PASSPHRASE`. Shared scripts `sign-linux-packages.sh` and `check-linux-signatures.sh`.	Editora, Torrenta, Termina, Modula; Mirada partly; nux not yet

Platform	How	Status in the family
Windows `.msi`	Authenticode. **Open decision**: SignPath Foundation (free for OSS), Certum Open Source cert, or Azure Trusted Signing. Until then MSIs are unsigned and SmartScreen warns.	**Not done** in any app
All	GitHub **build provenance attestations** (`actions/attest-build-provenance`) for every download; `gh attestation verify` checks them.	Editora, Termina

- Signing steps skip cleanly when the secrets are absent, so a fork or a dry run still builds.
- The certificate renews before 2027-11-30; the Apple membership before 2027-10-31.

CI/CD (GitHub Actions)

Workflow	When	What	Level
`ci.yml`	Every push to `master` and every pull request	**Format** (`spotless:check`) and **Build & Test** (`./mvnw verify` , coverage gate included) on JDK 25. Both are required status checks for `master` .	`MUST`
`full-tests.yml`	Daily (cron), on a pull request labeled `full-ci` , and before every release	The full suite on Linux, macOS and Windows; flaky tests listed in the job summary.	`SHOULD`
`release.yml`	A `v*` tag that matches the pom version; `workflow_dispatch` as a dry run	The five-target matrix, signing jobs, attestations, the release-asset check, JReleaser, then the bump to the next snapshot. A dry run builds and signs and publishes nothing.	`MUST`

Every workflow:

- Pins each third-party action to a commit SHA, with its version in a comment (`actions/setup-java@b843ebf… # v6.1.0`). Dependabot updates `github-actions` and `maven` weekly.
- Starts from `permissions: contents: read` ; only a job that publishes asks for write access, in its own block.
- Sets up Temurin JDK 25 with `actions/setup-java` and `cache: maven` .
- Puts `timeout-minutes` on every job, and uses `concurrency` to cancel a superseded pull-request run, never a run on `master` .
- Uploads the surefire reports when tests fail and the JaCoCo report always; writes the coverage totals to the job summary (Editora's "Summarise coverage" step in `ci.yml`).
- Skips signing cleanly when the secrets are absent (forks, dry runs); a tag without them fails (Editora's "Import signing certificate (macOS)" step checks `MACOS_CERT_P12` first).
- Launches the packaged app once on each OS before a release (a smoke test): Torrenta's "Smoke-test the packaged app" step runs the launcher in every target's image. Editora runs only the notarized macOS app (`--version`).

Feature matrix

Surveyed on 2026-11-05 from each repository's default branch (the rows added in editions 1.2.0, 1.3.0 and 1.4.0 on 2026-11-06). ● has it, ◐ partly or differently, — not found.

What the apps are.

App	What it is	Homepage	Source
Editora	Programmer's text editor (the reference)	editora-project.dev	adriandeleon/Editora
Torrenta	BitTorrent client	torrenta-project.dev	adriandeleon/torrenta
Termina	Terminal emulator	termina-project.dev	adriandeleon/Termina
Modula	Software-defined-radio receiver	—	adriandeleon/Modula
Mirada	Fast image viewer (GraalVM native image)	mirada-project.dev	adriandeleon/Mirada
nux	UPS monitor for Network UPS Tools, in the tray	—	adriandeleon/nux

Platform and repository

	Editora	Torrenta	Termina	Modula	Mirada	nux
JDK 25+ and JavaFX 27+, JPMS module, `mvnw`	●	●	●	●	●	●
AtlantaFX 3	●	●	◐ vendored CSS	● Primer	● Cupertino	● Primer
Inter UI font `SHOULD`	●	●	●	○ Plex Mono	○	○
NullAway + every package `@NullMarked`	●	●	●	●	●	○
Spotless (Palantir)	●	●	●	●	●	○
JaCoCo gate	● 0.90 L/M	● 0.90 L	◐ per-package	○	○	○
Headless FX tests	●	●	●	○	●	●
Six or more message catalogs	●	●	●	○	○	●
README badges + screenshot	●	●	●	●	◐ no shot	●
Keep a Changelog format	●	◐	◐	●	◐	●
MIT `LICENSE` + `NOTICE`	●	●	●	●	●	●
`AGENTS.md` canonical, `CLAUDE.md` a pointer	●	◐ `CLAUDE.md` only	●	◐ `CLAUDE.md` only	●	◐ copy
`CONTRIBUTING.md`	●	○	○	○	○	○
`docs/` and `branding/`	●	●	●	●	●	●
`docs/README.md` indexing the guides	●	○	○	○	○	○
User docs on the website (`/docs/`)	● 43 pages, versioned	● 19 pages	● 16 pages	○ README only	● 11 pages	○ README only
Help → Documentation command	●	○	○	○	○	○
`bug_report.yml` issue form	●	●	○	○	○	○
Dependabot	●	●	○	●	○	●

Release and signing

	Editora	Torrenta	Termina	Modula	Mirada	nux
JReleaser	●	●	●	○	○	○
Five-target matrix (Linux x64/arm64, macOS arm64/x64, Windows x64)	●	◐	●	◐ no Linux arm64	●	●

	Editora	Torrenta	Termina	Modula	Mirada	nux
macOS signed + notarized `.dmg`	●	●	●	●	●	●
Linux OpenPGP `.asc` signatures	●	●	●	●	◐ rpm + checksums	○
Windows `.msi` Authenticode	○	○	○	○	○	○
Build provenance attestations	●	○	●	○	○	○
Homebrew cask / winget manifests	○	●	●	○	◐ winget	○
`.desktop` file + AppStream metainfo	●	●	●	●	◐ no metainfo	◐ no metainfo
AOT start-up training	●	○	●	○	●	○

User-facing features

	Editora	Torrenta	Termina	Modula	
Command registry + Command Palette (in-scene overlay)	●	●	●	●	
QuickOpen-style pickers for lists	●	○ palette only	○ palette only	○ palette only	●
Find bar: in-scene, incremental, count, Esc returns focus	● find and replace	◐ filter box	● floating card	○	○
Keymap presets	● 9	○	○	○	
Keybinding editor / shortcuts reference	●	●	●	○	
Menu rows carry the command's icon	●	●	●	○	
Chord shown beside each menu row	● own column	◐ native accelerator	○	●	
Right-click menu: one at a time, any outside press closes it	● every window	◐ stock auto-hide	◐ terminal view	◐ radio pane	◐ sto
Icon-only buttons carry an accessible name	● `Icons.name`	●	○ tooltip only	○ tooltip only	◐ b
Toolbar tooltips show the command's chord	●	●	○ no toolbar	○	
Tab strip: unsaved and pinned states, drag to reorder, reopen closed	●	○	◐ reorder and reopen, no pin	○	
Panels toggled by commands, placement remembered	● stripes, dock, float	● sidebar and detail pane	◐ panes in a tab	○	◐ info
Settings window	●	●	●	●	
Settings search	●	●	●	○	
Settings sync (Git / GitHub via `gh`)	●	●	●	○	
Message Log (status-bar history)	●	●	○	○	
Debug Log + uncaught exception handler	●	●	●	○	
Update notification (GitHub releases)	●	●	●	●	
Report a Problem → GitHub issue (+ `--report`)	●	●	○	○	
About window with runtime versions	●	●	●	●	

	Editora	Torrenta	Termina	Modula	
Welcome page	●	●	○	○	
Confirm dialog: named red action button, Escape cancels, or undo instead	●	● `RemoveConfirm` + undo	◐ asks, button not red	○ presets remove on one click	◐ trash is un
Doctor / diagnosis	●	◐ per torrent	○	◐ native libs	
Single instance (hand off to running app)	●	●	○	○	
`--config-dir` and other CLI flags	●	●	●	○	
Light / dark / follow system, live switch	●	●	●	●	
Caret & Ink theme pair (Editora Light / Dark)	● authored	● vendored	● vendored	○	○
Icons as themed SVG paths, one style class per family	● two families	● two families	● filled, menus	◐ transport glyphs	● hand
State colors from theme roles, contrast-tested	● `StyleContrastTest`	◐ roles, no test	◐ roles; terminal schemes tested	○ fixed hexes	◐ ac
Extended title bar (`HeaderBar` , `StageStyle.EXTENDED`)	●	◐ opt-in	●	○	
Desktop notifications	●	●	●	●	
Tray icon (SNI over D-Bus, AWT fallback)	○	●	○	●	
Close to tray, only while an icon is up	○	●	○	●	
Start at login (XDG, LaunchAgent, `HKCU` Run) and `--minimized`	○	●	○	○	
Window geometry restore	●	●	●	○	
Config export / import	●	○	○	○	
Plugins	●	○	○	○	

Shared features

Beyond the list this guide started from, these already exist in two or more apps and belong in the guideline:

1. **`AppInfo` + `build-info.properties`.** Version, build time and commit come from a Maven-filtered resource, read by About, `--version`, the update check and problem reports. *(all six)*
2. **Shared `update/` package** (`UpdateCheck` pure parser, `UpdateService`, `ReleaseInfo`). *(5 apps)*
3. **Pure palette core shared**: `FuzzyMatch`, `MatchText`, `KeyChord`, `PaletteKeys` and the caret-key helper (`TextInputKeymap` in Editora, `TextInputKeys` in the copies). Emacs-style caret keys work inside the palette's query field. *(Editora → Termina → Mirada; Torrenta takes `MatchText` only)*
4. **Shortcuts reference window**: a searchable list of every key, opened from Help. *(Termina, Mirada)*
5. **Extended title bar** with JavaFX 27 `HeaderBar` and `StageStyle.EXTENDED`, falling back to a decorated window; the macOS system menu bar via `useSystemMenuBar`. *(Editora, Termina, Mirada, Torrenta opt-in)*
6. **macOS app-menu integration**: About and Settings in the application menu, `openFiles` Apple Events reaching the running app (`MacOpenFiles` / `MacHandlers`). *(Editora, Mirada, Torrenta)*
7. **Tray package** (`AwtTray`, `SniTray`, `StatusNotifierItem` over dbus-java) plus **autostart** (XDG autostart, LaunchAgent, `HKCU\…\Run`) and `--minimized`. *(Torrenta, Modula, nux)*
8. **Desktop notifications** through one `Notifier`: `notify-send` / `gdbus` on Linux, `osascript` on macOS, a PowerShell toast on Windows, the tray as fallback. *(Torrenta, Termina, Modula, nux, Editora)*
9. **Window geometry that stays on screen**: `WindowGeometry` / `WindowPlacement` restore size and position but move a window back when its monitor is gone. *(Editora, Torrenta, Termina, Mirada)*
10. **Linux desktop integration**: a `.desktop` file whose `StartupWMClass` matches the JavaFX app class (`com.<app>.App`), AppStream metainfo `io.github.adriandeleon.<App>`, multi-size PNG icons.
11. **Package-manager channels**: Homebrew cask and winget manifests generated by JReleaser. *(Torrenta, Termina, Mirada)*
12. **Atomic config writes** with a file lock, and values clamped to a valid range when read. *(Termina, nux, Mirada)*
13. **Dev capture / screenshot mode** (`DevCapture`, nux `--screenshot=`): renders the real UI headless at 2× for README, website and bug-report images. *(Torrenta, Termina, nux; Editora uses an external headless harness)*
14. **Smoke launch in CI**: the packaged app is started and checked (still running after 20 s, or `--version`) on every OS before release. *(all six)*
15. **Release dry run**: `workflow_dispatch` builds and signs with throwaway credentials and publishes nothing. *(Editora, Modula, Mirada, nux, Termina)*
16. `SECURITY.md` with how to report a vulnerability privately. *(Modula; worth adding everywhere)*
17. **Simple mode / density**: fewer menus and a compact layout for new users. *(Editora, Torrenta)*
18. **Zoom levels** stored per window kind, `Ctrl/Cmd +/-/0`. *(Editora, Termina)*
19. **Undo for destructive list actions** (remove a torrent, close a tab) and **reopen closed tab**. *(Torrenta `UndoStack`, Termina and Editora `ClosedTabs`)*

Gaps per app

App	Missing MUSTs
Editora	Windows MSI signing.
Torrenta	Help → Documentation; `docs/README.md`; Keep-a-Changelog format; `AGENTS.md` (only `CLAUDE.md` is committed); `CONTRIBUTING.md`; Windows signing; attestations.
Termina	Help → Documentation; `docs/README.md`; Message Log; Report a Problem; single instance; `bug_report.yml`; `CONTRIBUTING.md`; full coverage gate; Windows signing.
Modula	User docs on a website; Help → Documentation; `docs/README.md`; JaCoCo; headless FX tests; i18n; Settings search and sync; Message/Debug Log; Report a Problem; `AGENTS.md`; `CONTRIBUTING.md`; `bug_report.yml`; JReleaser; Linux arm64.
Mirada	Help → Documentation; `docs/README.md`; JaCoCo; i18n; Settings search and sync; Message/Debug Log; uncaught handler; update check; Report a Problem; `CONTRIBUTING.md`; `bug_report.yml`; JReleaser.
nux	User docs on a website; Help → Documentation; `docs/README.md`; NullAway; Spotless; JaCoCo; Settings search and sync; Message/Debug Log; Report a Problem; `CONTRIBUTING.md`; `bug_report.yml`; Linux signatures; JReleaser.